

Prozessoptimierung unter Concept Drift: Adaptive Simulationsmodelle im industriellen Umfeld

Dem Fachbereich Maschinenbau und Verfahrenstechnik
der Rheinland-Pfälzischen Technischen Universität Kaiserslautern Landau
zur Erlangung des akademischen Grades

Doktor-Ingenieur (Dr.-Ing.)

vorgelegte

Dissertation

von

Herrn

M.sc. Pascal Marijan

aus Trier

2026

Betreuer der Dissertation: Prof. Dr.-Ing. Martin Ruskowski

Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbständig verfasst und ohne unerlaubte, fremde Hilfe angefertigt habe. Alle Ausführungen, die aus anderen Schriften wörtlich oder sinngemäß übernommen wurden, sind kenntlich gemacht. Alle verwendeten Quellen sind im Literaturverzeichnis zitiert.

Mannheim, den 26. Juli 2026

PASCAL MARIJAN

Vorwort / Danksagung

Hier kommt das Vorwort oder die Danksagung hin

Weiterführung von Vorwort / Danksagung oder ...

Widmung

Inhaltsverzeichnis

Vorwort / Danksagung	III
Kurzfassung	VI
Abkürzungsverzeichnis	VII
Symbolverzeichnis	VIII
1 Einleitung	1
1.1 Motivation	1
1.2 Zielsetzung	2
2 Grundlagen	4
2.1 Vertikalrollenmühlen	4
2.1.1 Funktionsweise	4
2.1.2 Stellgrößen und Betriebspunkte	6
2.1.3 GPpro	8
2.2 Prozessmodellierung	9
2.2.1 Systemidentifikation	9
2.2.2 Modellarchitekturen	11
2.2.3 Datenselektion	14
2.2.4 Modellvergleich	16
2.3 Concept Drift	21
2.3.1 Terminologie	21
2.3.2 Drift-Erkennung	24
2.3.3 Drift-Adaption	28
3 Generalisierung	31
3.1 Synthetischer Datensatz	32
3.1.1 Concept Drift	33
3.1.2 Setpoints	35
3.1.3 Signal-Inputs	37
3.1.4 Rauschen	38
3.1.5 Systemantwort	39

3.1.6	Modellierung	41
3.1.7	Drift-Erkennung	46
3.1.8	Drift-Adaption	49
3.2	Der Tennessee-Eastman-Process	55
3.2.1	Concept Drift	57
3.2.2	Generierung des Datenstroms	59
3.2.3	Modellierung	59
3.2.4	Drift-Erkennung	59
3.2.5	Drift-Adaption	59
4	Kapitel 4	60
5	Kapitel 5	61
	Anhang	62
A	Appendix A	62
B	Appendix B	62

Kurzfassung

Die Zementindustrie ist gegenwärtig für rund 7 % der globalen CO₂-Emissionen verantwortlich, wobei ein erheblicher Teil des elektrischen Energiebedarfs auf die Vermahlung entfällt. Vor diesem Hintergrund bietet die Betriebspunktoptimierung von Mahlanlagen ein signifikantes Potenzial zur Reduktion der spezifischen Energie, also der benötigten Energie pro Tonne produzierten Materials. Die Grundlage solcher Optimierungen bilden Simulationsmodelle, deren Performance jedoch im zeitlichen Verlauf aufgrund von Einflüssen wie Materialvariationen, Verschleiß und Wartungen beeinträchtigt wird. Im Rahmen der Untersuchung kann dieses Phänomen, aufgrund der Art der zugrunde liegenden Verschiebung der Datengrundlage als Real Concept Drift klassifiziert werden.

Durch die künstliche Simulation von Concept Drift wird das identifizierte Problem anhand des Wiener Modells generalisiert und zusätzlich auf den Benchmark-Datensatz des Tennessee Eastman Process übertragen, der für diesen Zweck modifiziert wird.

Kern der Arbeit ist die Untersuchung von Adaption der Modelle an veränderliche Bedingungen im industriellen Umfeld unter Verwendung neuronaler Netze, um die langfristige Simulationsgüte sicherzustellen. Dabei wird das Stability-Plasticity-Dilemma adressiert, also die Herausforderung des Modells, gleichzeitig neues Wissen aufzunehmen und vorhandenes Wissen zu erhalten.

Die Ergebnisse zeigen eine signifikante Verbesserung der Modellperformance durch die Adaption an veränderte Prozessbedingungen. Die vorliegende Arbeit liefert damit einen Beitrag zur Schaffung robuster maschineller Lernmodelle für die industrielle Praxis und zeigt messbare Vorteile für die Energieeffizienz vergleichbarer Prozesse auf.

Abkürzungsverzeichnis

ADWIN	Adaptive Windowing
CCR	Leitstand (Central Control Room)
CD	Concept Drift
EKF	erweiterter Kalman-Filter (Extended Kalman Filter)
KF	Kalman-Filter
LSTM	Long Short-Term Memory
NN	neuronales Netz
TEP	Tennessee Eastman Process

Symbolverzeichnis

Lateinische Buchstaben

$c[k]$	Driftsignal
$\bar{d}$	mittlerer Detektionsverzug
$\mathcal{D}$	Datensatz bzw. Datenfenster
e	Simulationsfehler (Residuum)
f	Dynamikteil: Zustandsübergang bzw. LTI-Impulsantwort
g	Ausgangsabbildung des Modells
h	Beobachtungsmodell des KF/EKF
J	Zielfunktion der Hyperparameter-Optimierung
K	Zeithorizont des Datenstroms
K_p	statische Verstärkung des PT ₁ -Gliedes
m	harmonisches Mittel der Fensterlängen
$p(\cdot)$	Wahrscheinlichkeitsdichte
T	Zeitkonstante des PT ₁ -Gliedes
u	Systemeingang
v	Zwischensignal des Wiener-Modells
w	Setpoint-Zeitreihe
W	ADWIN-Datenfenster
x	Systemzustand
y	Systemausgang

Griechische Buchstaben

α_i	Gewichtungskoeffizienten des Wiener-Modells
γ_τ	Verschiebungsanteil der Segmentgrenzen

δ, δ'	globales bzw. angepasstes Signifikanzniveau
η_u, η_y	injiziertes Rauschen am Ein- bzw. Ausgang
ζ_p, ζ_m	Prozess- bzw. Messrauschen des KF/EKF
θ	Modellparameter
κ, κ_w	Driftkoeffizient am Ausgang bzw. Setpoint
λ	Schwellenwert
$\hat{\mu}_W$	empirischer Fenstermittelwert (ADWIN)
ν_j	normiertes Gewicht der Rauschkomponente
ρ	Recall der Sudden Drifts
σ	Standardabweichung (Streuung)
τ_i	innere Segmentgrenzen
φ	Regressionsvektor des KF/EKF
ω_*	Gewichte der Zielfunktion

Indizes und Zählgrößen

n_*	Dimension, Ordnung oder Länge
N_*	Anzahl
weitere	k (Zeitschritt), i, j (Laufindizes), d (Detektion), w (Setpoint), p/m (Prozess/Messung), s (Abtastung), a (Adaption), $\min/\max$ (Extrema)

Operatoren, Akzente und Superskripte

$(\hat{\cdot})$	Schätzwert bzw. identifiziertes Modell
$(\bar{\cdot})$	zeitlicher Mittelwert
$(\tilde{\cdot})$	drift-behaftete Größe
$(\cdot)^o$	rauschfreie Größe
$(\cdot)^f$	gefilterte (geglättete) Größe
$(\cdot)^T$	Transponierte
$ \cdot $	Betrag

1 Einleitung

TODO Kurze Einleitung dazu, was in dem Kapitel beschrieben wird.

1.1 Motivation

Im Kontext des Klimawandels, insbesondere im Industriesektor, erfährt der CO_2 Fußabdruck eine zunehmend zentralere Bedeutung. In diesem Kontext ist die Zementindustrie derzeit für rund 7 % der globalen CO_2 Emissionen verantwortlich [In18]. Angesichts der Tatsache, dass die globale Nachfrage nach Zement weiterhin wächst, besteht ein großes Interesse an einer Reduktion der Emissionen in diesem Sektor. Wie in Abbildung 1.1 am Beispiel von Deutschland dargestellt, können die Emissionen in der Zementindustrie in prozessbedingt und energiebedingt unterteilt werden. Ein signifikanter Anteil der prozessbedingten Emissionen ist auf die Freisetzung des CO_2 zurückzuführen, welches im Hauptbestandteil Kalkstein gebunden ist und während des Kalzinierungsprozesses freigesetzt wird. Eine Optimierung kann hier durch die Substitution des Kalksteins und des Klinkeranteils, beispielsweise durch das Beimischen von Recyclingmaterial, erzielt werden. Neben der Menge an CO_2 , die durch den Kalzinierungsprozess für die Klinkerproduktion und durch die Wärmeerzeugung im Drehrohrofen emittiert wird, wird eine erhebliche Menge an elektrischer Energie benötigt, die dem energiebedingten Anteil zugeordnet wird [SB21]. Dabei werden etwa 60 % der elektrischen Energie für die Rohmaterialaufbereitung und die Klinkervermahlung benötigt [Af15]. So übersteigt die Leistung einer großen Vertikalrollenmühle 10 Megawatt. Dies verdeutlicht das große Optimierungspotenzial innerhalb des Mahlprozesses. Ein vielversprechendes Optimierungspotenzial bietet die Betriebspunktoptimierung. Dabei ist das Ziel die Parameter des Mahlbetriebs so zu wählen, dass die Vermahlung mit reduziertem Energieverbrauch erfolgt, ohne jedoch den Durchsatz oder die Qualität des Produkts zu beeinflussen. Konkret geht es dabei um die Verbesserung der spezifischen Energie, also der Energie pro Tonne Material. Diese Art der Optimierung gelingt mithilfe eines Modells, welches die Grundlage der Arbeit bildet und dessen Entwicklung einen essenziellen Teil der Untersuchung darstellt. Zur Bewertung des Modells kann der Modellfehler herangezogen werden, der die Differenz von Modellsimulation und Realwert beschreibt. Bei der Betrachtung des Modellfehlers der spezifischen Energie wie dargestellt in Abbildung 1.2 fällt auf, dass die Performance des Modells im zeitlichen Verlauf variiert. Dieses Phänomen lässt sich durch Concept Drift (CD) erklären und ist Motivation der Arbeit.

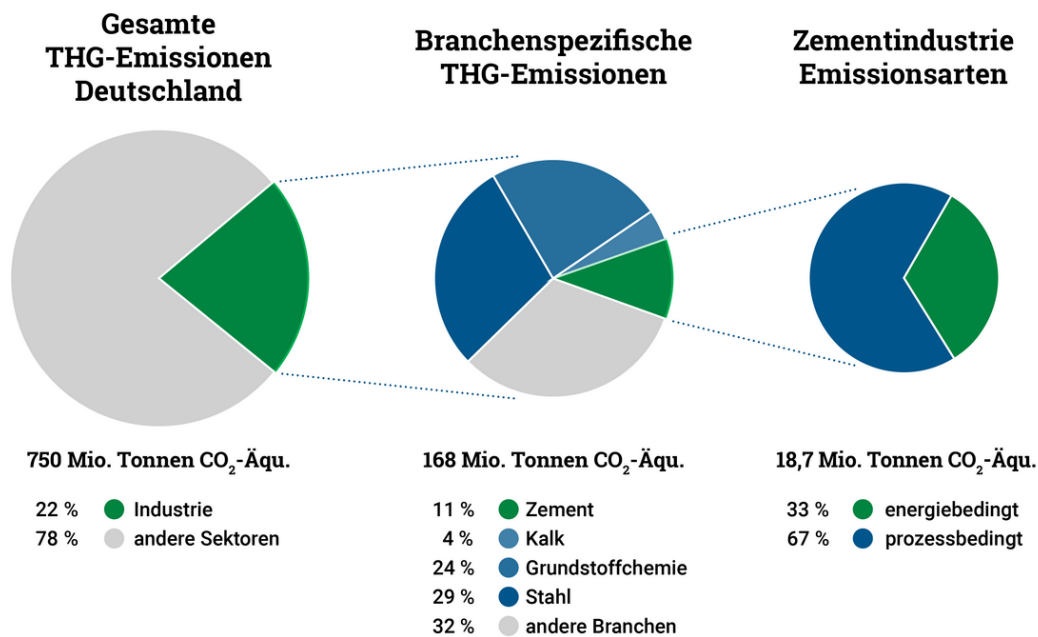


Abbildung 1.1: Treibhausgase (THG) der deutschen Industrie 2022. Grafik entnommen aus [Ko26]

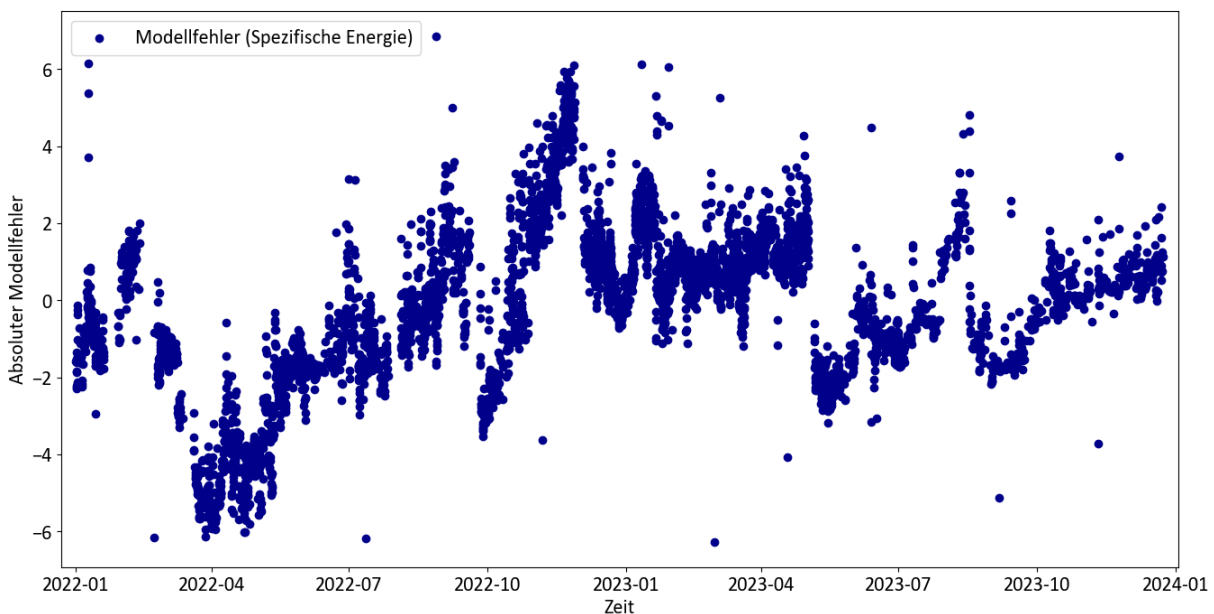


Abbildung 1.2: Scatterplot des absoluten Modellfehlers der spezifischen Energie über zwei Jahre

1.2 Zielsetzung

Motiviert durch die Problemstellung aus soll die Dissertation durch die folgenden Forschungsleitfragen strukturiert werden:

1. Wie kann eine effektive Adaption gegen Concept Drift entwickelt und in industriellen Mahlanlagen implementiert werden, um die Genauigkeit von Prognose- und Klassifika-

tionsmodellen trotz sich verändernder Datenbedingungen langfristig zu gewährleisten? Welche spezifischen Methoden und Techniken sind dabei geeignet, um robuste maschinelle Lernmodelle in diesem Anwendungsbereich zu schaffen?

2. Wie kann der entwickelte Ansatz zur Bekämpfung von Concept Drift so generalisiert werden, dass er auf eine breite Palette industrieller Anwendungen anwendbar ist, während gleichzeitig die spezifischen Anforderungen und Herausforderungen von Mahlanlagen berücksichtigt werden? Welche Schritte und Validierungsmethoden sind erforderlich, um die Verallgemeinerbarkeit des Ansatzes systematisch anhand dieses Beispiels zu überprüfen?
3. Welche messbaren Vorteile und Verbesserungen lassen sich durch die Implementierung eines adaptiven Ansatzes gegen Concept Drift in industriellen Mahlanlagen nachweisen, und wie können diese Ergebnisse genutzt werden, um Empfehlungen für die Optimierung vergleichbarer industrieller Prozesse abzuleiten?

2 Grundlagen

Die modellbasierte Betriebspunktoptimierung einer Vertikalrollenmühle erfordert ein grundlegendes Verständnis des zu optimierenden Prozesses, der eingesetzten Modellierungsverfahren sowie der Herausforderungen, die sich aus dem zeitvarianten Charakter des Prozesses ergeben. Dieses Kapitel legt die dafür notwendigen theoretischen Grundlagen und stellt den Stand der Technik dar. In Abschnitt 2.1 wird zunächst die Vertikalrollenmühle in ihrer technischen und systemischen Beschreibung eingeführt. Darauf aufbauend behandelt Abschnitt 2.2 die datengetriebene Prozessmodellierung, von der Systemidentifikation über relevante Modellarchitekturen bis hin zur Modellauswahl für die Betriebspunktoptimierung. Abschließend werden in Abschnitt 2.3 die Grundlagen zu CD erarbeitet, der als zentrales Phänomen die langfristige Modellgüte im realen Betrieb beeinflusst.

2.1 Vertikalrollenmühlen

Zunächst werden Aufbau und Funktionsweise der Vertikalrollenmühle erläutert sowie Stellgrößen und Betriebspunkte formal beschrieben. Ergänzend wird GPLink/GPpro als Erweiterung der Mühle zu einem cyberphysischen System dargestellt.

2.1.1 Funktionsweise

Vereinfacht gesprochen setzt sich Zement aus Kalkstein und Ton zusammen. Zur Zementherstellung erfolgt nach dem Abtragen der Gesteine zunächst eine Zerkleinerung mit Brecheranlagen, bevor das Gemisch zu Rohmehl vermahlen wird. Beim Brennen des Rohmehls im Drehrohrofen entsteht so genannter Zementklinker, der erneut vermahlen wird. Bei dem dabei entstehenden Produkt handelt es sich um das Endprodukt, Zement. In Abbildung 2.1 sind die beschriebenen Prozessschritte der Zementherstellung schematisch dargestellt. Die Mühlen sind dabei farblich in Gelb zu erkennen. Neben der Rohmehlmühle (a) und der Zementmühle (c) ist auch eine Kohlemühle (b) dargestellt, die oftmals bei kohlebefeuertem Drehrohrofen eingesetzt wird. Im Laufe der Zeit haben sich zur Vermahlung mehrere Verfahren etabliert. Neben Kugelmühlen werden hauptsächlich Vertikalrollenmühlen (VRM) eingesetzt, die aufgrund ihres hohen Wirkungsgrads als Stand der Technik gelten [Sc01]. Dennoch gibt es auch bei VRM ein erhebliches Potenzial zur Energieoptimierung. So kann beispielsweise eine modulare Bauweise

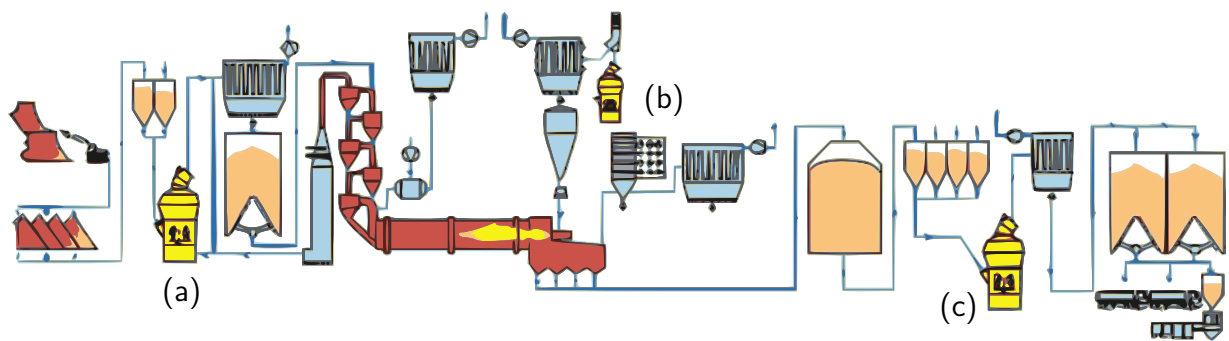


Abbildung 2.1: Schematische Abbildung der konsekutiven Prozessschritte in der Zementherstellung, Abbildung von Gebr. Pfeiffer SE

der Mühle den Prozess durch eine Reduzierung der Stillstandzeiten optimieren [Wo17]. Neben der Zementindustrie kommen VRM auch in der Kohleindustrie zum Einsatz. Zu den gängigen Mahlgütern zählen Kalkstein, Kohle, Ton, Gips und Schlacke. Das Hauptziel des Mahlvorgangs besteht darin, das Eingangsmaterial auf ein gewünschtes Maß zu zerkleinern. Während die Größe des Produkts erst nach dem Prozess gemessen werden kann, werden mehrere Sensoren eingesetzt, um den Zustand des Systems zu überwachen, sodass die resultierende Partikelgröße geschätzt werden kann. Der prinzipielle Aufbau einer VRM ist in Abbildung 2.2 schematisch dargestellt. Grundsätzlich besteht eine VRM aus einem rotierenden Mahlteller, auf dem die Walzen durch ihr Eigengewicht nach unten gedrückt werden. Häufig wird die Kraft in Richtung Mahlteller durch ein hydraulisches System unterstützt. Die Verschleißteile, einschließlich der Walzen und der Oberfläche des Mahltellers, bestehen in der Regel aus austauschbaren Teilen aus Hartgusslegierungen wie NiHard, die für ihre extreme Härte bekannt sind. Das Aufgabematerial wird von oben durch die Schurre über einen Trichter in die Mitte des Mahltellers eingeführt. Durch die Zentrifugalkräfte wird das Material auf dem Mahlteller zum Rand bewegt, wo es von den Walzen erfasst und zerkleinert wird. So entsteht ein Gemisch aus grobem und feinem Mahlgut. Dieses Gemisch bildet ein so genanntes „Mahlbett“ auf dem Mahlteller, das einen direkten Kontakt zwischen den Walzen und dem Tisch verhindert. Ein durch einen Ventilator erzeugter Luftstrom tritt oberhalb des Antriebs am äußeren Rand des Mahltellers ein und transportiert das feine Material nach oben durch die Mahlkammer. Der Siebtrichter oberhalb der Schurre trennt dann das Grobgut vom Feingut. Das feine Material verlässt die Mühle, während das grobe Material über den Trichter zurück auf den Mahlteller fällt. Material, das über den Rand des Mahltellers fällt, wird über ein externes Zirkulationssystem wieder in die Mühle zurückgeführt.

Bei der in dieser Arbeit betrachteten Mühle handelt es sich um eine VRM vom Typ MVR 2500 C-4 der Firma Gebr. Pfeiffer SE. Diese MVR wird für die Zerkleinerung von Hochofenschlacke verwendet, einem Nebenprodukt der Stahlindustrie, mit dem ein Anteil des Klinkers im Zement substituiert werden kann. Mit vier gleich großen Mahlwalzen und einem Mahltellerdurchmesser von 2,5 m ist die MVR 2500 C-4 für Durchsatzleistungen von bis zu 30 t/h ausgelegt und wird von einem 750-kW-Elektromotor angetrieben.

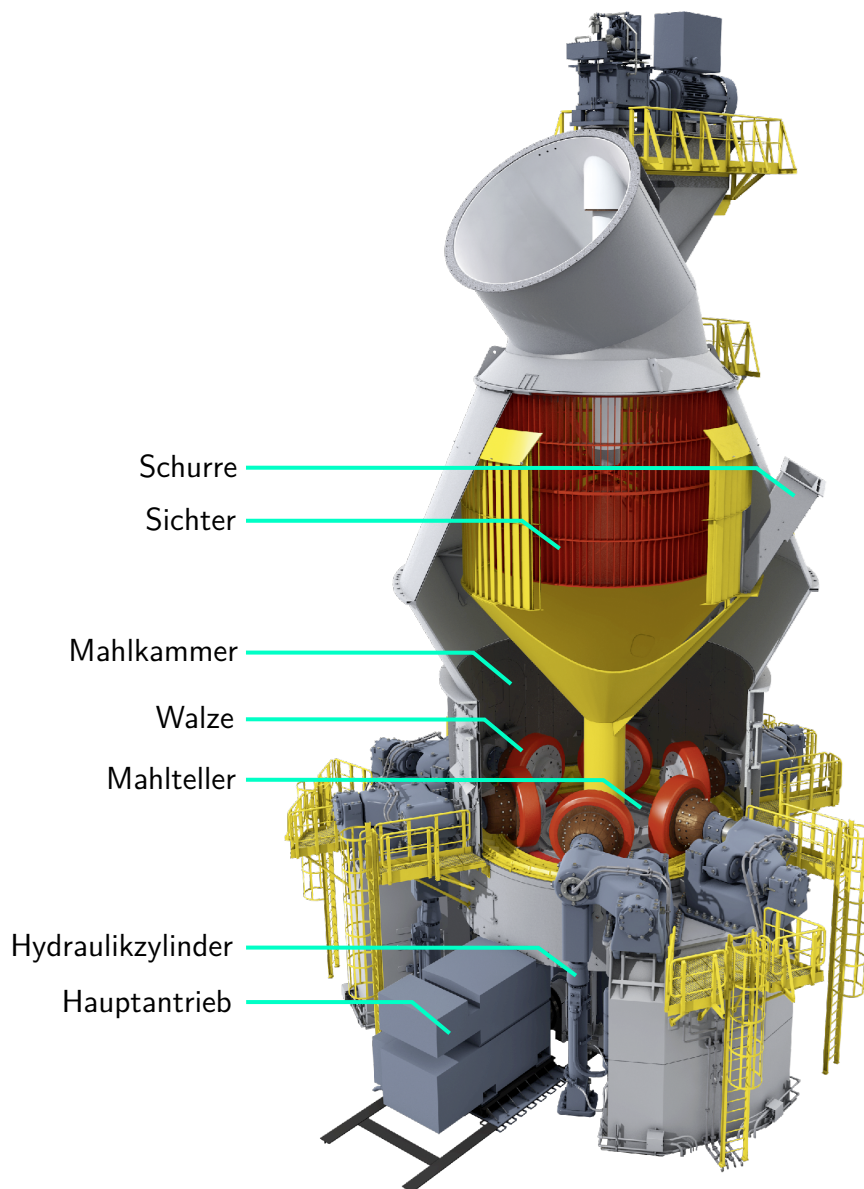


Abbildung 2.2: Schematische Abbildung einer VRM vom Typ MVR der Firma Gebr. Pfeiffer SE

2.1.2 Stellgrößen und Betriebspunkte

Zum Betrieb der Mühle stehen dem Betreiber, im Folgenden als Operator bezeichnet, mehrere Stellgrößen zur Verfügung, mit denen der Prozess gezielt beeinflusst werden kann. Dabei legen die Operatoren die Sollwerte, die so genannten Setpoints, der jeweiligen Aggregate fest, die anschließend durch eine speicherprogrammierbare Steuerung (SPS) geregelt werden. Eine konkrete Kombination aller eingestellten Stellgrößen zu einem bestimmten Zeitpunkt wird als Betriebspunkt bezeichnet. Jeder Betriebspunkt beschreibt somit eine eindeutig definierte Konfiguration der Anlage, die sich durch die Werte aller relevanten Stellgrößen charakterisieren lässt. Formal lässt sich ein Betriebspunkt als Vektor

$$w = [w_1, w_2, \dots, w_n]^T \in \mathbb{R}^n \quad (2.1)$$

auffassen, wobei die einzelnen Komponenten w_i die jeweiligen Stellgrößen der Mühle repräsentieren. Zu diesen Stellgrößen zählen unter anderem:

- **Die Aufgabemenge**

Abhängig von Mühlengröße und Produkt können die Mengen des Edukts oder der Edukte über mehrere Förderbänder variiert werden. Die Messung erfolgt mittels Bandwagen.

- **Die Mahltellergeschwindigkeit**

Sofern der Hauptantrieb mit einem Frequenzumrichter ausgestattet ist, kann die Drehzahl des Mahltellers variiert werden. Aus wirtschaftlichen Gründen wird bei einigen Anlagen jedoch auf den Einsatz eines Frequenzumrichters verzichtet.

- **Die Sichtergergeschwindigkeit**

Über die Sichtergergeschwindigkeit wird die Drehzahl des Sicherterrades geregelt. Mit zunehmender Drehzahl wird der Abscheidegrad erhöht, sodass weniger grobes Material mit dem Luftstrom zu den Filtern transportiert wird.

- **Die Ventilatorgergeschwindigkeit**

Die Ventilatorgergeschwindigkeit beeinflusst den Luftvolumenstrom innerhalb der Anlage. In manchen Anlagen wird der Luftvolumenstrom direkt als Stellgröße durch den Operator vorgegeben.

- **Der Spanndruck**

Der Spanndruck bezeichnet den hydraulischen Druck, mit dem die Walzen auf den Mahlteller wirken. Aufgrund von Leckagen im Hydrauliksystem weist der Spanndruck eine charakteristische Hysterese auf, wobei das Hydraulikaggregat bei Unterschreiten definierter Grenzwerte automatisch nachregelt.

- **Die Heißgastemperatur**

Der in der Regel mit fossilen Brennstoffen betriebene Heißgaserzeuger ist für die Einstellung der Temperatur des Luftvolumenstroms verantwortlich.

- **Die Klappenstellungen**

Die Frischluft- und Rezirkulationsklappen dienen insbesondere der Abführung von Feuchtigkeit und Wärme aus dem System und beeinflussen damit sowohl den thermischen als auch den strömungstechnischen Zustand der Mühle.

Da jede Stellgröße innerhalb eines definierten Wertebereichs variiert werden kann, spannt die Gesamtheit aller Stellgrößen einen hochdimensionalen Stellgrößenraum $\mathcal{W} = \{w \in \mathbb{R}^n\}$ auf. Entsprechend existiert eine sehr große Anzahl möglicher Betriebspunkte, in denen die Mühle betrieben werden kann. Die Identifikation geeigneter Betriebspunkte innerhalb dieses hochdimensionalen Raums stellt eine zentrale Herausforderung für den effizienten Betrieb der Mühle dar und soll durch modellbasierte Betriebspunktoptimierung adressiert werden.

2.1.3 GPpro

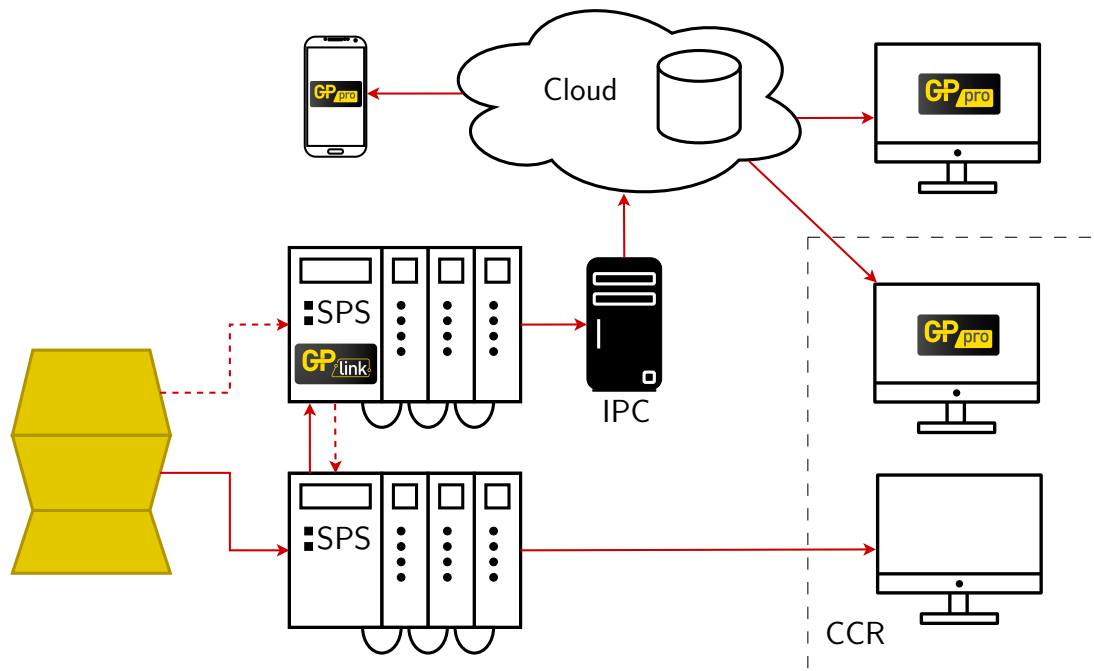


Abbildung 2.3: Schematische Abbildung der Datenverarbeitung mit GPlink/GPpro

GPlink/GPpro bezeichnet das Digitalisierungsprogramm der Firma Gebr. Pfeiffer SE, mit dem die Mahlanlagen zu cyberphysischen Systemen erweitert werden. Cyberphysische Systeme zeichnen sich dadurch aus, dass sie physikalische Prozesse eng mit rechnergestützten Verfahren zur Datenerfassung, -verarbeitung und -visualisierung koppeln und diese über Kommunikationsnetze verfügbar machen [Le08]. Die Architektur von GPlink/GPpro umfasst Hardware- und Softwarekomponenten; die verwendeten Bezeichnungen sind historisch gewachsen und daher nicht ganz trennscharf. In dieser Arbeit wird vereinfachend die Hardware-Infrastruktur als GPlink und die darauf aufbauenden Software- und Webdienste als GPpro bezeichnet. Die prinzipielle Funktionsweise des Systems ist in Abbildung 2.3 schematisch dargestellt. Die Sensordaten der Mahlanlage werden zunächst konventionell über SPS des Kunden erfasst. Ergänzend hierzu ist eine zusätzliche SPS installiert, die so genannte GPlink-SPS. Diese übernimmt zwei zentrale Aufgaben: Zum einen empfängt sie ausgewählte Prozesssignale aus der Kunden-SPS mit einer Frequenz von 1 Hz, zum anderen stellt sie zusätzliche Messdaten aus einer erweiterten Sensorik bereit, die Bestandteil von GPlink ist, und überträgt diese an die Kunden-SPS zurück. Auf diese Weise wird die bestehende Automatisierungsstruktur erweitert, ohne sie grundlegend zu verändern. Die von der GPlink-SPS erfassten Daten werden über eine OPC-UA-Schnittstelle an einen Industrie-PC (IPC) übertragen. Der IPC wiederum fungiert als Bindeglied zwischen der lokalen Automatisierungsebene und der Cloud-Infrastruktur. Die Daten werden so an cloudbasierte Dienste weitergeleitet, wo sie gespeichert und weiterverarbeitet werden. In der Cloud sind mehrere, mittels Docker containerisierte Dienste implementiert. Dazu zählen unter anderem *Kafka* zur Datenübertragung, *InfluxDB* zur Speicherung der Zeitreihendaten, *PostgreSQL* zur Ablage verarbeiteter Daten, *Grafana* zur Visualisierung sowie *Python*-basierte Dienste zur Datenana-

lyse. Die vom IPC gesendeten Prozessdaten werden über Kafka in die Cloud übertragen und in einer Influx-Datenbank gespeichert. Auf dieser Datenbasis werden mit Python Berechnungen, Analysen und Auswertungen durchgeführt, deren Ergebnisse anschließend in einer PostgreSQL-Datenbank gespeichert werden. Sowohl Roh- als auch verarbeitete Daten werden über ein webbasiertes Frontend bereitgestellt und mit Grafana visualisiert. Über die Cloud-Infrastruktur ist ein weltweiter Zugriff möglich. Die Operatoren und weitere Stakeholder erhalten dadurch ortsunabhängig Einblick in relevante Prozessgrößen, Kennzahlen und Auswertungen. Für die Operatoren im Leitstand (Central Control Room) (CCR) stehen die Prozessdaten somit auf zwei Wegen zur Verfügung: Zum einen weiterhin direkt über die Visualisierung der Kunden-SPS, zum anderen ergänzend über GPpro mit erweiterten Analyse-, Historisierungs- und Visualisierungsfunktionen. GPLink/GPpro erweitert damit die klassische Automatisierungsebene um eine digitale Service- und Analyseplattform und bildet eine Grundlage für datengetriebene Prozessüberwachung und -optimierung.

2.2 Prozessmodellierung

Aufbauend auf der Beschreibung der Mühle wird im Folgenden die Prozessmodellierung eingeführt, beginnend mit grundlegenden Aspekten der Systemidentifikation, einer kompakten Einführung in neuronale Netze und abschließend der Modellauswahl zur Betriebspunktoptimierung.

2.2.1 Systemidentifikation

Der Vermahlungsprozess lässt sich, wie viele ingenieurwissenschaftliche Prozesse, als dynamisches System beschreiben. Obwohl reale Systeme ihrem Kern nach zeitkontinuierlich sind, wird für deren Systemidentifikation auf zeitdiskrete Modelle zurückgegriffen. Der Grund dafür ist der zeitdiskrete Charakter der Sensoren, die zur Datenerfassung verwendet werden. In der Systemidentifikation können Systeme anhand verschiedener Eigenschaften klassifiziert werden [Ke11]. Dazu zählen:

- die Kausalität,
- der Prozesscharakter (deterministisch oder stochastisch),
- die Zeit(in)varianz,
- die (Nicht-)Linearität,
- die Dynamik.

Grundsätzlich kann ein System entweder durch ein Zustandsraummodell oder ein Input-Output-Modell beschrieben werden. Ausgehend von einem Datensatz $\mathcal{D} = \{(u[k], y[k])\}_{k=0}^{N-1}$ aus N

gemessenen Input-Output-Paaren kann das System S durch folgende Zustandsraumdarstellung beschrieben werden:

$$\begin{aligned} x[k+1] &= f(x[k], u[k]) \\ y^o[k] &= g(x[k]) \end{aligned} \quad (2.2)$$

wobei $x[k] \in \mathbb{R}^{n_x}$ der Zustand des Systems, $y^o[k] \in \mathbb{R}^{n_y}$ der rauschfreie Output und $u[k] \in \mathbb{R}^{n_u}$ der Input jeweils zum Zeitschritt k ist. Weiterhin sind $f(\cdot)$ und $g(\cdot)$ die Zustands- bzw. Output-Mappings [Ke11]. Der gemessene Output $y[k]$ wird durch ein mittelwertfreies, farbiges oder weißes Rauschen $\eta[k]$ verfälscht, d. h. $y[k] = y^o[k] + \eta[k]$. Dieses Zustandsraummodell kann entweder „klassisch“ nach Linearisierung in den Frequenzbereich überführt oder in der „modernen“ Identifikation direkt im Zeitbereich betrachtet werden.

Alternativ kann statt des Zustandsraumansatzes eine Input-Output-Modellstruktur verwendet werden. Der durch ein Modell geschätzte Output sei dabei mit $\hat{y}[k]$ bezeichnet. Modelle, die sowohl die gemessenen Inputs als auch die gemessenen Outputs zur Vorhersage verwenden, werden als Prädiktionsmodelle bezeichnet:

$$\hat{y}[k] = g(y[k-1], \dots, y[k-n_a], u[k-1], \dots, u[k-n_b]; \theta) \quad (2.3)$$

wobei n_a bzw. n_b die Länge der jeweils berücksichtigten Historien festlegen und $g(\cdot)$ das durch $\theta \in \mathbb{R}^{n_\theta}$ parametrisierte Output-Mapping ist [FP20]. Im Gegensatz dazu werden Modelle, bei denen ausschließlich die gemessenen Inputs auf den Output gemappt werden, Simulationsmodelle genannt [FP20]. Ein solcher Fall liegt vor, wenn der Output aus einer endlichen Historie vergangener Inputs berechnet wird:

$$\hat{y}[k] = g(u[k-1], \dots, u[k-n_b]; \theta) \quad (2.4)$$

Wird darüber hinaus die Dynamik des Systems im Modell vernachlässigt, spricht man von einem statischen Modell. Der Zustand x - und damit das Gedächtnis des Systems, über das vergangene Inputs nachwirken - wird dabei vernachlässigt, sodass der Output ausschließlich vom aktuellen Input abhängt:

$$\hat{y}[k] = g(u[k]; \theta) \quad (2.5)$$

wobei $u[k]$ den System-Input zum Zeitpunkt k darstellt. Die Abweichung $e[k] = y[k] - \hat{y}[k]$ zwischen Messung und Modelloutput beschreibt den (Simulations-)Fehler. Ist $g(\cdot)$ linear in den Parametern θ , spricht man von linearer Regression; andernfalls ist $g(\cdot)$ ein nichtlineares Mapping, das den Input mit dem Output in Beziehung setzt.

Zur Identifikation des in Abschnitt 2.1 beschriebenen Vermahlungsprozesses kann zunächst festgehalten werden, dass es sich um einen kausalen Prozess handelt, da alle relevanten Ausgangsgrößen ausschließlich von aktuellen und vergangenen Eingangsgrößen abhängen und keine zukünftigen Informationen benötigen. Obwohl die zugrunde liegenden physikalischen Gesetzmäßigkeiten prinzipiell deterministisch sind, wird das reale Anlagenverhalten durch zufällige Einflüsse wie schwankende Materialeigenschaften, Umwelteinflüsse sowie Prozess- und Messrauschen geprägt, sodass der Prozess praktisch stochastischen Charakter besitzt. Zudem ist der Prozess

zeitvariant, da sich die Systemdynamik infolge von Verschleiß, Materialwechseln, Wartungsmaßnahmen und veränderten Betriebsbedingungen kontinuierlich verändert. Weiterhin weist die Vermahlung eine ausgeprägte Nichtlinearität auf, die sich unter anderem in betriebspunkt-abhängigem Verhalten und Wechselwirkungen zwischen Materialfluss, Druck, Temperatur und Energieeintrag äußert. Schließlich handelt es sich um einen dynamischen Prozess, da interne Speicher- und Verzögerungseffekte, insbesondere durch das Mahlbett und thermische Trägheiten, dazu führen, dass der aktuelle Prozesszustand wesentlich von der Systemhistorie abhängt. Der Vermahlungsprozess ist folglich als kausaler, stochastischer, zeitvarianter, nichtlinearer und dynamischer Prozess zu klassifizieren.

Eine physikalisch motivierte Zustandsraumbeschreibung dieses Systems würde eine detaillierte Abbildung der zugrunde liegenden mechanischen, thermischen und materialabhängigen Wechselwirkungen erfordern. Aufgrund der hohen Prozesskomplexität sowie der Vielzahl nur schwer oder gar nicht messbarer Zustandsgrößen ist die Herleitung eines hinreichend genauen physikalischen Modells jedoch mit erheblichem Aufwand und Unsicherheiten verbunden. Bisherige Versuche, ein solches Modell zu entwickeln, führten zu keinem Erfolg. Ein datenbasierter Input-Output-Modellierungsansatz hingegen ermöglicht es, das dynamische Prozessverhalten direkt aus gemessenen Ein- und Ausgangsdaten zu identifizieren. Dieser Ansatz eignet sich insbesondere für komplexe industrielle Prozesse mit zeitvarianten stochastischen Einflüssen und erlaubt eine Modellierung des Systems ohne Kenntnis der internen Zustände. Als Modellstruktur dieses Ansatzes wurde das neuronale Netz gewählt, welches eine flexible Approximation komplexer, nichtlinearer Zusammenhänge erlaubt.

2.2.2 Modellarchitekturen

Durch die prominente Präsenz großer Sprachmodelle (Large Language Models, LLMs) im öffentlichen Diskurs finden die Begriffe „Künstliche Intelligenz“, „Machine Learning“ und „neuronale Netze“ zunehmend Verwendung im allgemeinen Sprachgebrauch. Diese Begriffe bezeichnen jedoch eine Vielzahl unterschiedlicher Modellklassen, deren inhaltliche Breite durch eine Reduktion auf LLMs nicht angemessen abgebildet wird. Gleichzeitig ist im industriellen Kontext zu beobachten, dass zum Teil auch vergleichsweise einfache oder etablierte algorithmische Ansätze unter diesen Begriffen subsumiert werden, was zu einer begrifflichen Verwässerung beiträgt und die fachliche Trennschärfe weiter reduziert. Um dieser Unschärfe entgegenzuwirken, sollen im Folgenden die für diese Arbeit relevanten Modellstrukturen eingeführt und voneinander abgegrenzt werden.

Zunächst werden dazu Verfahren betrachtet, die nicht in die Kategorie der neuronalen Netze (NN) fallen, aber durch ihre Einfachheit auszeichnen und häufig als Baseline dienen.

Dabei sei als erstes die Lineare Regression (LR) aufgeführt, die den Zusammenhang zwischen einer abhängigen Zielgröße und einer oder mehreren unabhängigen Inputvariablen durch eine

lineare Gleichung beschreibt. Sie ist leicht interpretierbar, bei komplexeren, nicht-linearen Zusammenhängen stößt sie jedoch an ihre Grenzen und kann hier lediglich zur Approximation dienen.

Die Lasso-Regression erweitert dieses Konzept der LR um einen Regularisierungsterm [Ti96]. Dieser bestraft große Modellgewichte und kann diese auf Null setzen, was nicht nur Overfitting entgegenwirkt, sondern auch zu einer automatischen Feature-Auswahl führt.

Ein weiteres Verfahren mit linearer Modellannahme ist der Kalman-Filter (KF), ein rekursives Zustandsschätzverfahren auf Basis eines Zustandsraummodells gemäß (2.2) [Ka60]. Der KF ist ein rekursives Schätzverfahren, das den verborgenen internen Zustand eines linearen dynamischen Systems aus einer Reihe von verrauschten Messungen ermittelt. Der Algorithmus arbeitet dabei in zwei iterativen Schritten:

- **Prädiktionsschritt**

Schätzung des Systemzustands für den folgenden Zeitschritt basierend auf einer Zustandsgleichung des mathematischen Systemmodells

- **Korrekturschritt**

Abgleichen der Schätzung mit der tatsächlichen Messung über die Messgleichung

Unter der Annahme, dass das System linear ist und das Rauschen einer Normalverteilung unterliegt, liefert der Kalman-Filter so theoretisch die mathematisch optimale Zustandsschätzung.

Beim erweiterter Kalman-Filter (Extended Kalman Filter) (EKF) handelt es sich um eine Erweiterung des KF, der das Konzept in nicht-lineare Zustandsraum-Modelle überführt. Dazu werden System- und Messgleichungen im jeweiligen Arbeitspunkt linearisiert [WB+95]. Dies geschieht durch eine Taylor-Reihenentwicklung erster Ordnung. Durch die lokale Linearisierung kann die Prädiktions- und Korrekturlogik des KF durch den EKF auf komplexere, nicht-lineare Problemstellungen angewendet werden.

Während die zuvor genannten Verfahren entweder auf linearen Annahmen beruhen oder explizite mathematische Systemmodelle erfordern, bieten NN einen rein datengetriebenen Ansatz zur Modellierung komplexer, nicht-linearer Zusammenhänge. Grundsätzlich lassen sich maschinelle Lernverfahren auf verschiedene Art und Weise kategorisieren. Eine fundamentale Unterscheidung beschreibt die zwischen überwachtem Lernen (engl. Supervised Learning) und unüberwachtem Lernen (engl. Unsupervised Learning) [BN06]. Beim unüberwachten Lernen erhält das Modell ausschließlich Input-Daten (engl. *Features*), ohne dass eine zugehörige Zielgröße (engl. *Label*) vorgegeben ist. Durch das Modell können so intrinsische Strukturen in den Daten analysiert oder Muster und Korrelationen identifiziert werden.

Demgegenüber basiert das Modelltraining beim überwachten Lernen auf einem Datensatz, der aus Features und den dazugehörigen, bekannten Labels besteht. Man spricht auch von gelabelten Daten. Das Ziel ist es, eine Funktion zu erlernen, die den Fehler zwischen der Mo-

dellvorhersage und dem tatsächlichen Zielwert minimiert, um präzise Vorhersagen für neue, ungesehene Daten treffen zu können. Die vorliegende Arbeit beschränkt sich auf überwachtes Lernen, da die betrachtete Problemstellung eine Regressionsaufgabe darstellt und die betrachteten Systemgrößen sensorisch gemessen und somit als gelabelte Daten für das Modelltraining verfügbar sind.

Die grundlegendste Form eines künstlichen neuronalen Netzes ist das Multi-Layer Perceptron (MLP), welches zur Klasse der Feedforward Neural Networks gehört [Go16]. Der Informationsfluss in einem MLP ist streng gerichtet und verläuft von der Eingabe zur Ausgabe, ohne Rückkopplungen. Die Architektur gliedert sich in drei Hauptkomponenten, die jeweils aus einem oder mehreren so genannten Neuronen bestehen:

- **Input Layer** (Eingabeschicht)
Erfasst die Input-Daten u
- **Hidden Layers** (Verborgene Schichten)
Jedes Neuron dieser Schichten ist mit allen Neuronen der vorherigen Schicht verbunden, berechnet eine gewichtete Summe von deren Outputs und transformiert diese mit einer vorgegebenen Aktivierungsfunktion.
- **Output Layer** (Ausgabeschicht)
Gibt die Prädiktion des Outputs $\hat{y}$ zurück. Bei Regressionsaufgaben besteht sie typischerweise aus einem einzelnen Neuron mit linearer Aktivierungsfunktion.

Sowohl die Anzahl der Neuronen in den Layern als auch die Wahl der Aktivierungsfunktionen sind variabel und können als Hyperparameter des Modells spezifiziert werden. Das Training erfolgt durch den Backpropagation-Algorithmus [RHW86]. Dabei wird nach einer Vorhersage über eine Verlustfunktion der Unterschied zwischen dem tatsächlichen Output y und dem prädizierten Output $\hat{y}$ berechnet. Anschließend wird der Fehler rückwärts durch das Netz propagiert. Mithilfe von gradientenbasierten Optimierungsverfahren (engl. gradient descent) werden die Gewichte iterativ angepasst, um so den Gesamtfehler zu minimieren [Go16]. Klassischerweise ist das MLP ein statisches Modell. Um es für die Regression von Zeitreihendaten nutzbar zu machen und Dynamiken zu erfassen, kann ein Sliding-Window-Ansatz verwendet werden [BBL12]. Dabei wird ein Datenfenster festgelegter Breite aus vergangenen Zeitschritten zu einem Vektor transformiert und dem MLP als statische Eingabe übergeben. So kann das Modell historische Abhängigkeiten in seine Vorhersage einbeziehen.

Dem MLP gegenüber stehen neuronale Architekturen, die zeitliche Abhängigkeiten nicht ausschließlich über die Eingangsrepräsentation, sondern auch über ihre interne Struktur erfassen. Convolutional Neural Networks (CNN) wurden ursprünglich für die Bilderkennung entwickelt [Le02]. Sie nutzen Faltungsschichten (engl. Convolutional Layers), in denen Filter über die Eingabedaten gleiten, um lokale Features zu extrahieren, wobei die Gewichte geteilt werden. Obwohl ursprünglich für 2-dimensionale Bilddaten konzipiert, lassen sich CNNs auf Zeitreihen-

daten anwenden. Dabei werden 1D-CNNs eingesetzt, bei denen ein Filter entlang der Zeitachse gleitet [Ki21]. Dadurch erkennt und extrahiert das Modell automatisch lokale zeitliche Muster und Trends innerhalb der Zeitreihe.

Für die Modellierung komplexer, langfristiger zeitlicher Abhängigkeiten finden typischerweise rekurrente neuronale Netze Anwendung. Eine besonders leistungsfähige Form ist das Long Short-Term Memory (LSTM) [HS97]. Im Gegensatz zu Feedforward-Netzen besitzen LSTMs Rückkopplungsschleifen, die einen internen Zustand über die Zeit aufrechterhalten. Durch eine Zellstruktur von so genannten Input-, Output- und Forget-Gates steuert das LSTM gezielt, welche Informationen behalten und welche vergessen werden. Dadurch sind LSTMs prädestiniert für Zeitreihendaten, da sie die Dynamik der Zeitreihen nativ abbilden.

2.2.3 Datenselektion

Bei der Modellierung realer industrieller Prozesse spielen die erfassten experimentellen Daten eine essenzielle Rolle, da sie in datengetriebenen Ansätzen die primäre Informationsquelle darstellen. Gleichzeitig ist die Datenerhebung in Produktionsumgebungen typischerweise durch technische Randbedingungen erschwert, sodass eines der zentralen Ziele darin besteht, die Datengrundlage so zu gestalten, dass der Informationsgehalt maximiert wird. Für den vorliegenden Anwendungsfall der Mühle ist zudem zu berücksichtigen, dass der eigentliche Materialzerkleinerungsprozess nicht direkt messbar ist. Stattdessen müssen indirekte Indikatoren herangezogen werden, die das Prozessverhalten über messbare Betriebsparameter beschreiben.

Die in dieser Arbeit betrachtete Zielgröße ist der spezifische Energiebedarf, also der Gesamtenergiebedarf pro Tonne produzierten Produkts, der hier auf die drei größten Hauptaggregate: Hauptantrieb, Ventilator und Sieb beschränkt ist. Um aus der Vielzahl verfügbarer Sensorsignale diejenigen Größen zu bestimmen, die einen signifikanten Einfluss auf diese Zielgröße besitzen, stützt sich die Sensorauswahl auf eine Kombination aus ingenieurwissenschaftlichem Fachwissen sowie datengetriebenen Analyseverfahren wie Principal Component Analysis und Mutual-Information-Analyse. Die resultierende Menge relevanter Stell- und Messgrößen entspricht den in Abschnitt 2.1.2 eingeführten Parametern. Dabei werden die Stellgrößen als exogene Setpoints durch die Operatoren vorgegeben, während die resultierenden Prozesssignale sensorisch erfasst und als Messgrößen in das Modell eingehen. Mit der Identifikation der für die Zielgröße relevanten Stellgrößen stellt sich die Frage nach der geeigneten Modellstruktur zur Beschreibung des Prozesses.

Wie bereits einleitend in Abschnitt 1.1 beschrieben, liegt das Ziel der Modellierung des Mahlprozesses darin, den Prozess durch Betriebspunktoptimierung zu verbessern. Dazu sollen in einem gegebenen Betriebspunkt die partiellen Ableitungen der Zielgröße nach den Eingangsgrößen bestimmt werden, um diejenigen Stellgrößen zu identifizieren, die lokal den größten Einfluss auf die Zielgröße ausüben. Auf Basis dieser Information kann anschließend eine Anpassung der

entsprechenden Stellgrößen im realen Prozess erfolgen. Vor diesem Hintergrund stellt sich die Frage, welche Modellarchitektur mit Blick auf den Informationsgehalt der verfügbaren Daten für diese Aufgabe am geeignetsten ist. Einerseits liegen Zeitreihendaten vor, deren sequenzieller Charakter prinzipiell dynamische Information enthält. Andererseits ist die Betriebspunktoptimierung in erster Linie an eingeschwungenen, quasi-stationären Zuständen interessiert, sodass eine Reduktion auf stationäre Betriebspunkte denkbar ist. Die Modellkomplexität soll daher empirisch überprüft werden.

In Abschnitt 2.2.1 wurde der Mahlprozess als dynamischer Prozess identifiziert. Außerdem wurde das NN als datenbasierter Input-Output-Modellierungsansatz des Prozesses gewählt. Die Entscheidung für ein NN als Modellstruktur lässt jedoch offen, ob zur Abbildung des Prozesses ein dynamisches Modell gemäß (2.4) oder ein statisches Modell gemäß (2.5) verwendet werden soll. Während statische Input-Output-Modelle den Systemausgang ausschließlich als Funktion der aktuellen Eingangsgrößen beschreiben, berücksichtigen dynamische Input-Output-Modelle zusätzlich die zeitliche Abhängigkeit vergangener Ein- und Ausgangswerte. Damit weisen statische Modelle aufgrund der fehlenden zeitlichen Rekursion grundsätzlich eine geringere strukturelle Komplexität auf als dynamische Modelle [Ne20]. Vor diesem Hintergrund stellt sich die zentrale Frage, welche Modellkomplexität für den vorliegenden Prozess unter den gegebenen Informationsbedingungen adäquat ist. Die verfügbaren Messdaten mit einer Abtastrate von 1 Hz bilden sowohl eingeschwungene, quasi-stationäre Betriebszustände als auch Übergänge zwischen diesen ab, die als transient bezeichnet werden. Die Daten im quasi-stationären Zustand schwanken, bedingt durch Messrauschen und stochastische Prozesseinflüsse, um einen Mittelwert. In diesem Fall liegt ein klassisches Regressionsproblem vor, bei dem die zeitliche Reihenfolge der Datenpunkte vernachlässigt wird. Demgegenüber besteht bei transienten Zuständen die Erwartung, dass sich zusätzliche Information aus der zeitlichen Abfolge der Messwerte ergibt, da das dynamische Verhalten des Systems infolge einer Stellgrößenänderung sichtbar wird. Daraus ergibt sich eine Aufteilung der Datengrundlage in zwei Datensätze.

Für den Datensatz der quasi-stationären Zustände ist es essenziell, einen möglichst breiten und gleichmäßig verteilten Datenraum abzudecken, um robuste Aussagen über den stationären Zusammenhang zwischen Ein- und Ausgangsgrößen zu ermöglichen. Zu diesem Zweck kann statistische Versuchsplanung (Design Of Experiment, DOE) herangezogen werden. Konkret findet dazu der so genannte Latin-Hypercube-Ansatz Anwendung, bei dem Stichproben aus einem Parameterraum so gezogen werden, dass jeder Parameter über seinen definierten Wertebereich gleichmäßig verteilt ist [MBC00]. Innerhalb der minimalen und maximalen Grenzen aller betrachteten Stellgrößen wird so ein zufällig verteilter Datenraum mit einer vorgegebenen Anzahl von Stichproben erzeugt. Die resultierenden Betriebspunkte werden anschließend hinsichtlich ihrer technischen Umsetzbarkeit geprüft und gefiltert. Im Rahmen eines Feldexperiments an der in Abschnitt 2.1.1 beschriebenen Mühle konnten auf diese Weise insgesamt 18 Betriebspunkte realisiert werden, welche den Datenraum des regulären Produktionsbetriebs

signifikant erweitern. Die Anforderung an einen stationären Prozesszustand bestand bei dem Experiment darin, dass nach der letzten Änderung einer Stellgröße alle Regelgrößen für einen Zeitraum von mindestens zehn Minuten unverändert bleiben mussten.

Für den zweiten Datensatz, der die transienten Prozesszustände abbilden soll, reicht die Anzahl der im Experiment erzeugten Übergänge zwischen stationären Betriebspunkten nicht aus. Daher ist eine Ergänzung des Datensatzes durch Ereignisse aus dem laufenden Produktionsbetrieb erforderlich. Um sicherzustellen, dass der Prozess während dieser Zeiträume keinen grundlegenden Veränderungen unterliegt, ist die Datenauswahl auf einen Zeitraum weniger Wochen um das Experiment zu beschränken, in dem die Modellperformance des statischen Modells als konstant beobachtet wird (vgl. Abbildung 1.2). Die Identifikation der transienten Bereiche erfolgt in diesem Kontext anhand der Setpoint-Änderungen ausgewählter Stellgrößen, die durch die Operatoren initiiert werden. Hinsichtlich der zeitlichen Länge der betrachteten Intervalle ist a priori jedoch unklar, über welchen Zeitraum der zeitliche Verlauf der Daten einen relevanten Einfluss auf die Zielgröße ausübt. Um diesen Einfluss zu untersuchen, werden für jeden Übergang vor und nach der Setpoint-Änderung Zeitfenster mit den Längen 30, 60 und 120 Sekunden betrachtet. Die Auswertung zeigt, dass die Intervalllänge innerhalb dieser Grenzen lediglich einen marginalen Einfluss auf die Modellgüte besitzt. Vor diesem Hintergrund wurden für alle weiteren Untersuchungen Zeiträume von 60s verwendet.

Da einige Stellgrößen im Produktionsbetrieb deutlich häufiger verändert werden als andere, ist es darüber hinaus notwendig, die Anzahl dieser häufig wechselnden Signale zu reduzieren. Auf diese Weise wird eine möglichst uniforme Verteilung der betrachteten transienten Zustände über alle relevanten Stellgrößen angestrebt, um eine Verzerrung des Datensatzes zu vermeiden.

2.2.4 Modellvergleich

Aufbauend auf der Trennung der beiden Datensätze soll nun die Leistungsfähigkeit verschiedener statischer und dynamischer Modellansätze systematisch miteinander verglichen werden. Als Grundlage dazu dient eine umfassende Grid-Search-basierte Hyperparameter-Optimierung, die es erlaubt, sowohl unterschiedliche Modelltypen als auch relevante Vorverarbeitungsschritte zu evaluieren [Ma26]. Vor dem eigentlichen Modelltraining ist eine einheitliche Vorverarbeitung beider Datensätze erforderlich. Dazu werden die Daten zunächst in Trainings-, Validierungs- und Testdatensätze im prozentualen Verhältnis 70 : 20 : 10 aufgeteilt. Anschließend werden Ausreißer identifiziert und aus den Daten entfernt. Um den Einfluss hochfrequenter Störungen zu reduzieren, werden verrauschte Signale außerdem je nach Hyperparameter-Konfiguration entweder mithilfe eines digitalen Butterworth-Filters oder durch einen gleitenden Mittelwert geglättet [OS10]. Abschließend erfolgt eine Normalisierung der Daten auf Basis manuell festgelegter Minima und Maxima, um eine vergleichbare Skalierung sicherzustellen.

Im Rahmen der Hyperparameter-Optimierung werden neben den unterschiedlichen Varianten

Tabelle 2.1: Übersicht der verwendeten Modelltypen

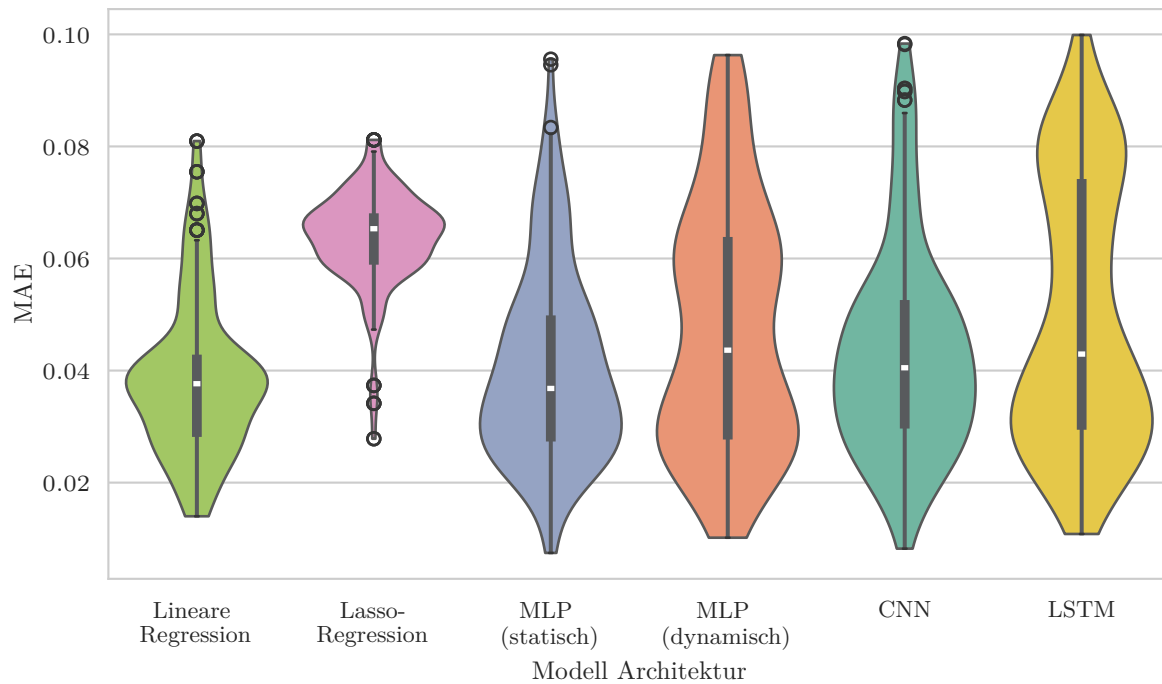
Modellname	Dynamik-Charakter	Aktivierungsfunktion
Lineare Regression	statisch	–
Lasso-Regression	statisch	–
MLP	statisch	Tanh
MLP	dynamisch	Tanh
CNN	dynamisch	ReLU
LSTM	dynamisch	Tanh

zur Datenglättung für die zeitabhängigen Modellansätze verschiedene Fensterbreiten berücksichtigt. Der Modellvergleich und dessen Bewertung erfolgt innerhalb des Hyperparameter-Tuning-Prozesses, indem mehrere Modelltypen auf beiden Datensätzen trainiert und bewertet werden. Eine konsistente und vergleichbare Bewertung der Modelle wird gewährleistet, da beide Datensätze dieselben Features und dasselbe Label enthalten. Zur Sicherstellung der Reproduzierbarkeit sämtlicher Experimente für alle Trainingsläufe wird ein fester Zufallsstartwert (*Seed*) verwendet. Insgesamt umfasst der Vergleich sechs unterschiedliche Modellklassen, die entweder statischen oder dynamischen Charakter besitzen und in Tabelle 2.1 zusammengefasst sind. Zusammenfassend werden bei der Hyperparameter-Optimierung mittels Grid-Search-Strategie folgende Parameter variiert:

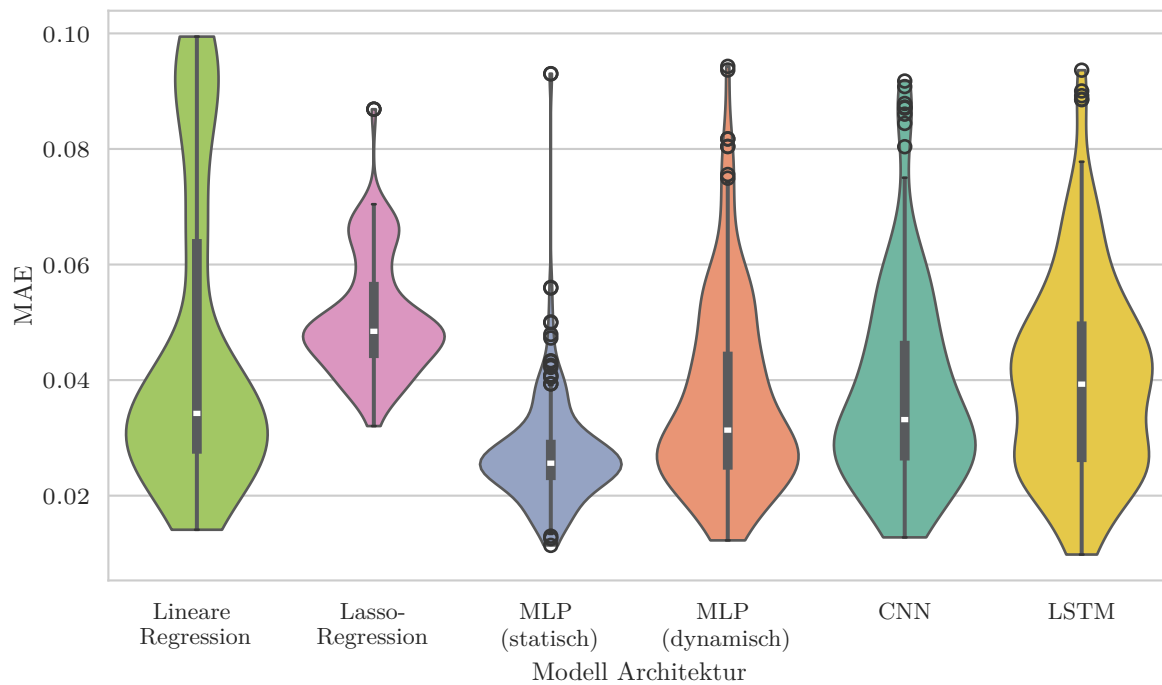
- der Modelltyp (siehe Tabelle 2.1)
- die Art der Datenglättung (keine Glättung, digitale Filterung, gleitender Mittelwert)
- die Anzahl der Neuronen in den Hidden-Layern (8, 16 und 64), sofern für den Modelltyp anwendbar
- die Fensterbreite mit den Werten 5 s, 10 s, 20 s, 30 s und 60 s

Durch die Kombination dieser Parameter ergeben sich insgesamt über 500 unterschiedliche Modell- und Hyperparameterkonfigurationen. Die Bewertung der Modelle erfolgt anhand des mittleren quadratischen Fehlers (Mean Squared Error, MSE) und des mittleren absoluten Fehlers (Mean Absolute Error, MAE). Unabhängig davon, auf welchem der beiden Datensätze ein Modell trainiert ist, werden beide Metriken stets auf den Testdaten beider Datensätze berechnet. Diese wechselseitige Evaluation ermöglicht eine robuste Einschätzung der Generalisierungsfähigkeit der Modelle sowohl für stationäre als auch für transiente Prozesszustände. Alle Trainings- und Evaluationsläufe wurden auf einem vom Deutschen Forschungszentrum für Künstliche Intelligenz (DFKI) bereitgestellten Linux-Hochleistungsrechner durchgeführt.

Zur Bewertung der im Rahmen der Hyperparameter-Grid-Search untersuchten Modelle werden



(a) Quasi-stationäre Testdaten



(b) Transiente Testdaten

Abbildung 2.4: Kombinierte Violin- und Boxplots der MAE-Verteilung aller Modelle auf quasi-stationären und transienten Testdaten

Tabelle 2.2: Vergleich der besten Modellarchitekturen auf stationären und transienten Daten

Modelltyp	Stationäre Daten		Transiente Daten	
	Bester MAE	Δ Best	Bester MAE	Δ Best
Lineare Regression	0,01400	+87,7%	0,01410	+43,7%
Lasso-Regression	0,02783	+273,3%	0,03205	+226,6%
MLP (statisch)	0,00745	0,0%	0,01140	+16,2%
MLP (dynamisch)	0,01017	+36,5%	0,01226	+25,0%
CNN	0,00822	+10,3%	0,01278	+30,3%
LSTM	0,01084	+45,4%	0,00981	0,0%

alle Konfigurationen auf trainingsdisjunkten Testdaten beider Datensätze evaluiert. Zunächst ist festzuhalten, dass sich der eingesetzte Butterworth-Filter als die effektivste Glättungstechnik erweist, da er die Vorhersageleistung sämtlicher Modellarchitekturen konsistent verbessert. In Bezug auf die Performance der jeweiligen Modellarchitekturen lässt sich grundsätzlich festhalten, dass die erzielten Performances aller Modellarchitekturen sehr dicht beieinander liegen. Die Unterschiede im mittleren absoluten Fehler (MAE) manifestieren sich häufig erst in der dritten oder vierten Nachkommastelle. So wurden geringfügig abweichende Ergebnisse in einem anderen Durchgang der gleichen Versuchsreihe in [Ma26] erreicht. Diese hohe Ergebnisdichte impliziert, dass viele Modelle aus statistischer Sicht als gleichwertig betrachtet werden können, wobei minimale Abweichungen plausibel auf geringe Schwankungen im Trainingsprozess zurückzuführen sind.

Grundsätzlich schneiden lineare Modelle, also die lineare Regression und die Lasso-Regression, schlechter ab als ihre nichtlinearen Pendanten. Ihre dennoch vergleichsweise gute Leistung deutet auf eine insgesamt moderate Systemkomplexität hin. Gleichzeitig zeigt der direkte Vergleich, dass lineare Modelle nicht in der Lage sind, die vorhandenen nichtlinearen Zusammenhänge in gleichem Maße zu erfassen wie komplexere Modellarchitekturen.

Unter Berücksichtigung der beiden unterschiedlichen Datensätze sind die besten Modellperformances basierend auf dem jeweils minimalen Simulationsfehler für alle untersuchten Modellarchitekturen in Tabelle 2.2 zusammengefasst. Zusätzlich ist für jedes Modell die relative Abweichung Δ zum jeweils besten Modell innerhalb derselben Kategorie angegeben. Ergänzt werden die tabellarischen Ergebnisse durch die Verteilungsplots in Abbildung 2.4. Während Abbildung 2.4a die Ergebnisse auf den quasi-stationären Testdaten darstellt, zeigen die Plots in Abbildung 2.4b die der transienten Testdaten. Bei den Plots handelt es sich um eine Kombination aus Violin- und Boxplots. Hierbei ermöglicht der Violinplot durch seine Kerndichteschätzung

die Visualisierung der strukturellen Eigenschaften der Verteilung. Zusätzlich bietet der Boxplot eine statistische Einordnung und berücksichtigt auch untere „Ausreißer“, die zur Identifikation der besten Ergebnisse relevant sind.

Die Analyse der quasi-stationären Testdaten ergibt, dass sich die Fehlerverteilungen der nicht-linearen Modelle nur marginal unterscheiden. Dennoch ist bei den besten Konfigurationen eine Überlegenheit von Feedforward-Architekturen erkennbar. Insbesondere erzielt das statische MLP hier die beste Performance über alle durchgeführten Experimente. Jedoch zeigen auch die dynamischen Modelle, die zeitliche Fenster berücksichtigen, auf den quasi-stationären Daten insgesamt gute Ergebnisse. Insbesondere das CNN weist eine vergleichbare Performance auf. Das LSTM hingegen weist mit seiner rekurrenten Architektur sowohl eine schlechtere Best-Performance als auch eine insgesamt nach oben verschobene Fehlerverteilung auf (siehe Abbildung 2.4a). Dies deutet darauf hin, dass die interne Zustandsrückkopplung des LSTM bei stationären und zeitlich invarianten Mustern keinen zusätzlichen Informationsgewinn liefert, sondern zu einem Overfitting auf Rauschanteile führt.

Bei der Betrachtung der transienten Testdaten, die durch dynamische zeitliche Veränderungen charakterisiert sind, ergibt sich ein anderes Bild. Erwartungsgemäß erzielt hier das LSTM den geringsten Simulationsfehler, da seine Architektur explizit für die Modellierung sequenzieller Abhängigkeiten konzipiert ist. Die Anzahl der Neuronen je Hidden-Layer in den Modellen mit der besten Performance liegt zwischen 8 und 16. Diese relativ geringe Anzahl deutet darauf hin, dass auch bei Auswahl des komplexeren dynamischen Modelltyps eine geringe Komplexität ausreichend ist. Bemerkenswert ist jedoch die hohe Leistungsfähigkeit des statischen MLP auch auf dem dynamischen Datensatz. Das bestperformenste MLP liegt nur knapp hinter dem deutlich komplexeren LSTM. Darüber hinaus zeigt die Verteilung der MAEs aller statischen MLP-Konfigurationen eine hohe Dichte im unteren Fehlerbereich, was in Abbildung 2.4b deutlich wird. Dass die nicht-linearen dynamischen Modelle in den transienten Bereichen nicht signifikant besser performen, lässt vermuten, dass die Datenerfassung mit lediglich einem Hertz die Dynamik des Systems nicht ausreichend abbildet.

Für die Auswahl einer geeigneten Modellarchitektur zur Beschreibung des betrachteten Prozesses lässt sich zusammenfassend festhalten, dass keine Architektur auf beiden Datensätzen gleichermaßen als überlegen identifiziert werden kann. Daher ist die Generalisierungsfähigkeit über beide Domänen hinweg ein zentrales Auswahlkriterium, welches für den Einsatz des statischen MLP spricht. Die Fehlerverteilungen zeigen außerdem, dass dieses Modell eine hohe Dichte guter Ergebnisse aufweist und somit weniger sensitiv gegenüber suboptimalen Hyperparameter-Wahlen reagiert. Es erweist sich folglich als robuster im Trainingsprozess im Vergleich zu den übrigen Architekturen. Darüber hinaus ist zu berücksichtigen, welcher Systemzustand für die angestrebte Anwendung von höherer Relevanz ist. Diese Frage lässt sich zwar nicht allgemeingültig beantworten, im Kontext der Betriebspunktoptimierung kommt dem stationären Zustand jedoch eine größere Bedeutung zu. Schließlich ist bei vergleichbarer Leistungsfähigkeit gemäß

dem Prinzip der Parsimonie das einfachere Modell zu bevorzugen. Vor diesem Hintergrund stellt das statische MLP einen geeigneten und ausgewogenen Allrounder für die vorliegende Problemstellung dar.

2.3 Concept Drift

Datengetriebene Modelle wie das in Abschnitt 2.2 entwickelte statische MLP setzen implizit voraus, dass die zugrunde liegende Datenverteilung über die Zeit stabil bleibt. In realen Industrieumgebungen ist diese Annahme jedoch häufig verletzt, da sich Prozesse infolge von Verschleiß, Materialwechseln oder veränderten Betriebsbedingungen kontinuierlich verändern können. Dieses Phänomen wird als CD bezeichnet und stellt eine zentrale Herausforderung für den dauerhaften Einsatz datengetriebener Modelle dar. Aufbauend auf der in Abschnitt 2.2 eingeführten Prozessmodellierung werden in diesem Abschnitt die theoretischen Grundlagen zum Umgang mit CD erarbeitet. Dazu wird zunächst in Abschnitt 2.3.1 eine einheitliche Terminologie etabliert und die verschiedenen Ausprägungen von CD mathematisch abgegrenzt. Anschließend werden in Abschnitt 2.3.2 ausgewählte Methoden zur Erkennung von CD im Datenstrom vorgestellt. Abschließend behandelt Abschnitt 2.3.3 Strategien zur Modelladaptation unter CD.

2.3.1 Terminologie

Viele Anwendungen der datengetriebenen Prozessmodellierung arbeiten in der industriellen Praxis in dynamischen Umgebungen, in denen die zugrunde liegenden Prozesse nicht streng stationär sind. Werden die Ursachen dieser Dynamiken während des Modelltrainings nicht erfasst, die den Prozess jedoch erheblich beeinflussen, spricht man von einem so genannten *Hidden Context* [WK96]. Darüber hinaus ist es möglich, dass sich die dem Modell zugrunde liegende Zielfunktion über die Zeit hinweg verändert [HPC12]. In der Praxis werden diese veränderlichen Einflüsse bei der Modellentwicklung jedoch häufig nicht ausreichend berücksichtigt. Dies kann sich dann im laufenden Betrieb in einem Leistungsabfall des Modells äußern.

Wie sich dieser Leistungsabfall phänomenologisch darstellen kann, ist in Abbildung 2.5 veranschaulicht. Dabei werden vier typische Arten unterschieden, nämlich (a) die plötzliche Veränderung (engl. *Sudden Drift*), (b) die inkrementelle Veränderung (engl. *Incremental Drift*), (c) die schleichende Veränderung (engl. *Gradual Drift*) sowie (d) die wiederkehrende Veränderung (engl. *Recurring Drift*). Diese systematischen Veränderungen sind jedoch von (e) Ausreißern und (f) statistischem Rauschen abzugrenzen.

Obwohl das beschriebene Phänomen der Modelldegradation in der Literatur weitreichend bekannt ist, werden für die verschiedenen Ausprägungen oft unterschiedliche und teils inkonsis-

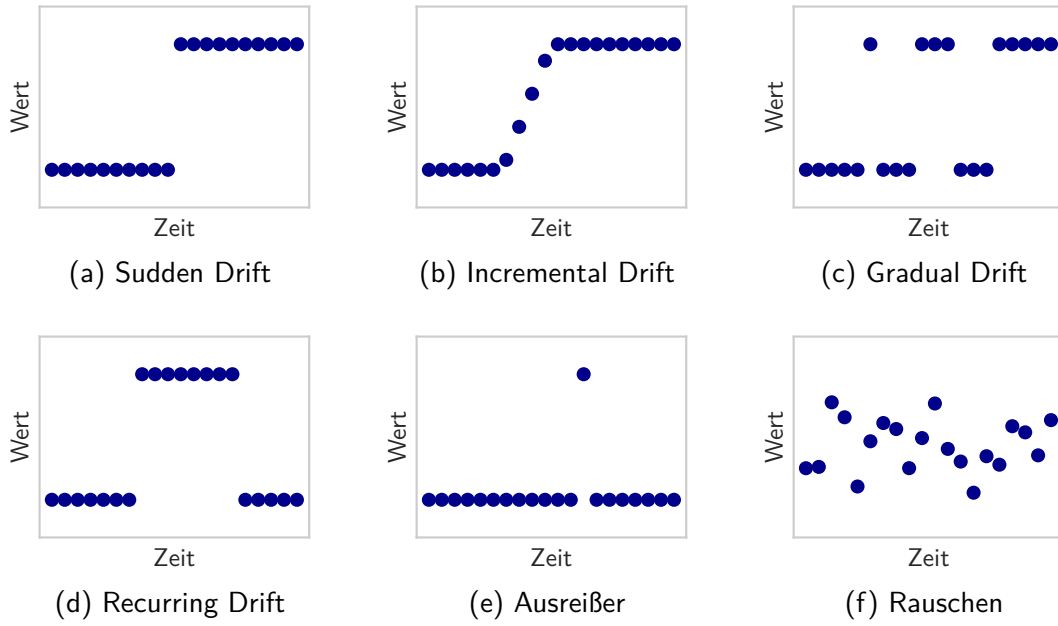


Abbildung 2.5: Phänomenologisches Verhalten von Datenverschiebungen

tente Begriffe verwendet. In manchen Fällen finden sich sogar identische Begrifflichkeiten für gänzlich unterschiedliche Effekte [BAK22]. Häufig verwendete Begriffe stellen in diesem Kontext *Dataset Shift* und *Concept Drift* dar. Grundlegend ist jedoch jede Art von Drift oder Shift durch eine spezifische Veränderung der zugrunde liegenden Datenverteilung gekennzeichnet. Der Begriff des Dataset Shift wird dabei verwendet, um das Abweichen der Datenverteilung zwischen einem statischen Trainings- und einem Test-Szenario zu beschreiben [Qu08]. Concept Drift hingegen bezieht sich primär auf den zeitlichen Verlauf von Verteilungsänderungen im Kontext von kontinuierlichen Datenströmen und Online-Learning-Szenarien [Ga14]. Webb et al. bezeichnen den Concept Drift im Bereich des Stream-Learnings als das direkte Pendant zum Dataset Shift im Batch-Learning [We16].

Obwohl es sich bei der vorliegenden Problemstellung methodisch um Batch-Learning handelt, resultiert die in Abschnitt 1.1 beschriebene Veränderung aus einem zeit-kontinuierlichem Datenstrom. Daher fokussiert sich diese Arbeit auf den Concept Drift. Dazu soll zunächst definiert werden, was unter *Concept* zu verstehen ist. Gama et al. definieren in dem Zusammenhang ein Concept C_t als multivariate Wahrscheinlichkeitsverteilung der Features X und der Labels y zu einem bestimmten Zeitpunkt t [Ga14; We16]. Damit gilt:

$$C_t = P_t(X, y) \quad (2.6)$$

Im Zusammenhang der Definition des Concept Drifts ist anzumerken, dass auch hier in der Literatur einige Begriffe existieren, die als unterschiedliche Ausprägungsformen des Concept Drifts verstanden werden können. Dazu zählen *Permanent Drift*, *Temporary Drift*, *Sampling Shift*, *Feature Change*, *Conditional Change*, *Global Drift*, *Label Shift* und *Target Shift*, um nur eine Auswahl zu nennen [BAK22]. Das Fehlen einer einheitlichen Terminologie wurde 2012 erstmals maßgeblich von Moreno-Torres et al. thematisiert [Mo12].

Tabelle 2.3: Terminologie und ihre zugehörigen mathematischen Definitionen

Terminologie	Mathematische Definition	
Covariate Shift	$P_t(X) \neq P_{t+\Delta t}(X)$	(2.7)
Prior probability Shift	$P_t(y) \neq P_{t+\Delta t}(y)$	(2.8)
Concept Drift	$P_t(X, y) \neq P_{t+\Delta t}(X, y)$	(2.9)
Real Concept Drift	$P_t(y X) \neq P_{t+\Delta t}(y X)$	(2.10)
Actual Drift	$P_t(y X) \neq P_{t+w}(y X)$ und $P_t(X) = P_{t+\Delta t}(X)$	(2.11)
Virtual Drift	$P_t(y X) = P_{t+\Delta t}(y X)$ und $P_t(X) \neq P_{t+\Delta t}(X)$	(2.12)

Um die terminologische Unschärfe aufzulösen, ist es daher zielführend, die verschiedenen Effekte mathematisch voneinander abzugrenzen. Einen solchen Ansatz verfolgen Bayram et al., die die Begriffe der Literatur anhand ihrer mathematischen Definitionen strukturieren [BAK22]. Basierend auf dieser Systematik sind die verschiedenen Arten von Drifts in Tabelle 2.3 aufgeführt, wobei für jede mathematische Definition je ein in der Literatur verwendeter Begriff ausgewählt wurde.

Demzufolge lässt sich Concept Drift formal gemäß (2.9) als die Veränderung der multivariaten Verteilung P der Features X und der Labels y zwischen zwei Zeitpunkten t und $t + \Delta t$ beschreiben. Wird lediglich die Veränderung der Verteilung der Features betrachtet, also der veränderlichen Kovariablen (engl. covariate), ist gemäß (2.7) die Rede von *Covariate Shift*. Analog wird beim *Prior probability Shift* gemäß (2.8) ausschließlich die Veränderung der Zielgröße betrachtet. Für die genauere Betrachtung der verschiedenen Formen von Concept Drift sind *Real Concept Drift* gemäß (2.10), *Actual Drift* gemäß (2.11) und *Virtual Drift* gemäß (2.12) zu unterscheiden. Ihnen gemein ist die Betrachtung der a-posteriori-Wahrscheinlichkeitsverteilung der Labels $P_t(y | X)$ zu verschiedenen Zeitpunkten. Gemäß dem Satz von Bayes lässt sich die bedingte Wahrscheinlichkeit wie folgt umformulieren:

$$P_t(y | X) = \frac{P_t(X | y) \cdot P_t(y)}{P_t(X)} \quad (2.13)$$

Dabei bezeichnet $P_t(y | X)$ die a-posteriori-Wahrscheinlichkeitsverteilung der Labels und $P_t(X)$ die Wahrscheinlichkeitsverteilung der Eingabedaten. $P_t(y)$ stellt weiter die a-priori-Wahrscheinlichkeitsverteilung der Labels und $P_t(X|y)$ die bedingte bedingte Wahrscheinlichkeitsdichteverteilung (engl. Likelihood) dar.

2.3.2 Drift-Erkennung

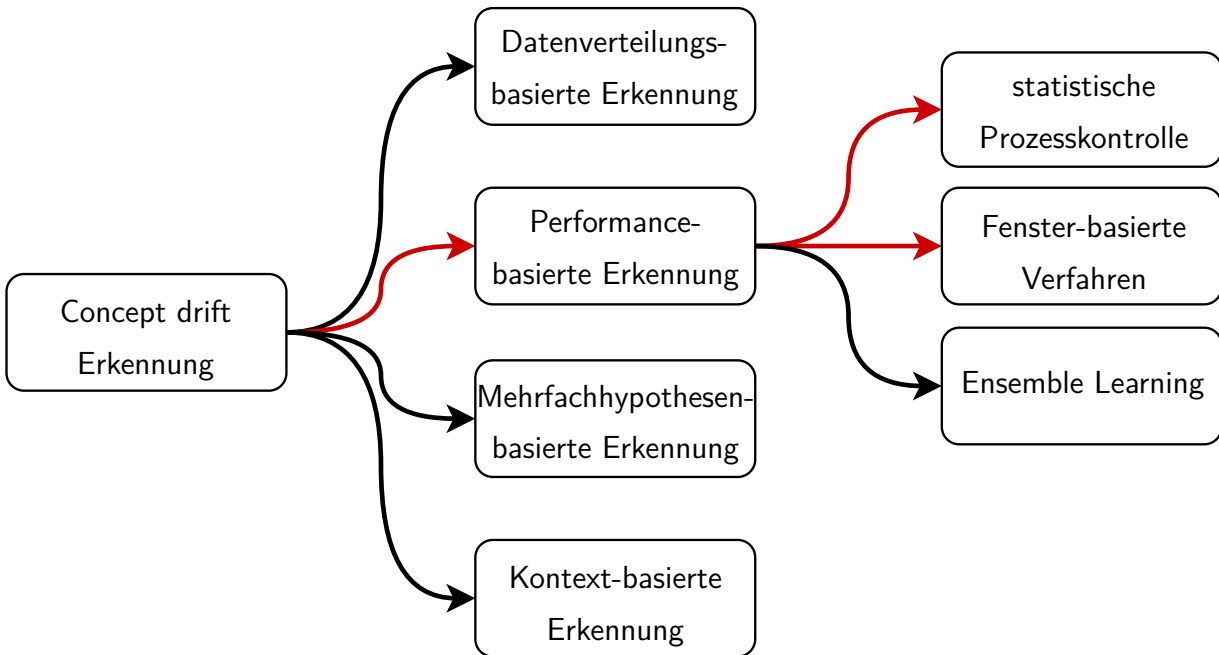


Abbildung 2.6: Methoden zur Erkennung von CD angelehnt an [BAK22]

Um einen CD gemäß (2.9) in einem kontinuierlichen Datenstrom festzustellen, existiert in der Literatur eine Vielzahl von Erkennungsverfahren, die als Drift-Detektoren bezeichnet werden. Im Kern basieren die Detektionsmethoden unabhängig von ihrer algorithmischen Implementierung auf statistischen Hypothesentests [BAK22]. Hierfür wird kontinuierlich eine Teststatistik berechnet, welche die strukturelle Ähnlichkeit oder Abweichung zwischen einer historischen Referenzdatenmenge $\mathcal{D}_t$ und einer aktuell erhobenen Datenmenge $\mathcal{D}_{t+w}$ zu einem späteren Zeitpunkt $t + w$ quantifiziert. Beide Datenmengen bestehen aus Paaren von Eingangsvariablen X und zugehörigen Zielvariablen y und stellen empirische Stichproben der jeweiligen zugrunde liegenden gemeinsamen Wahrscheinlichkeitsverteilungen dar.

Die zugrunde liegende Nullhypothese H_0 nimmt stets an, dass sich die wahren Wahrscheinlichkeitsverteilungen der beiden Datenmengen nicht signifikant unterscheiden – folglich also kein CD vorliegt. Formal lässt sich dies durch einen Hypothesentest und eine Teststatistik D ausdrücken:

$$H_0 : P(\mathcal{D}_t) = P(\mathcal{D}_{t+w}) \quad \text{gegen} \quad H_1 : P(\mathcal{D}_t) \neq P(\mathcal{D}_{t+w}) \quad (2.14)$$

Um diese Hypothese zu überprüfen, wird ein empirischer Wert der Teststatistik $D(\mathcal{D}_t, \mathcal{D}_{t+w})$ berechnet und mit einem vordefinierten Schwellenwert λ abgeglichen [DR09]. Solange $D(\mathcal{D}_t, \mathcal{D}_{t+w}) \leq \lambda$ gilt, kann H_0 nicht verworfen werden und das System geht von einer stabilen Verteilung aus. Wird der Schwellenwert hingegen überschritten, also $D > \lambda$, wird die Nullhypothese zum gewählten Signifikanzniveau verworfen und das System deklariert einen CD.

Abhängig von der Teststatistik können verschiedene Methoden der Drift-Erkennung unterschieden werden, wie in Abbildung 2.6 dargestellt [BAK22]. Die rot markierten Pfade stellen dabei

diejenigen Ansätze dar, die in dieser Arbeit verwendet wurden. Allen Drift-Detektionsmethoden liegt dieselbe Nullhypothese zugrunde, wonach keine signifikante Änderung der Datenverteilung vorliegt. Die Unterschiede ergeben sich hingegen aus der Wahl der Teststatistik und der Art der Auswertung des Datenstroms.

Bei der *Datenverteilungs-basierten Erkennung* werden Distanzmaße verwendet, um die Ähnlichkeit zwischen den Datenverteilungen in zwei verschiedenen Zeitfenstern abzuschätzen [KBG04]. Zwar ist dieser Ansatz auch für ungelabelte Daten geeignet, es wird jedoch nicht berücksichtigt, ob die Änderungen tatsächlich die Vorhersageleistung des Modells beeinflussen [Ga14]. *Multiple hypothesis-based Drift-Detektoren* stellen hybride Verfahren dar, die mehrere Detektionsmethoden parallel oder aufeinanderfolgend auswerten [Lu18]. Während parallele Ansätze Entscheidungsregeln kombinieren, nutzen hierarchische Verfahren eine Struktur aus Warnung und Validierung. Unter *Kontext-basierter Drift-Erkennung* lassen sich solche Ansätze zusammenfassen, die klassische Detektoren um kontextuelle Informationen aus System oder Modell erweitern [BAK22]. Der Begriff kann hier als Sammelbegriff für unterschiedliche methodische Ausprägungen verstanden werden.

Ansätze der *Performance-basierten Erkennung* verfolgen Abweichungen im Output-Fehler des Modells, um Veränderungen zu erkennen. Ihr Hauptvorteil besteht darin, dass so Drifts nur detektieren werden, wenn auch die Leistung beeinträchtigt wird. Wie in Abbildung 2.6 veranschaulicht, lassen sich diese Methoden je nach der zur Erkennung von Leistungseinbrüchen verwendeten Strategie weiter in folgende Kategorien einteilen: *statistische Prozesskontrolle*, *Fenster-basierte Verfahren* und *Ensemble-Learning*.

Bei der statistischen Prozesskontrolle wird die Qualität des (kontinuierlichen) Lernprozesses anhand der zeitlichen Entwicklung der Fehlerrate überwacht. Überschreitet die Fehlerrate eine statistisch definierte Schwelle, wird ein CD detektiert. Die wohl am häufigsten zitierte Methode dieser Kategorie ist die so genannte **Drift-Erkennungsmethode (DDM)** von Gama et al. [Ga04]. DDM überwacht die Fehlerrate im Datenstrom und basiert auf der Annahme, dass jede Modellentscheidung entweder korrekt oder fehlerhaft ist. Entsprechend werden Fehlklassifikationen als Bernoulli-Zufallsvariable modelliert, sodass pro Beobachtung lediglich zwei Ausprägungen (Richtig oder Falsch) möglich sind. Nach i Beobachtungen werden die Fehlerrate p_i sowie deren Standardabweichung

$$\sigma_i = \sqrt{\frac{p_i(1 - p_i)}{i}} \quad (2.15)$$

geschätzt. Unter der Annahme eines stationären Zustands und für eine hinreichend große Anzahl an Beobachtungen kann die zugrunde liegende Binomialverteilung durch eine Normalverteilung approximiert werden. DDM speichert dabei die bislang minimal beobachteten Fehlerraten $p_{\min}$ und Standardabweichungen $\sigma_{\min}$, die fortlaufend aktualisiert werden, sofern $p_i + \sigma_i < p_{\min} + \sigma_{\min}$. Ein Warnzustand wird dann ausgelöst, wenn die aktuelle Fehlerrate signifikant vom bisherigen Minimum abweicht, konkret wenn $p_i + \sigma_i \geq p_{\min} + 2 \sigma_{\min}$. Über-

schreitet die Abweichung eine Schwelle von $p_i + \sigma_i \geq p_{\min} + 3 \sigma_{\min}$, wird ein CD detektiert. Der Warnzustand erlaubt dabei eine frühzeitige Indikation eines potenziellen Drifts und wird häufig zur Anpassung des Beobachtungsfensters genutzt.

Eine Erweiterung der DDM stellt die **Early Drift Detection Method (EDDM)** dar [Ba06]. EDDM wurde insbesondere zur verbesserten Erkennung gradueller Konzeptdrifts entwickelt, ohne dabei bei abrupten Driftformen an Performance einzubüßen. Im Gegensatz zu DDM analysiert EDDM die zeitlichen Abstände zwischen aufeinanderfolgenden Fehlklassifikationen. Die zentrale Annahme ist dabei, dass ein gradueller Drift zu einer sukzessiven Verkürzung der Abstände zwischen aufeinanderfolgenden Fehlklassifikationen führt. Für jedes Zeitintervall i wird daher die mittlere Distanz zwischen zwei Fehlklassifikationen p'_i sowie deren Standardabweichung σ'_i geschätzt. Während des Lernprozesses speichert EDDM die Referenzwerte $p'_{\max}$ und $\sigma'_{\max}$, sofern $p'_i + 2 \sigma'_i > p'_{\max} + 2 \sigma'_{\max}$. Diese Referenzwerte entsprechen dem Zustand, in dem das Modell das zugrunde liegende Konzept am besten approximiert. Zur Drift-Erkennung sei die dimensionslose Kennzahl

$$r_i = \frac{p'_i + 2 \sigma'_i}{p'_{\max} + 2 \sigma'_{\max}} \quad (2.16)$$

definiert, welche den aktuellen Abstandsdruck relativ zum bislang maximal beobachteten Referenzwert beschreibt. Auf Basis dieser Größe werden zwei Schwellenwerte unterschieden:

- Ein Warnzustand wird ausgelöst, wenn $r_i < \alpha$.
- Ein Konzeptdrift wird detektiert, wenn $r_i < \beta$, wobei $\beta < \alpha$.

Nach Erreichen des Driftzustands werden die Referenzwerte $p'_{\max}$ und $\sigma'_{\max}$ zurückgesetzt.

Bei Fenster-basierten Verfahren wird ein Datenstrom in Referenz- und Detektionsfenster geteilt. Drifts werden anhand signifikanter Unterschiede von statistischen Tests zwischen beiden Fenstern erkannt. Die Fenster können dabei zeit- oder größenbasiert definiert und flexibel positioniert werden. Ein Fenster-basierter Ansatz ist das **Adaptive Windowing (ADWIN)** von Bifet et al. (2007) [BG07]. Dabei wird CD anhand sequentieller statistischer Hypothesentests über ein adaptives Datenfenster erkannt. Ziel des Algorithmus ist es, ein Fenster W so groß wie möglich zu halten, solange die Nullhypothese einer konstanten Erwartung innerhalb dieses Fensters nicht verworfen werden kann. Dazu wird W fortlaufend in zwei aufeinanderfolgende Teilfenster W_0 und W_1 mit Längen n_0 und n_1 unterteilt, sodass $n = n_0 + n_1$. Mit der Zerlegung des aktuellen Fensters wird geprüft, ob sich die beobachteten Mittelwerte $\hat{\mu}_{W_0}$ und $\hat{\mu}_{W_1}$ signifikant unterscheiden. Die Entscheidungsregel basiert auf der Hoeffding-Schranke ε_{cut} , mit

$$\varepsilon_{\text{cut}} = \sqrt{\frac{1}{2m} \cdot \ln\left(\frac{4}{\delta'}\right)}, \quad m = \frac{1}{1/n_0 + 1/n_1} \quad \text{und} \quad \delta' = \frac{\delta}{n}, \quad (2.17)$$

wobei m das harmonische Mittel der Fensterlängen darstellt und δ das globale Signifikanzniveau des Hypothesentests bezeichnet. Das angepasste Niveau δ' wird zur Kontrolle multipler Hypothesentests eingeführt. Überschreitet die Differenz der Mittelwerte den Schwellenwert ε_{cut} ,

wird die Nullhypothese verworfen und auf einen CD geschlossen, woraufhin der ältere Fensteranteil W_0 entfernt wird. Andernfalls bleibt das Fenster unverändert, sodass sich das Fenster durch neue Datenpunkte sukzessive vergrößert. Als Schätzung der aktuellen Erwartung gibt ADWIN kontinuierlich den empirischen Mittelwert $\hat{\mu}_W$ des aktuellen Fensters aus. Das zugrunde liegende Prinzip von ADWIN ist unabhängig vom konkreten statistischen Test formuliert und erlaubt alternative Formen einer Teststatistik.

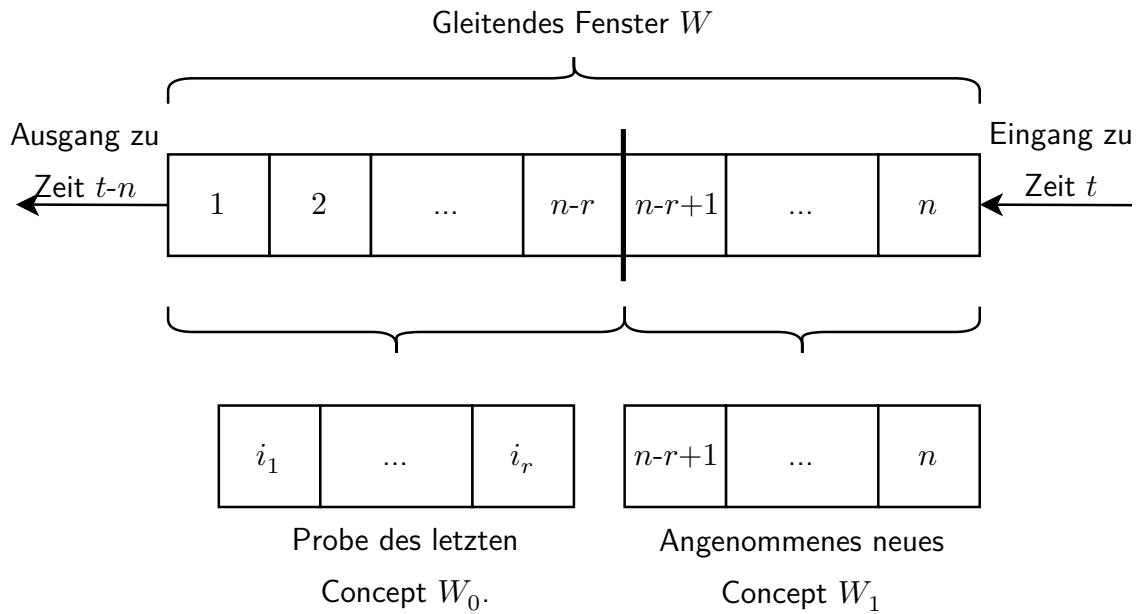


Abbildung 2.7: Speicherstrategie von KSWIN angelehnt an [RHS20]

Ein weiterer Fenster-basierter Ansatz zur Erkennung von CD ist das **Kolmogorov-Smirnov Windowing (KSWIN)** von Raab et al. (2020) [RHS20]. KSWIN basiert auf einem Gleitfenster fester Größe, das kontinuierlich die zuletzt beobachteten Datenpunkte eines Datenstroms speichert und mithilfe eines nicht-parametrischen statistischen Tests auf Verteilungsänderungen untersucht. Hierzu wird ein Fenster W der Größe n geführt, das mit jedem neu eintreffenden Datenpunkt aktualisiert wird, indem der älteste Wert entfernt und der neue Wert hinzugefügt wird. Aus diesem Fenster werden zwei Teilfenster konstruiert: Ein aktuelles Detektionsfenster W_1 , das die r zuletzt eingetroffenen Datenpunkte enthält, sowie ein Referenzfenster W_0 , das ebenfalls aus r Datenpunkten besteht. Diese werden uniform zufällig aus dem älteren Bereich des Fensters W gezogen, der nicht zu W_1 gehört. Auf diese Weise soll W_0 die historische Datenverteilung approximieren, ohne Annahmen über deren Form zu treffen. Die Speicherstrategie ist in Abbildung 2.7 veranschaulicht.

Zur Drift-Erkennung wird der *Kolmogorov-Smirnov-Test* zwischen den empirischen Verteilungsfunktionen der beiden Teilfenster W_0 und W_1 durchgeführt [Lo11]. Der Test basiert auf der maximalen absoluten Abweichung

$$\text{dist}(W_0, W_1) = \sup_x |F_{W_0}(x) - F_{W_1}(x)|, \quad (2.18)$$

wobei F_{W_0} und F_{W_1} die empirischen kumulativen Verteilungsfunktionen der jeweiligen Fenster

darstellen. Überschreitet diese Distanz einen durch ein Signifikanzniveau bestimmten Schwellenwert, wird die Nullhypothese gleicher Verteilungen verworfen und auf einen CD geschlossen.

2.3.3 Drift-Adaption

Wie einleitend in Abschnitt 1.1 exemplarisch dargestellt, kann die Modellperformance durch CD beeinflusst werden. Durch Methoden zur Erkennung von CD, wie sie in Abschnitt 2.3.2 beschrieben sind, lässt sich der Drift zwar detektieren, das durch den Drift beeinträchtigte Modell bleibt dabei jedoch unverändert. Zur Verbesserung der Performance bedarf es folglich einer Anpassung des Modells. Einen umfassenden und bis heute maßgeblichen Überblick über Strategien zur Anpassung von Modellen unter CD, der in der Literatur häufig als Referenz herangezogen wird, liefern Gama et al. in ihrer Survey-Arbeit zur CD-Adaption [Ga14].

Zunächst hängt die konkrete Ausgestaltung einer Anpassung gegenüber CD davon ab, welche Informationen im Datenstrom vorgehalten und wie sie verwaltet werden. Bei der Speicherung von Informationen im Datenstrom können grundlegend zwei Speichertypen unterschieden werden: dem Kurz- und dem Langzeitspeicher. Beim Kurzzeitspeicher handelt es sich um die Daten selbst, die verarbeitet werden. Der Langzeitspeicher hingegen stellt eine Generalisierung von Daten dar, nämlich das Modell. Beide Speichergrundlagen sind essenziell für die Handhabung von CD. Der Kurzzeitspeicher bestimmt, welche aktuellen Informationen beim Anpassungsprozess verwendet werden sollen, etwa einzelne Datenpunkte oder gleitende Fenster fixer oder variabler Größe. Ergänzend regeln Vergessensmechanismen, wie ältere Informationen verworfen werden. Hier wiederum kann zwischen abrupten und graduellen Ansätzen unterschieden werden. Ein schnelleres Vergessen erhöht in diesem Zusammenhang zwar die Reaktionsfähigkeit gegenüber Drifts, geht jedoch mit einer höheren Anfälligkeit gegenüber Rauschen einher.

Der zweite Speichertyp – das Modell – steht im Zentrum der Anpassung. Ziel modellbasierter Anpassungsstrategien ist es, die im Modell verankerte Generalisierung an veränderte Zusammenhänge anzupassen und dadurch die Prädiktionsgüte unter Driftbedingungen zu verbessern bzw. wiederherzustellen. Bei der Adaption gilt als eine besondere Herausforderung das so genannte *Stability-plasticity-dilemma*, dem Spagat zwischen der Fähigkeit eines Modells, neues Wissen aufzunehmen (Plastizität) und vorhandenes Wissen zu erhalten (Stabilität). Hierbei das richtige Gleichgewicht zu finden ist von zentraler Bedeutung, da zu viel Plastizität zu katastrophalem Vergessen führen kann, also dem Verlust bereits erlernter Informationen, wenn neue Informationen erfasst werden. Gleichzeitig kann zu viel Stabilität dafür sorgen, dass Anpassungen an Drifts nicht wirksam sind.

Zur Anpassung eines Modells an veränderte Datenverteilungen existieren grundsätzlich zwei Strategien. Einerseits kann ein vollständiges Neutrainieren erfolgen, bei dem auf Basis gepufferter, aktualisierter Daten ein neues Modell erzeugt und das bisherige Modell verworfen wird. Dieser Ansatz wird häufig als *Full Relearning* bezeichnet und dient in der Forschung als

robuste Referenz, ist jedoch mit erhöhtem Rechenaufwand und Datenbedarf verbunden. Insbesondere bei Notwendigkeit von statistischer Versuchsplanung zur Datenakquise stellt dies eine große Herausforderung dar. Andererseits kann das bestehende Modell inkrementell angepasst werden, indem neue Beobachtungen schrittweise in das Modell integriert werden. Dieses Vorgehen ist dem Bereich des *Continual Learning* bzw. des *Incremental Learning* zuzuordnen und zielt darauf ab, eine kontinuierliche Modelladaption ohne vollständig neues Modelltraining zu ermöglichen. Gerade unter CD stellt diese Strategie jedoch besondere Anforderungen, da fortlaufende Updates das Risiko katastrophalen Vergessens erhöhen und ein sorgfältiges Ausbalancieren von Stabilität und Plastizität erfordern. In der Praxis spiegelt sich dieser Zielkonflikt in einem Trade-off zwischen Adaptionsgeschwindigkeit, Robustheit und Validierbarkeit wider. Hybride Ansätze versuchen hier eine Lösung dieses Zielkonfliktes zu bieten [Ga04].

Neben der Art der Modellanpassung ist entscheidend, wann bzw. wodurch diese ausgelöst wird. Abhängig davon, ob eine Anpassung gezielt durch erkannte Veränderungen erfolgt oder unabhängig davon kontinuierlich durchgeführt wird, können folgende Ansätze unterschieden werden:

- **Aktive Anpassung (engl. auch informed adaptation)**

Die ergebnisgetriebene Anpassung wird durch den Output der Drift-Erkennung ausgelöst und kann entsprechend zielgerichtet erfolgen. Dieser reaktive Charakter macht sie jedoch vollständig abhängig von der Detektionsqualität, da nicht erkannte Drifts keine Anpassung auslösen. Die Methode eignet sich daher vor allem für plötzliche, klar identifizierbare Drifts.

- **Passive Anpassung (engl. auch blind adaptation)**

Die Anpassung erfolgt kontinuierlich und unabhängig von etwaigen Driftinformationen. Der Ansatz ist sehr einfach anzuwenden, da es keiner Drift-Erkennung bedarf. Bei zu niedriger Anpassungsfrequenz besteht jedoch das Risiko, auf bereits eingetretene Drifts nicht zu reagieren. Bei zu hoher Anpassungsfrequenz wird das Modell ohne vorliegenden Drift ohne Notwendigkeit angepasst, was zu Overfitting führen kann und das katastrophale Vergessen begünstigt. Diese Methode ist proaktiv und daher besonders geeignet für schleichende oder nicht klar trennbare Drifts, da die Anpassung kontinuierlich und unabhängig von einer Drift-Erkennung erfolgt.

Der Langzeitspeicher kann darüber hinaus auch so gestaltet sein, dass mehrere Modelle parallel vorgehalten und angepasst werden. Ein etablierter Ansatz ist dabei das Ensemble-Learning, bei dem eine Zusammenstellung aus mehreren Modellen gemeinsam eine Vorhersage generiert. Adaptive Ensembles basieren in dem Kontext auf der Annahme, dass Daten während einer Veränderung aus einer gemischten Verteilung stammen, die als gewichtete Kombination von Verteilungen betrachtet werden kann, die die Zielkonzepte charakterisieren. Jedes einzelne Modell bildet dabei eine Verteilung ab [SK07]. Die endgültige Vorhersage ergibt sich typischerweise als gewichtete Kombination der Einzelvorhersagen, wobei die Gewichtung die Leistung

der jeweiligen Modelle auf den neuesten Daten widerspiegelt und im zeitlichen Verlauf angepasst wird. Bei wiederkehrenden Konzeptdrifts bieten Ensemble-Ansätze in Kombination mit einer geeigneten Modellverwaltung besondere Vorteile. Dadurch kann der mit erneutem Training verbundene Rechenaufwand deutlich reduziert und die Reaktionsfähigkeit des Systems auf bereits bekannte Konzepte verbessert werden. Die konkrete Behandlung solcher rekurrenten Drifts ist dabei stark vom eingesetzten Verfahren und der Driftcharakteristik abhängig.

3 Generalisierung

Grundlage dieser Arbeit ist ein Regressionsmodell, das, wie in Abschnitt 2.2 beschrieben, mit einem DOE-basierten Datensatz aus Sensordaten im stationären Zustand eines Prozesses der Prozessindustrie trainiert wurde. Als geeignete Modellarchitektur wurde hierfür ein MLP identifiziert. Die Güte dieses Modells wird jedoch durch CD beeinträchtigt, dessen Grundlagen in Abschnitt 2.3 allgemein dargestellt sind. Verändert sich das zugrunde liegende Prozessverhalten über die Zeit, weicht die Datengrundlage zunehmend von der Trainingsverteilung ab, sodass die Modellperformance sinkt, wie in Kapitel 1 skizziert. Diese Problemstellung soll nicht an die Besonderheiten der Vertikalrollenmühle gebunden bleiben und wird daher zunächst von der spezifischen Anwendung gelöst und auf allgemeinere prozesstechnische Fragestellungen übertragen. Auf dem so generalisierten Problem setzt der hier verfolgte Lösungsansatz an, der dem drift-bedingten Leistungsabfall durch eine kontinuierliche Adaption des Modells begegnet.

Damit eine Datengrundlage diese generalisierende Rolle erfüllen kann, muss sie einerseits hinreichend allgemein sein, um eine Generalisierung zu ermöglichen, und andererseits genügend Ähnlichkeit zum ursprünglichen Prozess aufweisen, um methodische Konsistenz sicherzustellen. Grundlage einer validen Generalisierung ist daher die Auswahl einer geeigneten Datengrundlage, die folgende Voraussetzungen erfüllt:

- **Abbildung realer Prozesscharakteristik**

Der Datensatz muss reale physikalische Eigenschaften wie Systemträgheit, Sensorrauschen und komplexe Korrelationen zwischen den Variablen abbilden, wie sie für verfahrenstechnische Anlagen charakteristisch sind.

- **Klassifizierung der Variablen und Kausalität**

Der Datensatz muss eine klare Unterscheidung zwischen Stellgrößen, Messgrößen und Störgrößen sowie eine von den Eingangsgrößen abhängige Zielgröße bereitstellen, um die Kausalität des Modells korrekt abzubilden.

- **Steuerbarkeit und Betriebspunktabdeckung**

Die Führungsgrößen müssen aktiv ansteuerbar sein, um Betriebspunkte gezielt zu erzeugen, und es muss eine hinreichend große Anzahl unterschiedlicher, durch Stellgrößen definierter Betriebszustände vorliegen bzw. erzeugbar sein. Dies ist essenziell, um eine statistische Versuchsplanung zu simulieren und eine robuste Aufteilung in Trainings-, Validierungs- und Testdaten zu ermöglichen.

- **Erfassbarkeit stationärer Betriebspunkte**

Da das Modell auf stationären Betriebspunkten arbeitet, muss die Abtastrate ausreichend hoch sein, um stationäre Zustände zuverlässig von transienten Phasen trennen und je Betriebspunkt eine ausreichende Zahl an Messpunkten für die Extraktion bereitstellen zu können.

- **Abbildbarkeit von CD**

Der Datensatz muss über einen hinreichenden Zeithorizont Szenarien enthalten oder deren Simulation ermöglichen, in denen sich das Prozessverhalten signifikant ändert, sodass sowohl die Driftentwicklung als auch die wiederholte Evaluation der Adaptionsmechanismen abgebildet werden können.

- **Drift-freie Referenzdaten**

Für das initiale Training des Basismodells ist ein Zeitraum erforderlich, in dem der Prozess stabil und ohne systematische Abweichungen abläuft.

- **Kontinuierliche Verfügbarkeit der Zielgröße**

Die Zielgröße muss fortlaufend vorliegen, da die Adaption das Modell auf den drift-behafteten Labels nachtrainiert.

- **Öffentliche Verfügbarkeit und Reproduzierbarkeit**

Zur Gewährleistung der Vergleichbarkeit der Ergebnisse muss der Datensatz öffentlich zugänglich, die Datenerhebung transparent dokumentiert und die Ergebnisse reproduzierbar sein.

Die Generalisierung erfolgt in zwei Stufen zunehmender Realitätsnähe. Zunächst wird in Abschnitt 3.1 ein synthetischer Datenstrom erzeugt, der die genannten Voraussetzungen unter vollständig kontrollierten Bedingungen per Konstruktion erfüllt und den CD mit bekannter Ground Truth vorgibt. Darauf aufbauend betrachtet Abschnitt 3.2 mit dem Tennessee Eastman Process (TEP) einen realitätsnäheren, öffentlich verfügbaren Benchmark-Prozess, der reale, gekoppelte Prozessdynamik und Regelstrukturen einbringt und die Voraussetzungen weitgehend erfüllt, sodass allein der CD modifiziert werden muss.

3.1 Synthetischer Datensatz

Als erster Schritt zur Generalisierung wird ein synthetischer Datenstrom erzeugt, der unter kontrollierten Bedingungen die Erprobung des Adaptionsansatzes bei CD ermöglicht. Dabei sollen die wesentlichen Eigenschaften prozesstechnischer Messdaten synthetisch nachgebildet werden. Abbildung 3.1 gibt einen Überblick über die zugrunde liegende Simulationsstrategie. Die drei Hauptkomponenten - die exogenen Setpoints, die Aktuatordynamik und die Systemantwort - werden dabei von CD beeinflusst, dessen Modellierung in Abschnitt 3.1.1 beschrieben wird. Darauf aufbauend wird in Abschnitt 3.1.2 die Erzeugung stückweise konstanter exogener Setpoints

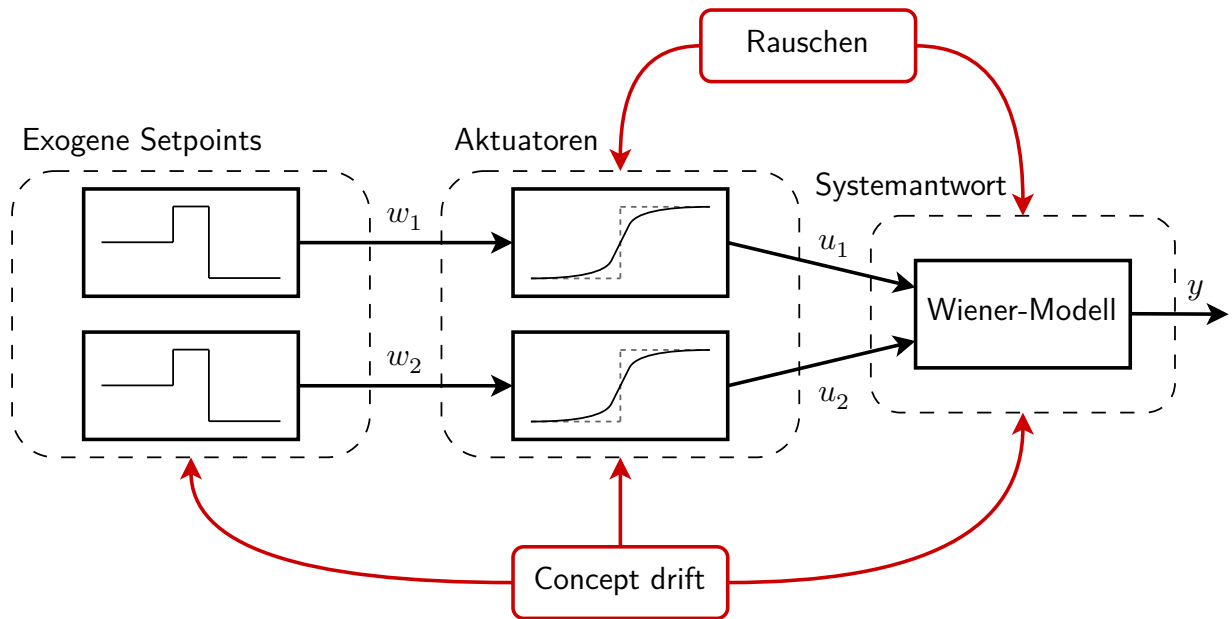


Abbildung 3.1: Übersicht zur Simulationsstrategie

erläutert. Diese durchlaufen PT_1 -Glieder, die als Spezialfall der in Abschnitt 2.2.1 eingeführten Zustandsraumdarstellung die Trägheit der Aktuatoren nachbilden. Deren Modellierung ist in Abschnitt 3.1.3 ausgeführt. Das heterogene Rauschmodell, beschrieben in Abschnitt 3.1.4, überlagert die erzeugten Signale additiv als Beobachtungsrauschen und wirkt damit unabhängig vom CD. Die rauschfreien Aktuator-Signale dienen als Eingang eines Wiener-Modells, das eine gewichtete Summation, ein weiteres PT_1 -Glied zur Abbildung der Prozessträgheit und eine statische Nichtlinearität in Serie schaltet; sein Ausgang y bildet die beobachtete Systemantwort. Das Wiener-Modell wird in Abschnitt 3.1.5 näher spezifiziert und auf den vorliegenden Fall angewandt. Die gesamte Datenstromsynthese wurde in Python implementiert. Der erzeugte Datenstrom bildet die Grundlage der anschließenden Verarbeitungskette aus Modellierung, Detektion und Adaption. Zunächst wird der synthetische Prozess in Abschnitt 3.1.6 modelliert, wobei ein MLP sowie ein Kalman- und ein Extended-Kalman-Filter gegenübergestellt werden. Aufbauend auf dem Simulationsfehler des MLP behandelt Abschnitt 3.1.7 die Detektion des CD. Abschnitt 3.1.8 schließt die Kette mit der Adaption des Modells an den drift-behafteten Prozess ab.

3.1.1 Concept Drift

Einen typischer Mechanismus in der industriellen Praxis, der CD hervorrufen kann, stellen kontinuierliche Veränderungen infolge von Einflüssen wie Verschleiß und sich verändernden äußeren Bedingungen dar. Wie in Abbildung 2.5 dargestellt, können sich diese Einflüsse in Form von *Incremental Drift* äußern. Dem gegenüber stehen kurzfristige Maßnahmen wie Wartungs- oder Instandhaltungsereignisse, die *Sudden Drifts* verursachen können. Bei der Synthese des Drift-signals $c[k]$ liegt der Fokus auf diesen beiden charakteristischen Phänomenen.

Das zeitabhängige Driftsignal $c[k]$ dient dabei als Grundlage für alle driftabhängigen Modifikationen des synthetischen Datenstroms. Es nimmt zu jedem Zeitschritt k einen skalaren Wert an und beeinflusst sowohl die Parameter der PT_1 -Glieder als auch die Setpoints sowie den stationären Arbeitspunkt der Systemantwort. Die beiden genannten Drifttypen werden durch die Bildung von N_{seg} Segmenten kombiniert: Innerhalb jedes Segments nimmt $c[k]$ einen linearen Verlauf negativer Steigung an, was einen *Incremental Drift* repräsentiert. An den Grenzen zwischen den Segmenten hingegen wird durch einen sprungartigen Übergang auf ein neues Ausgangsniveau ein *Sudden Drift* erzeugt. Diese Struktur erzeugt ein sägezahnartiges Muster, das beide Driftformen miteinander kombiniert.

Formal erstreckt sich das Signal über eine Gesamtlänge von K Zeitschritten. Die Wahl der Abtastfrequenz orientiert sich dabei am Ausgangsbeispiel der Mühle mit GPlink, das mit einer Frequenz von 1 Hz arbeitet. Die entsprechende Abtastzeit $T_s = 1$ s wird auf ein Kalenderjahr angewandt, welches in N_{seg} Segmente unterteilt ist. Die $N_{\text{seg}} - 1$ inneren Segmentgrenzen $\tau_1 < \tau_2 < \dots < \tau_{N_{\text{seg}}-1}$ werden durch gleichmäßige Aufteilung des Zeitbereichs mit einer zufälligen Verschiebung von maximal $\pm \gamma_\tau \cdot K / (2 N_{\text{seg}})$ erzeugt, wobei γ_τ den Verschiebungsanteil bezeichnet. Zusammen mit den festen Randpunkten $\tau_0 = 0$ und $\tau_{N_{\text{seg}}} = K$ sind damit alle Segmentgrenzen definiert.

Innerhalb jedes Segments $i \in \{1, \dots, N_{\text{seg}}\}$ wird $c[k]$ durch lineare Interpolation (*Incremental Drift*) vom Startniveau c_i^+ zum Zielniveau c_i^- geformt:

$$c[k] = c_i^+ + \frac{c_i^- - c_i^+}{\tau_i - \tau_{i-1}} (k - \tau_{i-1}), \quad k \in [\tau_{i-1}, \tau_i) \quad (3.1)$$

Das Zielniveau $c_i^- \sim \mathcal{U}(c_{\min}, c_{\min}/4)$ wird aus dem negativen Teilbereich gezogen, sodass $c[k]$ innerhalb jedes Segments monoton fällt. An der Segmentgrenze τ_i erfolgt ein sprungartiger Übergang (*Sudden Drift*) zum Startniveau des folgenden Segments. Sämtliche Startniveaus $c_i^+ \sim \mathcal{U}(c_{\max}/4, c_{\max})$, $i \in \{1, \dots, N_{\text{seg}}\}$, werden unabhängig aus dem positiven Teilbereich gezogen; das letzte Segment endet regulär bei $c[K-1] = c_{N_{\text{seg}}}^-$ ohne abschließenden Sprung. Das Driftsignal ist damit durch die Parameter $c_{\min}$, $c_{\max}$, N_{seg} und γ_τ vollständig charakterisiert. In der Simulation werden $c_{\min} = -0,5$, $c_{\max} = 0,5$, $N_{\text{seg}} = 5$ und $\gamma_\tau = 0,2$ verwendet. Abbildung 3.2 zeigt den resultierenden Verlauf des Driftsignals $c[k]$ über ein Kalenderjahr. Trotz des diskreten Charakters von $c[k]$ erfolgt die Darstellung als kontinuierlicher Verlauf, da der stückweise lineare Verlauf so unmittelbar erkennbar wird. Das sägezahnartige Muster aus linearem Abfall beim *Incremental Drift* innerhalb der Segmente und sprungartigen Anstiegen des *Sudden Drift* an den Segmentgrenzen ist dabei deutlich erkennbar. Da c_{i+1}^+ stets aus dem positiven Teilbereich und c_i^- stets aus dem negativen Teilbereich gezogen wird, sind die beiden Drifttypen im Signalverlauf klar voneinander unterscheidbar und das Signal deckt den Wertebereich $[c_{\min}, c_{\max}]$ näherungsweise ab.

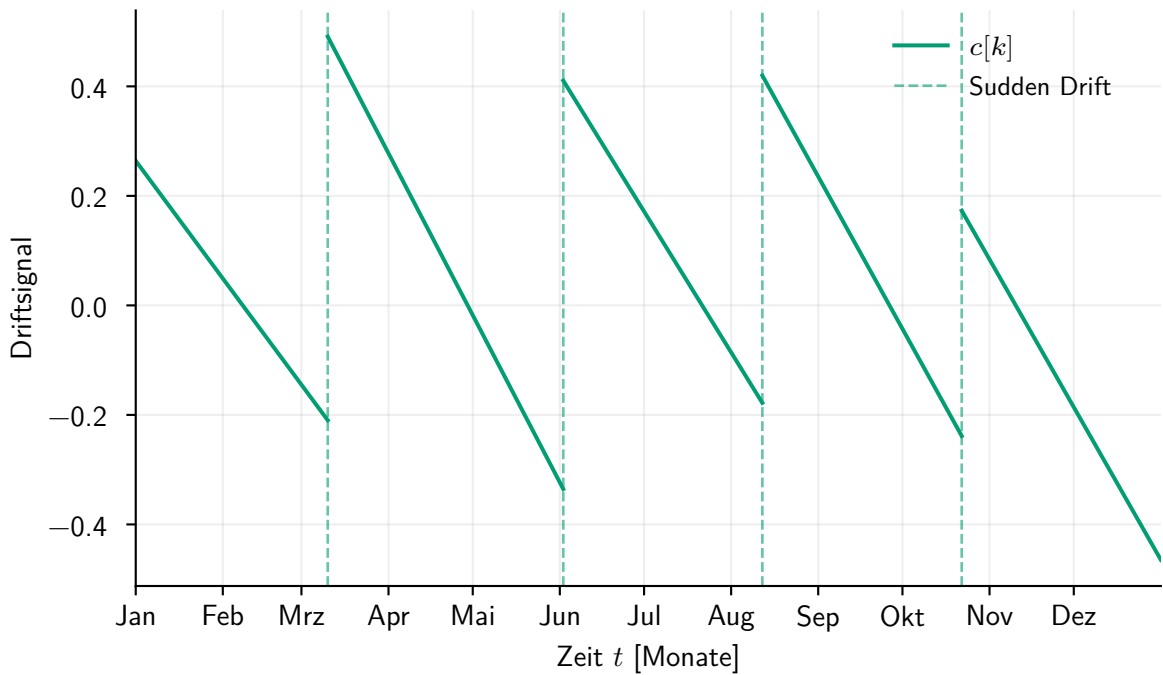


Abbildung 3.2: Verlauf des synthetischen Driftsignals $c[k]$ über ein Kalenderjahr

3.1.2 Setpoints

In industriellen Prozessen stellt der Setpoint den vom Anlagenoperator vorgegebenen Sollwert dar, auf den die beteiligten Aggregate und Regelkreise eingestellt werden. Der Setpoint definiert somit den angestrebten Betriebszustand der Anlage und wird in der Praxis abhängig von betrieblichen Zielgrößen sowie beobachteten Prozessbedingungen angepasst. Es wird angenommen, dass der Operator die Setpoints auch in Reaktion auf drift-bedingte Änderungen des Anlagenverhaltens anpasst. Folglich soll der indirekte Einfluss von CD auf die Setpoints berücksichtigt werden. Formal ergibt sich die resultierende, durch CD beeinflusste Setpoint-Zeitreihe $\tilde{w}[k]$ als Überlagerung einer Basis-Zeitreihe $w[k]$ und einer driftinduzierten Korrektur:

$$\tilde{w}[k] = w[k] + \kappa_w \cdot c_w[k], \quad k = 0, 1, \dots, K-1, \quad (3.2)$$

wobei k den diskreten Zeitindex mit Abtastzeit T_s bezeichnet, sodass $t = k \cdot T_s$. Weiter stellen κ_w den Setpoint-Driftkoeffizienten und $c_w[k]$ das segmentweise konstante Driftsignal dar. Die Basis-Setpoint-Zeitreihe $w[k]$ wird als stückweise konstante Folge definiert und stellt die exogene Anregung des Systems dar. Zwischen aufeinanderfolgenden Änderungsindizes $0 = k_0 < k_1 < \dots < k_n = K$ nimmt sie jeweils einen konstanten Wert $w[k] = s_i$ für $k \in [k_{i-1}, k_i)$, $i = 1, \dots, n$, an. Die Werte s_i werden zufällig aus einem vorgegebenen Wertebereich $(r_{\min}, r_{\max})$ gezogen. Daraus resultiert eine stückweise konstante Setpoint-Zeitreihe, die als Eingangsgröße für die Generierung von Prozesssignalen dient. Die daraus abgeleiteten Signale approximieren Betriebszustände, sodass das Systemverhalten unter sich verändernden Umgebungsbedingungen untersucht werden kann.

Die Generierung erfolgt durch das Ziehen von N_c zufälligen, eindeutigen inneren Änderungs-

indizes $k_1 < \dots < k_{N_c}$ aus dem Indexvektor $\mathcal{K} = \{1, \dots, K-1\}$, die zusammen mit den festen Rändern $k_0 = 0$ und $k_n = K$ die insgesamt $n = N_c + 1$ Indexsegmente begrenzen. Jedem Segment wird ein zufällig aus dem vorgegebenen Wertebereich gezogener Setpoint-Wert zugewiesen.

Diese Anpassung der Setpoint-Zeitreihe als indirekte Reaktion auf CD wird über das Driftsignal $c_w[k]$ modelliert. Da der Operator nur zu den Änderungsindizes eingreift, wird das Driftsignal $c[k]$ am jeweiligen Segmentanfang k_{i-1} abgetastet und über das gesamte Segment $[k_{i-1}, k_i)$ konstant gehalten (Zero-Order Hold):

$$c_w[k] = c[k_{i-1}], \quad k \in [k_{i-1}, k_i), \quad i = 1, \dots, n \quad (3.3)$$

Der Setpoint-Driftkoeffizient κ_w skaliert die Stärke der Operatorreaktion und wird relativ zur Spannweite ($r_{\max} - r_{\min}$) des Setpoint-Bereichs gewählt. Die driftinduzierte Korrektur $\kappa_w \cdot c_w[k]$ hat damit dieselbe Segmentstruktur wie $w[k]$ und ist zwischen zwei Änderungsindizes konstant. Dadurch bildet die resultierende Setpoint-Zeitreihe $\tilde{w}[k]$ sowohl die vorgegebene Anregungsstruktur ab als auch die Reaktion des Operators auf nicht-stationäre Prozessbedingungen.

Abbildung 3.3 zeigt exemplarisch eine Basis-Setpoint-Zeitreihe $w[k]$ sowie die daraus resultierende, durch CD beeinflusste Setpoint-Zeitreihe $\tilde{w}[k]$ für einen Zeitraum von einem halben Jahr. Analog zur Darstellung der Drifts in Abbildung 3.2 sind die Signale hier trotz des diskreten Charakters der Zeitreihe kontinuierlich dargestellt, um den stückweise konstanten Verlauf und die Übergänge hervorzuheben. Die Basis-Zeitreihe besitzt dabei einen Wertebereich von $(r_{\min}, r_{\max}) = (0, 5)$ sowie eine Anzahl von Setpoint-Änderungen über ein Jahr von $N_c = 500$.

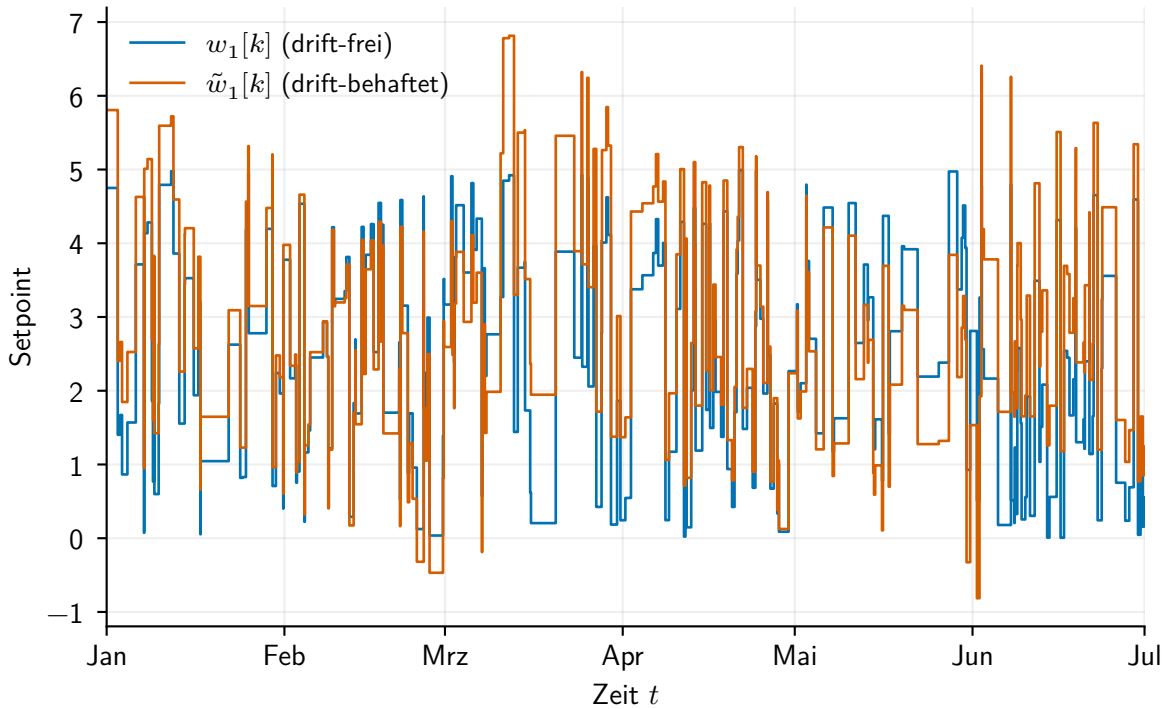


Abbildung 3.3: Einfluss von CD auf eine Setpoint-Zeitreihe über ein halbes Jahr

Die driftinduzierte Korrektur ist mit dem Driftkoeffizienten $\kappa_w = 4$ berechnet, was 80 % der Spannweite des Setpoint-Bereichs entspricht. Die Abbildung verdeutlicht, dass die durch CD beeinflusste Setpoint-Zeitreihe gegenüber der Basis-Zeitreihe systematisch verschoben wird und dabei sowohl Erweiterungen des Wertebereichs als auch veränderte Lagen der stationären Segmente auftreten.

3.1.3 Signal-Inputs

Zur Simulation der Prozesssignale wird ein lineares dynamisches System erster Ordnung (PT₁-Glied) verwendet, welches das verzögerte Anpassungsverhalten vieler technischer Systeme in kompakter Form beschreibt. Es erlaubt die Approximation von Einschwingvorgängen, wie sie beispielsweise in thermischen Systemen, Speicherprozessen oder mechanisch trägen Aggregaten auftreten. Dadurch können realistische Übergänge zwischen stationären Betriebszuständen erzeugt werden. Zwar sind die transienten Übergänge für die nachfolgende Modellierung anhand eines stationären MLP nicht unmittelbar relevant, werden aber aus Gründen der Realitätsnähe und Reproduzierbarkeit mitberechnet. Das PT₁-Glied kann dabei als Spezialfall der in Abschnitt 2.2.1 eingeführten allgemeinen Zustandsraumdarstellung interpretiert werden. Während Gleichung (2.2) jedoch eine allgemein nicht-lineare Systembeschreibung zugrunde legt, wird beim PT₁-Glied ein vereinfachtes Modell mit linearer Dynamik betrachtet. Konkret wird angenommen, dass die Zustandsänderung proportional zur Abweichung zwischen aktuellem Zustand und dem Eingangssignal ist. Daraus ergibt sich die (zeit-kontinuierliche) Differentialgleichung

$$\dot{u}(t) = -\frac{1}{T} u(t) + \frac{K_p}{T} w(t), \quad (3.4)$$

wobei T die Zeitkonstante, K_p die statische Verstärkung, $w(t)$ das Eingangssignal in Form der zeit-kontinuierlichen Setpoint-Zeitreihe und $u(t)$ den Ausgang des PT₁-Gliedes bezeichnen.

Mit dem konstanten Abtastintervall T_s und dem Zeitindex k ergibt sich so die zeitdiskrete rekursive Darstellung

$$u^o[k] = \frac{T}{T + T_s} u^o[k - 1] + \frac{K_p \cdot T_s}{T + T_s} w[k], \quad (3.5)$$

wobei $w[k]$ die diskrete Setpoint-Zeitreihe und $u^o[k]$ das resultierende rauschfreie Signal zum Zeitschritt k bezeichnen. $u^o[k]$ beschreibt damit die dynamische Antwort des PT₁-Gliedes auf die Setpoint-Zeitreihe und dient als Eingangssignal für das nachfolgend betrachtete System. Es werden $N_{\text{in}} = 2$ Eingangssignale erzeugt, die jeweils ein eigenes PT₁-Glied durchlaufen.

Im Kontext von CD wird angenommen, dass sich der Drift in einer Veränderung der dynamischen Systemeigenschaften und veränderten stationären Zuständen äußert. Die Parameter K_p und T aus Gleichung (3.5) werden dazu als zeitvariant modelliert und hängen vom CD-Wert $c[k]$ zum jeweiligen Zeitschritt ab:

$$K_p[k] = K_p \cdot (1 + c[k]), \quad T[k] = T \cdot (1 + c[k]), \quad \forall k \quad (3.6)$$

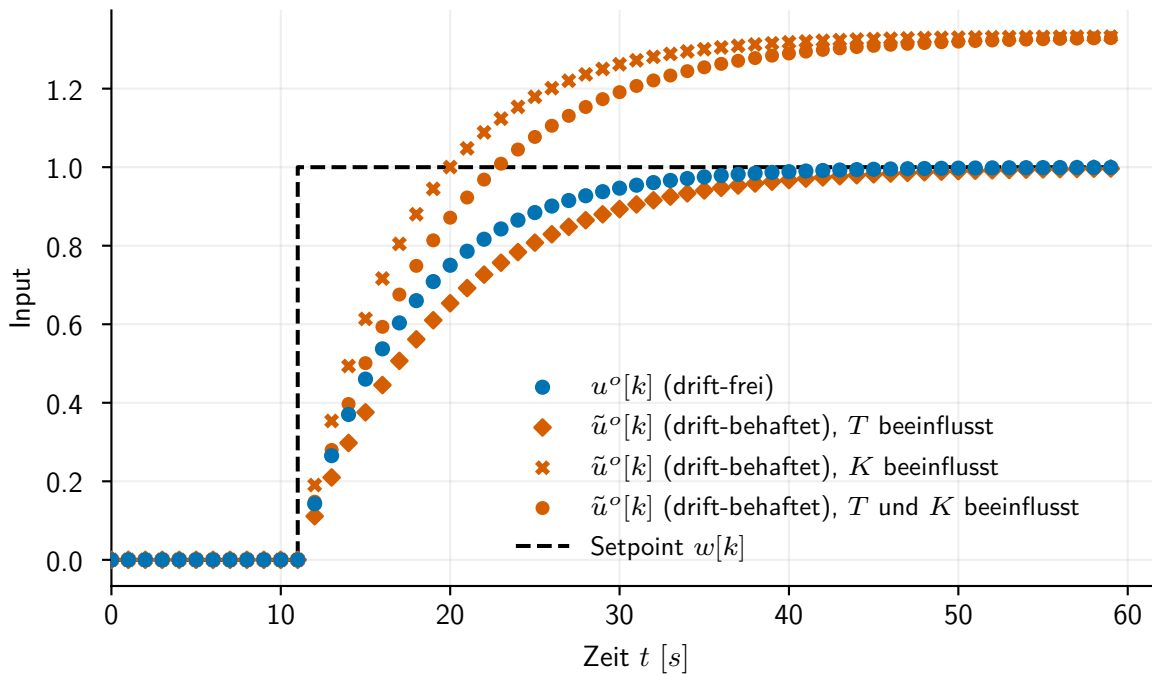


Abbildung 3.4: Qualitative Darstellung des Einflusses von CD auf ein PT_1 -Glied

Für die drift-behafteten Signale wird der Setpoint $\tilde{w}[k]$ (vgl. Abschnitt 3.1.2) als Eingangsgröße verwendet. Die resultierenden drift-behafteten Ausgangssignale der PT_1 -Glieder werden im Folgenden als $\tilde{u}_i^o[k]$ bezeichnet.

Abbildung 3.4 stellt qualitativ den Einfluss von CD auf das Verhalten des PT_1 -Gliedes dar. Während das System ohne Drift ein konstantes Einschwingverhalten aufweist, führt die drift-abhängige Variation der Parameter $T[k]$ und $K_p[k]$ zu einer veränderten Dynamik, die sich in unterschiedlichen Anstiegszeiten und veränderten stationären Werten äußert. Dabei ist der Einfluss des CD hier exemplarisch durch eine proportionale Skalierung der Parameter mit einem konstanten Driftfaktor $c = \frac{1}{3}$ modelliert.

3.1.4 Rauschen

Reale Sensordaten unterliegen typischerweise mehreren überlagerten Störquellen unterschiedlicher Charakteristik wie feinem thermischen Grundrauschen, gelegentlichen Messimpulsen sowie selteneren Ausreißereignissen. Zur realistischen Synthese des Datenstroms wird der Signal-Input daher zusätzlich mit einem Rauschterm überlagert. Ein einfaches additives weißes gaußsches Rauschen mit einer einzigen Standardabweichung σ ist dabei nicht in der Lage, diese Heterogenität adäquat abzubilden. Daher basiert das verwendete Rauschmodell auf einem *Gaussian Mixture Model* (GMM), das N_q überlagerte Rauschquellen mit unterschiedlichen Standardabweichungen und Auftretswahrscheinlichkeiten beschreibt. Die Wahrscheinlichkeits-

dichtefunktion des Rauschterms η lautet:

$$p(\eta) = \sum_{j=1}^{N_q} \nu_j \cdot \mathcal{N}(\eta \mid 0, \sigma_j^2), \quad \sum_{j=1}^{N_q} \nu_j = 1, \quad \nu_j \geq 0, \quad (3.7)$$

wobei ν_j das normierte Gewicht und σ_j die Standardabweichung der j -ten Komponente bezeichnen. $\mathcal{N}(\eta \mid 0, \sigma_j^2)$ bezeichnet dabei die Normalverteilung mit Mittelwert Null und Varianz σ_j^2 , d. h. jede Komponente modelliert mittelwertfreies Rauschen mit komponentenspezifischer Streuung. Das Modell ist zeitinvariant, d. h. die Gewichte und Standardabweichungen bleiben über alle Zeitschritte konstant. Zum Zeitschritt k ergibt sich somit der gestörte Ausgangswert $u[k]$ aus dem rauschfreien Wert $u^o[k]$ und einem unabhängig aus $p(\eta)$ gezogenen Rauschterm:

$$u[k] = u^o[k] + \eta_u[k], \quad \eta_u[k] \sim p(\eta) \quad (3.8)$$

Die konkret verwendeten Komponentenkonfigurationen für die beiden synthetisierten Eingangssignale sind in Tabelle 3.1 zusammengefasst. Abbildung 3.5 zeigt exemplarisch in Blau den

Tabelle 3.1: Parametrisierung des GMM-Rauschmodells je Eingangssignal

Eingangssignal	Komponente	σ_j	ν_j
Eingangssignal $u_1^o[k]$	Grundrauschen	0,02	0,6
	Störimpulse	0,12	0,35
	Ausreißer	0,5	0,05
Eingangssignal $u_2^o[k]$	Grundrauschen	0,003	0,9
	Störimpulse	0,02	0,08
	Ausreißer	0,2	0,02

Verlauf des Signals $u_1^o[k]$ sowie in Grau den der verrauschten Variante $u_1[k]$. Das GMM führt zu zufälligen Streuungen um das Referenzsignal, wobei sowohl im stationären Bereich als auch während des Übergangs vereinzelt größere Ausreißer auftreten.

3.1.5 Systemantwort

Die Systemantwort beschreibt grundsätzlich, wie ein technisches System auf eine Eingangsfolge reagiert und dabei eine messbare Ausgangsgröße erzeugt. Zur Modellierung stehen Ansätze unterschiedlicher Komplexität zur Verfügung. Für den vorliegenden Anwendungsfall wird ein Modell gesucht, das einen guten Kompromiss zwischen Realitätsnähe und Modellkomplexität bietet. Blockorientierte nichtlineare Modelle bieten hierfür einen geeigneten Ansatz, da sie lineare dynamische und statische nichtlineare Blöcke in Serie schalten [GB10]. Das 1930 eingeführte *Hammerstein-Modell* ordnet dabei eine statische Nichtlinearität einem LTI-Block

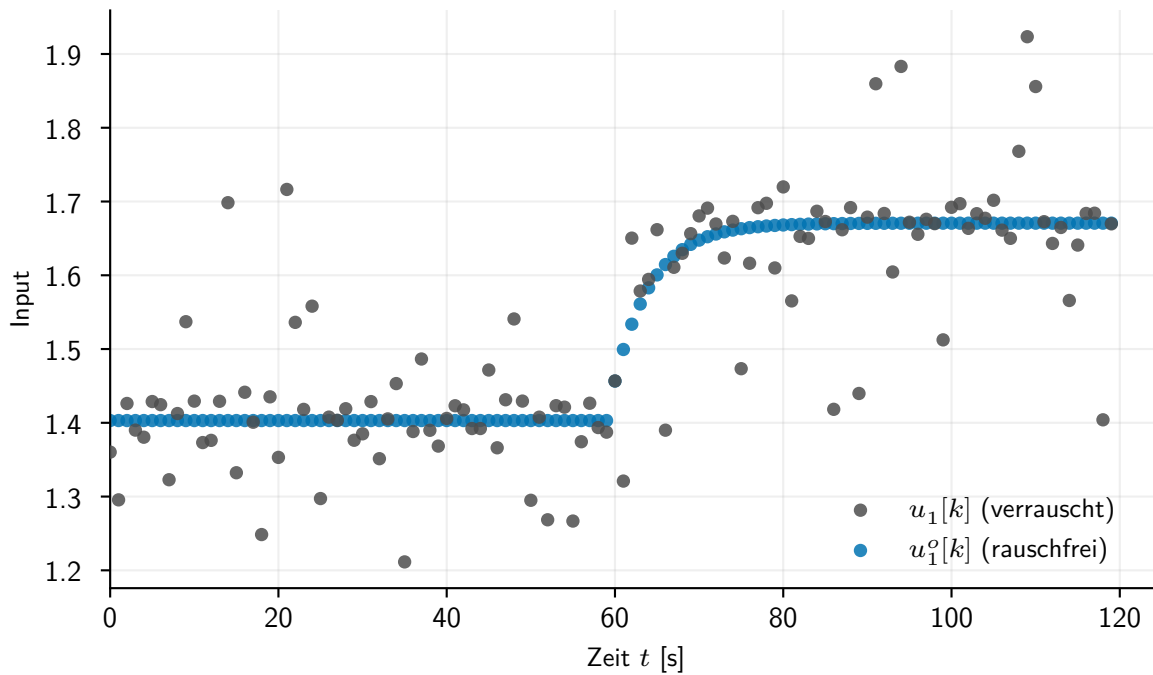


Abbildung 3.5: Zeitverlauf des PT_1 -Ausgangssignals $u_1^o[k]$ und des mit gemischtem gaußschen Rauschen überlagerten Signals $u_1[k]$ während eines transienten Übergangs

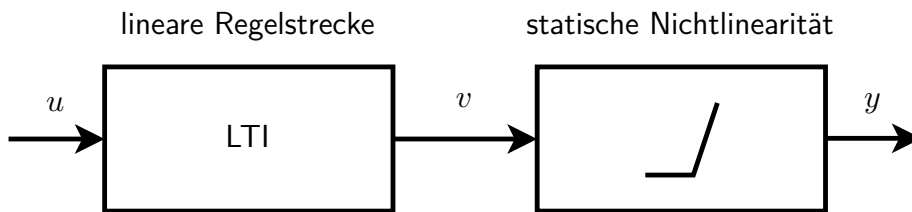


Abbildung 3.6: Struktur des Wiener-Modells

vor. Die Vertauschung dieser Blöcke führt zum 1958 eingeführten *Wiener-Modell*, bei dem zunächst der LTI-Block das Eingangssignal dynamisch formt, bevor die statische Nichtlinearität $g(\cdot)$ wirkt [SL19; Wi58]. Die Struktur des Wiener-Modells ist in Abbildung 3.6 grafisch veranschaulicht.

Das Wiener-Modell kann eine breite Klasse nichtlinearer Systeme approximieren und findet in der Praxis u. a. bei chemischen Prozessen und biologischen Systemen Anwendung [GB10]. Für den vorliegenden Aufbau ist es besonders geeignet, da der Prozess auf die bereits dynamisch geformten Eingangssignale aus Abschnitt 3.1.3 träge und nichtlinear reagieren soll und damit genau der Blockfolge des Wiener-Modells folgt. Im Wiener-Modell werden die Eingangssignale mit der Impulsantwort des LTI-Blocks gefaltet, gewichtet aufsummiert und durch die statische Nichtlinearität geführt. Formal lässt sich der Ausgang als

$$y^o[k] = g(v^o[k]), \quad v^o[k] = \sum_{i=1}^{N_{\text{in}}} \alpha_i \cdot (f * u_i^o)[k] \quad (3.9)$$

schreiben, wobei α_i die Gewichtungskoeffizienten, f die Impulsantwort des LTI-Blocks und $v^o[k]$ den rauschfreien Ausgang des linearen Teilsystems bezeichnen.

Als LTI-Block kommt ein weiteres PT_1 -Glieder gemäß Gleichung (3.5) zum Einsatz, das die Trägheit des Prozesses nachbildet und mit der Prozesszeitkonstante T_y sowie der Verstärkung $K_p = 1$ parametrisiert ist. Die Faltung lässt stationäre Werte damit unverändert und glättet lediglich die Übergänge zwischen Betriebspunkten. Die Prozessdynamik wird als drift-frei angenommen. Als statische Nichtlinearität wird eine linear-kubische Kennlinie $g(v) = v + \beta v^3$ mit $\beta > 0$ gewählt, wie sie beispielsweise progressive Federkennlinien oder Verfestigungsverhalten technischer Systeme abbildet und in der Literatur zum Wiener-Modell etabliert ist. Die Kennlinie ist streng monoton und invertierbar, sodass Drift-bedingte Verschiebungen durchgängig auf die Systemantwort wirken.

Im Fall von CD wirkt der Drift auf zwei Wegen auf die Systemantwort. Zum einen verändert sich durch die zeitvariante Parametrierung der PT_1 -Glieder (vgl. Abschnitt 3.1.3) das Verhalten der Eingangssignale. Zum anderen geht das Driftsignal, gewichtet mit dem Driftkoeffizienten κ_y , als zusätzlicher Eingang in das Modell ein und bewirkt eine drift-bedingte Verschiebung des stationären Arbeitspunkts der Ausgangsgröße. Das PT_1 -Glieder des LTI-Blocks selbst bleibt dabei unverändert, sodass dessen Zeitinvarianz gewahrt bleibt:

$$\tilde{v}^o[k] = \sum_{i=1}^{N_{in}} \alpha_i \cdot (f * \tilde{u}_i^o)[k] + \kappa_y \cdot (f * c)[k] \quad (3.10)$$

Die drift-behaftete Systemantwort ergibt sich entsprechend gemäß Gleichung (3.9) als $\tilde{y}^o[k] = g(\tilde{v}^o[k]) = \tilde{v}^o[k] + \beta (\tilde{v}^o[k])^3$.

Analog zur Vorgehensweise in Abschnitt 3.1.4 wird die Systemantwort abschließend mit additivem GMM-Rauschen überlagert. Die beobachteten, verrauschten Signale lauten

$$y[k] = y^o[k] + \eta_y[k], \quad \tilde{y}[k] = \tilde{y}^o[k] + \tilde{\eta}_y[k], \quad \eta_y[k], \tilde{\eta}_y[k] \sim p(\eta), \quad (3.11)$$

wobei $p(\eta)$ die in Gleichung (3.7) definierte Gaussian-Mixture-Dichte bezeichnet.

Abbildung 3.7 zeigt exemplarisch den Verlauf der verrauschten drift-freien Systemantwort $y[k]$ sowie der verrauschten drift-behafteten Systemantwort $\tilde{y}[k]$ in einem Zeitfenster um einen *Sudden drift*. Die Linearkombination der beiden Eingangssignale erfolgt dabei mit den Gewichtungskoeffizienten $\alpha_1 = \frac{2}{5}$, $\alpha_2 = \frac{1}{3}$ und $\beta = \frac{1}{5}$, der Driftkoeffizient beträgt $\kappa_y = \frac{3}{8}$ und die Zeitkonstante der Prozessdynamik $T_y = 3$ s. Die systematische Verschiebung des Ausgangsniveaus, die durch den direkten Drifteinfluss sowie die veränderte PT_1 -Dynamik verursacht wird, ist dabei gut zu erkennen.

3.1.6 Modellierung

Nachdem ein industrieller Prozess anhand des synthetischen Datensatzes in den vorigen Abschnitten nachempfunden wurde, soll dieser Prozess nun modelliert werden. Dazu werden drei Modellansätze auf dem drift-freien Referenzzeitraum trainiert bzw. initialisiert und anschließend

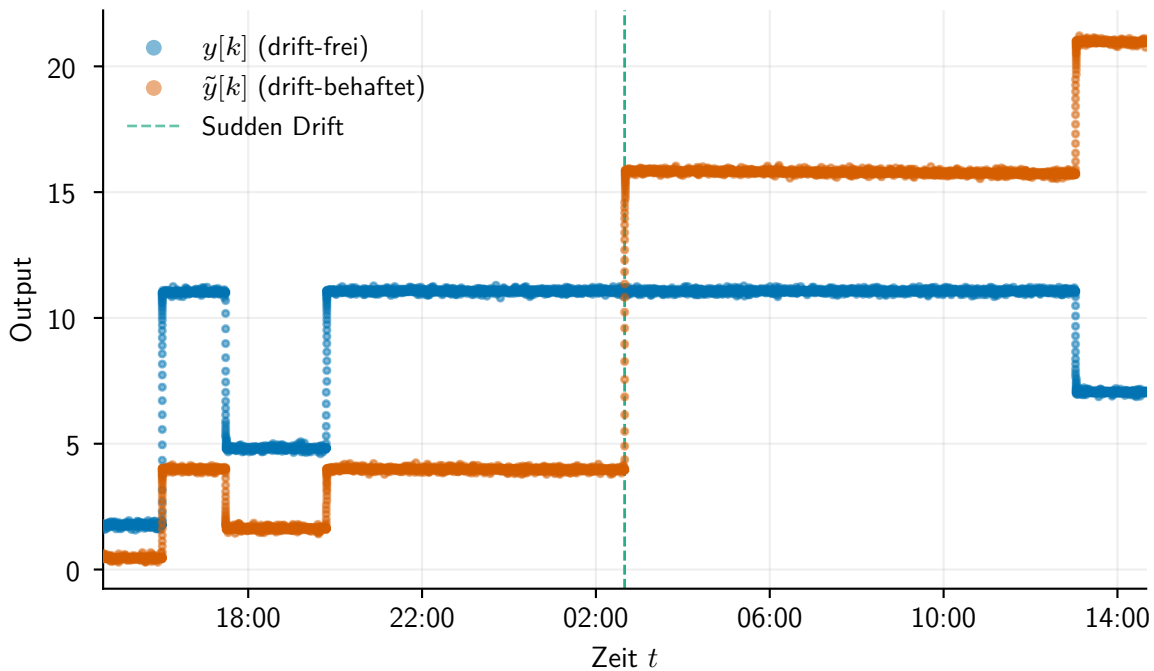


Abbildung 3.7: Vergleich der verrauschten drift-freien und drift-behafteten Systemantworten $y[k]$ und $\tilde{y}[k]$ am Übergang eines *Sudden Drifts*

auf den Datenstrom angewandt. Die Beschränkung des Trainings auf den drift-freien Zeitraum entspricht dabei der Anforderung an drift-freie Referenzdaten.

Die Auswahl der Modellansätze fußt auf den in Kapitel 2 eingeführten Architekturen. Für den Mühlen-Anwendungsfall wurde das statische MLP in Abschnitt 2.2.4 bereits als geeigneter Kompromiss identifiziert. Mit dem Ziel der Generalisierung dieses Anwendungsbeispiels auf allgemein industrielle Prozesse ist die Beibehaltung dieser Modellarchitektur naheliegend. Ergänzend werden der Kalman-Filter und der Extended Kalman-Filter (vgl. Abschnitt 2.2.2) als Vergleichsverfahren herangezogen. Beide Verfahren zählen zu den klassischen Ansätzen der rekursiven Zustands- und Parameterschätzung linearer bzw. schwach nichtlinearer dynamischer Systeme und finden in der Prozessindustrie weiterhin breite Anwendung [Ka60; WB+95]. Im Gegensatz zum MLP, das als Black-Box-Modell ohne strukturelles Vorwissen auskommt, nutzen die Filterverfahren die bekannte Struktur des zugrunde liegenden Systems dabei explizit aus. Der Vergleich erlaubt damit eine Aussage darüber, ob der drift-bedingte Leistungsabfall des Modells ein spezifisches Problem datengetriebener Black-Box-Modelle ist oder auch klassische, strukturbasierte Schätzverfahren betrifft.

Vor dem Training werden die verrauschten Signale mit einem gleitenden Mittelwert über ein Fenster von 60 s geglättet, analog zur Glättung der realen Sensordaten in Abschnitt 2.2.4. Die geglätteten Signale werden durch das Superskript f markiert, sodass $u_i^f[k]$ bzw. $\tilde{u}_i^f[k]$ und $y^f[k]$ bzw. $\tilde{y}^f[k]$ die gefilterten Inputs und Outputs darstellen. Als Trainingsdaten dient ein unabhängiger, drift-freier Referenzdatensatz, der analog zur Datenselektion im Mühlen-Anwendungsfall nach dem Latin-Hypercube-Ansatz wie in Abschnitt 2.2.3 beschrieben zur Simulation eines DOE

erzeugt wird. Die Stellgrößen werden dabei gleichmäßig über ihren Wertebereich gezogen und die quasi-stationären Betriebspunkte gemäß den Abschnitten 3.1.3, 3.1.4 und 3.1.5 simuliert. Beim Modelltraining bilden die drift-freien Eingangssignale $u_i^f[k]$ die Features und die ebenfalls drift-freie Systemantwort $y^f[k]$ das Label. Die Features werden vor dem Training, analog zum Anwendungsfall in Abschnitt 2.2.4, mittels Min-Max-Normierung auf den Wertebereich $[0, 1]$ skaliert.

Das MLP folgt der in Abschnitt 2.2.2 eingeführten Feedforward-Struktur und besteht aus zwei Hidden-Layern mit jeweils 32 Neuronen und Tanh-Aktivierung sowie einem linearen Ausgabeneuron. Diese bewusst minimale Komplexität ist an die geringe Dimensionalität des synthetischen Systems angepasst und steht im Einklang mit der Erkenntnis aus Abschnitt 2.2.4, dass Netze geringer Komplexität für derartige Regressionsaufgaben ausreichen. Das Training erfolgt mit dem Adam-Optimierer und dem MSE als Verlustfunktion über maximal 500 Epochen bei einer Batchgröße von 32. Dabei werden 20 % der Trainingsdaten zur Validierung zurückgehalten.

Als rekursiver Parameterschätzer wird der Kalman-Filter formuliert. Hierzu wird ausgenutzt, dass die linear-kubische Kennlinie $g(v) = v + \beta v^3$ streng monoton und damit eindeutig invertierbar ist. Die transformierte Messung $z[k] = g^{-1}(y^f[k])$ ist linear in den Gewichtungskoeffizienten der Linearkombination. Angewandt auf die geglätteten Signale ergibt sich mit dem Parametervektor $\theta = [\theta_1, \theta_2]^T$ und dem Regressionsvektor $\varphi[k]$ das lineare Beobachtungsmodell

$$z[k] = g^{-1}(y^f[k]) = \varphi[k]^T \theta[k] + \tilde{\zeta}_m[k], \quad \varphi[k] = [u_1^f[k], u_2^f[k]]^T, \quad (3.12)$$

wobei $\tilde{\zeta}_m[k] \approx \zeta_m[k]/g'(v[k])$ das durch die Inverse transformierte Messrauschen bezeichnet, das hier in erster Näherung als gaußsch mit konstanter Varianz R angesetzt wird. Das im Datenstrom enthaltene GMM-Rauschen wird hier also durch gaußsches Messrauschen approximiert. Als Zustandsübergang wird ein Random-Walk-Modell $\theta[k+1] = \theta[k] + \zeta_p[k]$ mit dem Prozessrauschen $\zeta_p[k] \sim \mathcal{N}(0, Q)$ und dessen Kovarianzmatrix Q angesetzt. Die Initialisierung θ_0 erfolgt durch eine Least-Squares-Schätzung auf den transformierten drift-freien Trainingsdaten. Die Messrauschvarianz R wird als Varianz der zugehörigen Residuen bestimmt.

Der EKF schätzt demgegenüber die Gewichtungskoeffizienten der Linearkombination direkt. Mit dem Parametervektor $\theta = [\theta_1, \theta_2]^T$ lautet die nichtlineare Beobachtungsfunktion

$$y^f[k] = h(\theta[k]) + \zeta_m[k], \quad h(\theta) = v[k] + \beta v[k]^3, \quad v[k] = \theta_1 u_1^f[k] + \theta_2 u_2^f[k], \quad (3.13)$$

die gemäß Abschnitt 2.2.2 im jeweiligen Arbeitspunkt über die Jacobi-Matrix $H[k] = \partial h / \partial \theta|_{\theta[k]}$ linearisiert wird. Der Zustandsübergang folgt analog zum KF einem Random-Walk-Modell. Initialisiert wird der EKF mit den nominalen Gewichtungskoeffizienten der Simulation $\theta_0 = [\alpha_1, \alpha_2]^T$ (vgl. Abschnitt 3.1.5). Die Messrauschvarianz R wird als Residualvarianz des nominalen Modells auf den drift-freien Trainingsdaten im Ausgangsraum bestimmt, da die Innovation des EKF — anders als die des KF — in der ursprünglichen Messgröße y^f vorliegt.

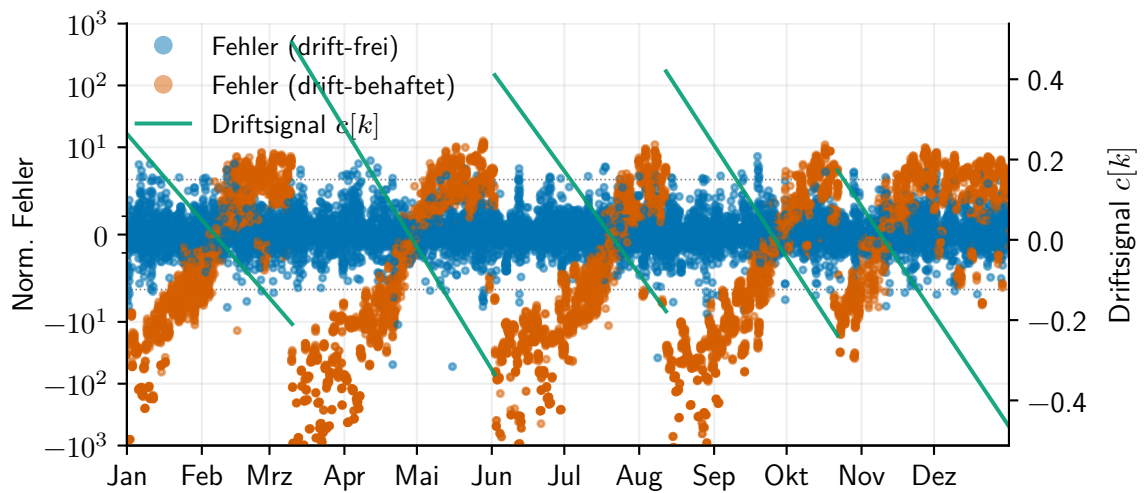
Tabelle 3.2: Initialisierung und Parametrierung der Filterverfahren

Parameter	KF	EKF
Initialschätzung θ_0	LS-Schätzung auf Trainingsdaten	$[\alpha_1, \alpha_2]^T$
Kovarianz P_0	$10^{-4} \cdot I_2$	$10^{-4} \cdot I_2$
Prozessrauschen Q	$10^{-10} \cdot I_2$	$10^{-10} \cdot I_2$
Messrauschen R	Residualvarianz der LS-Schätzung	Residualvarianz im Ausgangsraum

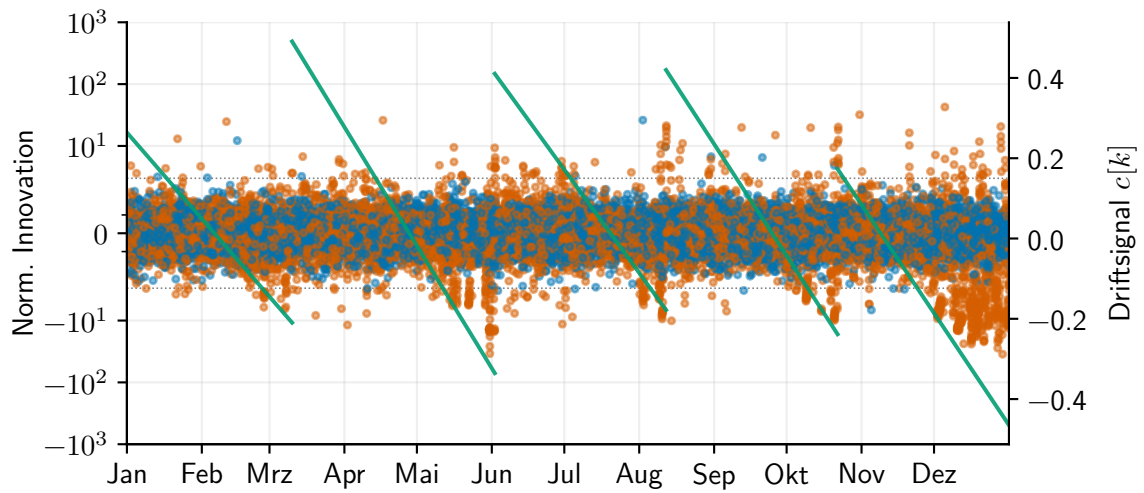
Die vollständige Parametrierung beider Filterverfahren ist in Tabelle 3.2 zusammengefasst. Die Prozessrauschkovarianz Q wird dabei bewusst sehr klein gewählt, sodass die Parameter nach der Initialisierung nahezu konstant bleiben und beide Filter als quasi-statische Referenzmodelle agieren.

Zur Evaluation werden alle drei Modelle auf einen Zeitraum der ersten sechs Monate des Datenstroms angewandt, sowohl auf den drift-freien als auch auf den drift-behafteten Signalen. Als Vergleichsgröße dient beim MLP der Simulationsfehler $\hat{y}[k] - y^f[k]$, bei den Filterverfahren die Innovation, also die Abweichung zwischen Messung und Prädiktion vor dem Korrekturschritt. Beide Größen messen, wie gut das jeweilige Modell die beobachtete Systemantwort beschreibt. Um die drei Ansätze in einer gemeinsamen Darstellung gegenüberstellen zu können, werden Simulationsfehler und Innovationen je Modell und Szenario mittelwertbereinigt und auf Einheitsvarianz skaliert. Die Innovation des KF liegt dabei im transformierten Messraum $z = g^{-1}(y)$; durch die abschließende Skalierung auf Einheitsvarianz bleiben die drei Ansätze dennoch vergleichbar.

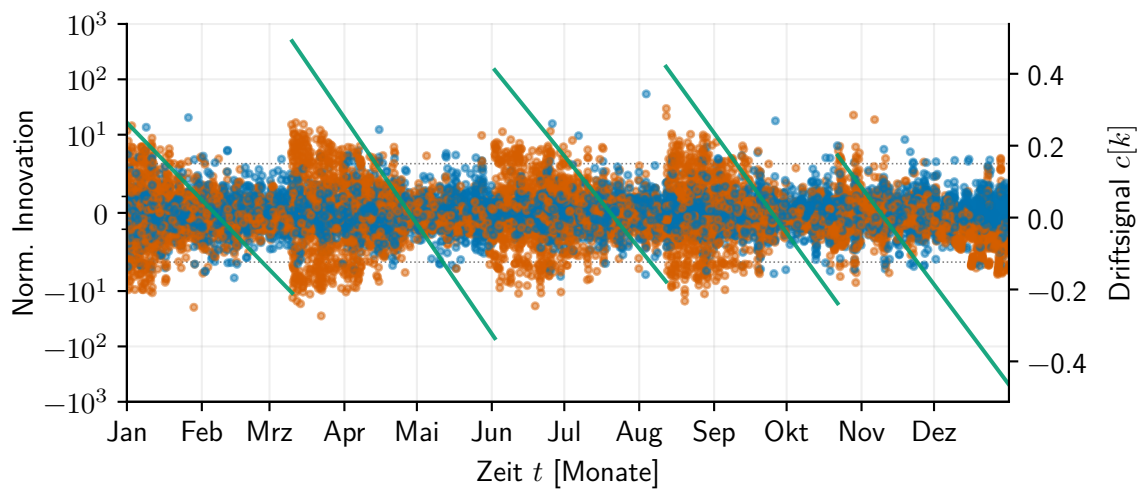
Abbildung 3.8 stellt für das MLP (a), den KF (b) und den EKF (c) den normierten Simulationsfehler bzw. die normierte Innovation auf dem drift-freien (blau) und dem drift-behafteten (orange) Teststrom dar, jeweils überlagert mit dem Driftsignal $c[k]$ (grün). Auf dem drift-freien Datenstrom verbleiben die Fehler aller drei Modelle als mittelwertfreies, strukturloses Band um Null. Dies bestätigt die korrekte, drift-freie Initialisierung. Auf dem drift-behafteten Datenstrom hingegen folgt der Fehlerverlauf bei allen drei Ansätzen der Sägezahnstruktur des Driftsignals. Der *Incremental Drift* innerhalb der Segmente äußert sich in einem mit dem Driftbetrag $|c[k]|$ systematisch anwachsenden Fehlerbetrag, während der *Sudden Drift* an der Segmentgrenze einen sprunghaften Wechsel des Fehlerniveaus verursacht. Dementsprechend korreliert der Betrag des Fehlers im drift-behafteten Fall mit dem Betrag des Driftsignals, und die zuvor um Null zentrierten Residuen nehmen einen systematischen Bias an. Quantitativ bestätigt sich diese Beobachtung: Der RMSE des Fehlers bzw. der Innovation steigt bei allen drei Modellen vom drift-freien zum drift-behafteten Fall deutlich an (siehe Tabelle 3.3). Die Degradation betrifft damit alle drei Modellklassen in vergleichbarer Größenordnung.



(a) Statisches MLP



(b) Kalman-Filter



(c) Extended Kalman-Filter

Abbildung 3.8: Normierter Simulationsfehler bzw. normierte Innovationen der drei Modellansätze auf dem drift-freien und dem drift-behafteten Datenstrom im Vergleich zum Driftsignal $c[k]$

Tabelle 3.3: Kennzahlen der drift-bedingten Modellverschlechterung

Modell	RMSE		Streuung σ		Mittelwert $\bar{e}$ (drift-behaftet)
	drift-frei	drift-behaftet	drift-frei	drift-behaftet	
MLP	0,065	15,090	0,064	14,406	−4,492
KF	0,009	0,015	0,009	0,015	−0,001
EKF	0,061	0,157	0,061	0,157	−0,001

Worin sich die drei Ansätze hingegen unterscheiden, ist die Zusammensetzung dieses Fehleranstiegs, die sich anhand der visuellen Darstellung sowie der Kennzahlen in Tabelle 3.3 erkennen lässt. Beim MLP entsteht ein gerichteter, systematischer Fehler. Der Fehler im drift-behafteten Fall nimmt einen deutlich von Null verschiedenen Mittelwert $\bar{e}$ an, der dem Drift in einer Vorzugsrichtung folgt. Bei KF und EKF bleibt die Innovation demgegenüber nahezu mittelwertfrei. Die Verschlechterung äußert sich hier überwiegend in einer wachsenden, zeitlich aber ungeordneten Streuung σ . Beim MLP korreliert also der Fehler selbst richtungstreu mit $c[k]$ und verschiebt sich mit dem Drift, während bei den Filterverfahren nicht die Innovation selbst, sondern deren Streuung mit $c[k]$ korreliert. Die Schwere des Drifts schlägt sich bei den Filtern somit in der Breite und nicht in der Lage der Innovationsverteilung nieder.

Erwartungsgemäß können auch die Filterverfahren den Drift mit ihrer quasi-statischen Parametrierung nicht kompensieren. Der drift-bedingte Leistungsabfall des Modells ist somit kein spezifisches Defizit datengetriebener Black-Box-Modelle, sondern eine grundsätzliche Eigenschaft statischer Modelle unter CD. Daraus kann gefolgert werden, dass der synthetische Datenstrom die beabsichtigte Degradation reproduziert und sich als kontrollierte Testumgebung für die Evaluation von Detektions- und Adaptionsmechanismen eignet.

3.1.7 Drift-Erkennung

Zur Detektion des in Abschnitt 3.1.1 generierten CD werden die in Abschnitt 2.3.2 eingeführten Performance-basierten Drift-Detektoren herangezogen. Als Eingangssignal der Detektion dient der normierte Simulationsfehler des in Abschnitt 3.1.6 eingeführten MLP, dessen Architektur auch im ausgehenden Anwendungsfall der Mühle verwendet wird. Konkret kommen als Detektionsverfahren die beiden Fenster-basierten Verfahren *KSWIN* und *ADWIN* sowie die beiden auf statistischer Prozesskontrolle beruhenden Verfahren *DDM* und *EDDM* zum Einsatz.

Wie bereits in Abschnitt 2.3.2 beschrieben, verarbeiten die Fenster-basierten Verfahren eine kontinuierliche Größe und verwenden daher hier den Betrag des Simulationsfehlers $|e[k]|$. DDM und EDDM hingegen basieren auf Fehlerraten und sind ursprünglich auf binäre Fehlersignale

ausgelegt, wie sie bei Klassifikationsaufgaben durch korrekte oder falsche Modellentscheidungen entstehen. Ein solches binäres Fehlersignal existiert für die vorliegende Regressionsaufgabe jedoch nicht, sodass der kontinuierliche Fehler vor der Auswertung binarisiert werden muss. Dazu wird ein Schwellenwert λ_{bin} eingeführt, der den Fehler in einen korrekten ($|e[k]| \leq \lambda_{\text{bin}}$) und einen fehlerhaften Zustand ($|e[k]| > \lambda_{\text{bin}}$) übersetzt. Da der Fehler in Abschnitt 3.1.6 auf Einheitsvarianz normiert wurde, entspricht der Schwellenwert einem Vielfachen der Standardabweichung.

Ein wesentlicher Unterschied zum Mühlen-Anwendungsfall besteht darin, dass der synthetische Datenstrom die Drift-Zeitpunkte als *Ground Truth* bereitstellt. Aus $c[k]$ werden dazu zwei Referenzgrößen abgeleitet: die Menge der *Sudden Drifts* als Indizes der sprungartigen Anstiege an den Segmentgrenzen sowie eine Unterscheidung zwischen drift-aktiven und drift-inaktiven Phasen. Obwohl sich $c[k]$ kontinuierlich verändert, gilt ein Zeitschritt als drift-inaktiv (bzw. *stabil*), wenn die Driftamplitude unter 10 % ihres Maximalwerts bleibt:

$$|c[k]| < 0,1 \cdot \hat{c}, \quad \hat{c} = \max_k |c[k]|, \quad (3.14)$$

andernfalls gilt der Zeitschritt als drift-aktiv. Auf dieser Grundlage wird die Detektionsgüte über drei Kennzahlen erfasst:

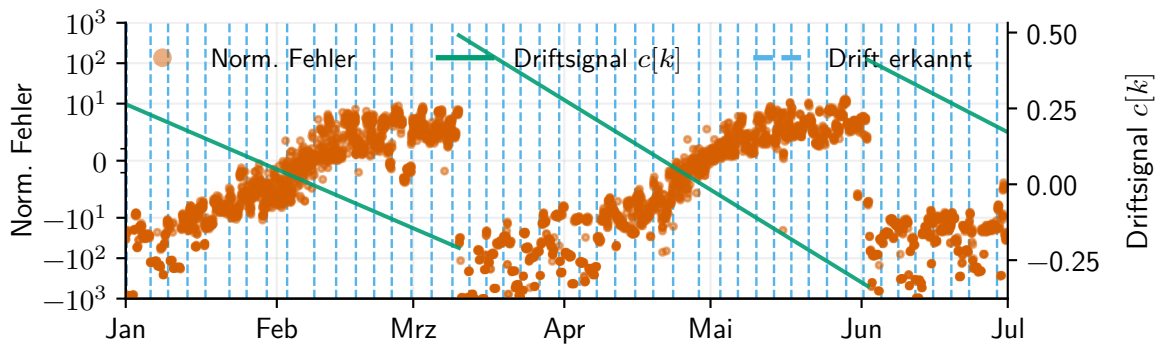
- der *Recall* ρ der Sudden Drifts, also der Anteil der erkannten Sudden Drifts (innerhalb eines Toleranzfensters von $n_{\text{tol}} = 1000$ Abtastschritten)
- der mittlere *Detektionsverzug* $\bar{d}$ zwischen Sprung und zugehöriger Detektion
- die Zahl N_{fa} der *Fehlalarme* in stabilen Phasen

Diese drei Kennzahlen werden zu einer skalaren Zielfunktion

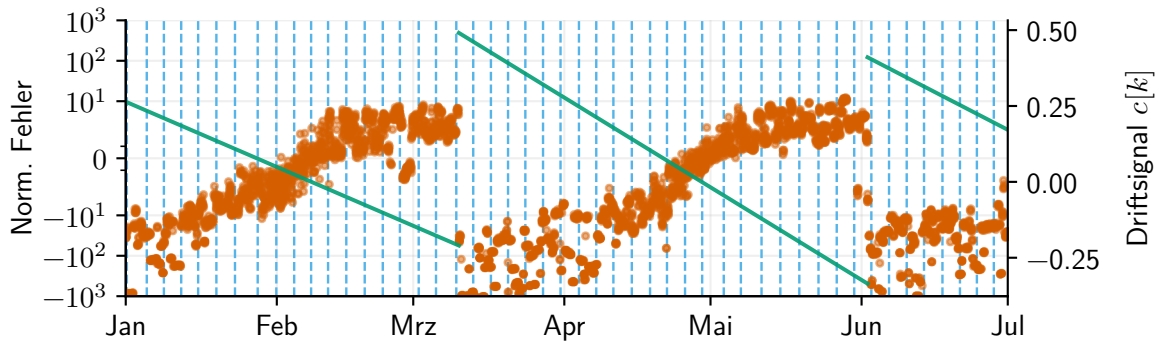
$$J_d = \omega_\rho (1 - \rho) + \omega_d \frac{\bar{d}}{n_{\text{tol}}} + \omega_{\text{fa}} \frac{N_{\text{fa}}}{N_{\text{sd}}} \quad (3.15)$$

zusammengefasst, wobei N_{sd} die Zahl der Sudden Drifts bezeichnet. Die Gewichte werden mit $\omega_\rho = 3,0$, $\omega_d = 0,3$ und $\omega_{\text{fa}} = 0,05$ so gewählt, dass eine vollständige Detektion der Sudden Drifts gegenüber Verzug und Fehlalarmen priorisiert wird.

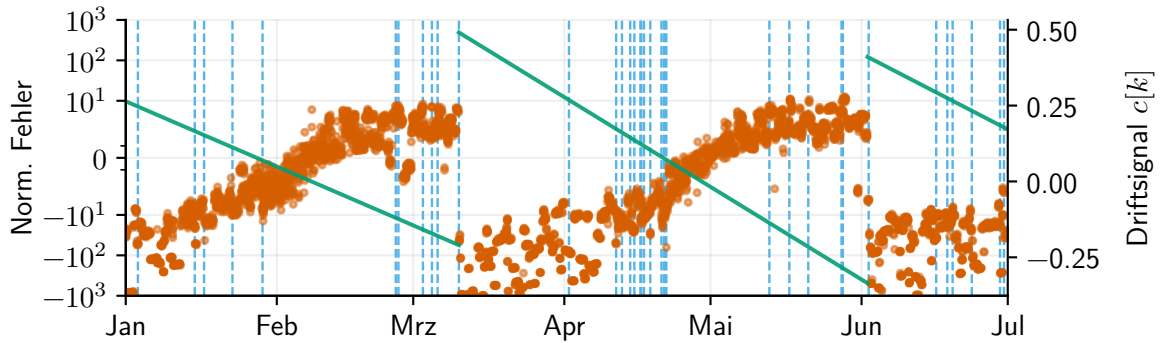
Abbildung 3.9 stellt die erkannten Drifts der vier Parameter-optimierten Detektoren (a) KSWIN, (b) ADWIN, (c) DDM und (d) EDDM dem Driftsignal $c[k]$ gegenüber. Die zugehörigen Kennzahlen fasst Tabelle 3.4 zusammen. Ausgewertet wird der gesamte Datenstrom über ein Kalenderjahr, der an seinen Segmentgrenzen vier Sudden Drifts aufweist (vgl. Abbildung 3.2). Die Abbildung zeigt davon exemplarisch die ersten sechs Monate. Eine Besonderheit der Fenster-basierten Verfahren ist, dass sie den fortlaufenden *Incremental Drift* innerhalb eines Segments nicht als einmaliges Ereignis, sondern kontinuierlich als wiederholte Verteilungsänderung detektieren. Dies führt zu einer dichten Folge von Detektionen über die gesamte aktive Driftphase. Für die grafische Darstellung wird daher für KSWIN und ADWIN eine Refraktärzeit von 5000 Abtastschritten eingeführt, innerhalb derer nach einer Detektion weitere Detektionen



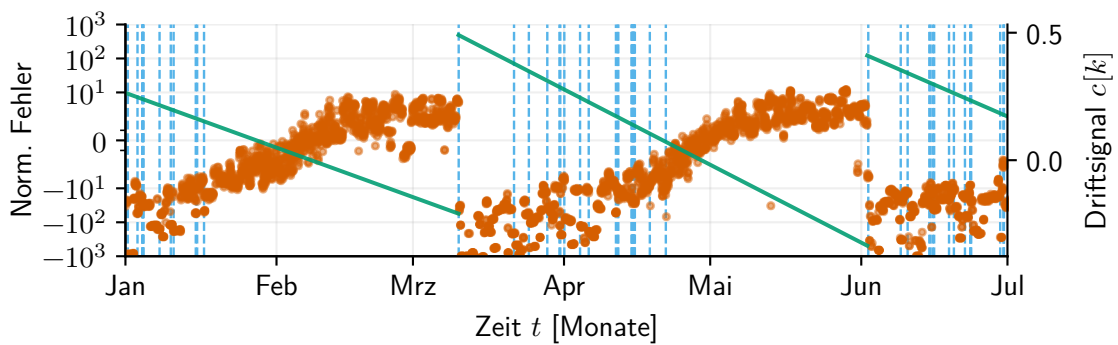
(a) KSWIN



(b) ADWIN



(c) DDM



(d) EDDM

Abbildung 3.9: Erkannte Drifts verschiedener Detektionsmethoden auf dem normierten Simulationsfehler im Vergleich zum Driftsignal $c[k]$

Tabelle 3.4: Kennzahlen der vier Drift-Detektoren auf dem normierten Simulationsfehler des drift-behafteten Stroms

Detektor	Recall ρ	Verzug $\bar{d}$	Fehlalarme N_{fa}	Detektionen
KSWIN	1,00	145	42	854
ADWIN	1,00	148	99	1017
DDM	1,00	329	0	72
EDDM	1,00	332	0	89

unterdrückt werden. DDM und EDDM bleiben davon unberührt. Die in Tabelle 3.4 dargestellten Kennzahlen sind jedoch auf der vollständigen Detektionsfolge ohne Refraktärzeit berechnet.

Auswertend lässt sich festhalten, dass KSWIN und ADWIN sämtliche Segmentsprünge zuverlässig und mit geringem Verzug erkennen (vgl. Tabelle 3.4). Zugleich zeichnen sie den fortlaufenden Incremental Drift nach, was sich ohne Refraktärzeit in einer Vielzahl von Einzeldetektionen äußert. Mit Refraktärzeit reduziert sich diese auf wenige Detektionen, die die aktiven Drifts zuverlässig markieren. Diese hohe Sensitivität geht jedoch mit vergleichsweise vielen Fehlalarmen in stabilen Phasen gemäß (3.14) einher.

DDM und EDDM verhalten sich demgegenüber deutlich konservativer. Während DDM sämtliche Segmentsprünge erfasst, verfehlt EDDM den schwächsten Sudden Drift, sodass dessen Recall hinter dem der übrigen Verfahren zurückbleibt. Zugleich markieren beide den fortlaufenden Incremental Drift deutlich spärlicher und lösen in stabilen Phasen gemäß (3.14) keine Fehlalarme aus. Dieses Verhalten ist eine unmittelbare Folge der Binarisierung: Da nur Fehlerbeträge oberhalb von λ_{bin} einfließen, reagieren beide Verfahren vorrangig auf ausgeprägte Fehlerexkursionen und bleiben gegenüber schwächeren Änderungen vergleichsweise unempfindlich.

Insgesamt erfassen die Fenster-basierten Verfahren KSWIN und ADWIN sämtliche Sudden Drifts sowie den Incremental Drift fortlaufend, was eine hohe Reaktionsfähigkeit ermöglicht, jedoch mit zahlreichen Fehlalarmen einhergeht. Die Fehlerraten-basierten Verfahren DDM und EDDM liefern demgegenüber eine spärlichere, fehlalarmfreie Detektion, wobei EDDM einen *Sudden Drift* verfehlt. Welches Verhalten vorzuziehen ist, hängt damit von der angestrebten Adaptionsstrategie ab.

3.1.8 Drift-Adaption

Nachdem der CD im vorigen Abschnitt detektiert werden konnte, besteht der abschließende Schritt der Verarbeitungskette darin, das in Abschnitt 3.1.6 eingeführte MLP an den drift-

behafteten Prozess zu adaptieren und damit der Modellverschlechterung entgegenzuwirken. Methodisch werden drei Varianten dem drift-behafteten Modell gegenübergestellt:

- eine *passive* Adaption
- eine *aktive* Adaption
- deren *Kombination*

Passive und aktive Adaption sowie deren Kombination werden im Folgenden zunächst definiert und anschließend methodisch ausgestaltet.

Allen adaptiven Strategien liegt ein inkrementelles Nachtraining des MLP zugrunde. Anstatt das Modell vollständig neu zu trainieren, wird das bestehende Netz schrittweise an die veränderten Zusammenhänge angepasst. Dabei werden ausgewählte Layer des Netzes eingefroren, während die verbleibenden Schichten mit kleiner Lernrate über wenige Epochen mit dem Adam-Optimierer und dem MSE als Verlustfunktion nachtrainiert werden. Ein Early-Stopping-Kriterium bricht das Training bei ausbleibender Verbesserung vorzeitig ab und begrenzt so das Risiko eines Overfitting an einen kurzen, lokalen Datenausschnitt. Zusätzlich wird jedes Nachtraining validiert: Senkt es den MSE auf dem eigenen Trainingsfenster nicht, wird das Update verworfen und der vorherige Gewichtsstand wiederhergestellt. Das Einfrieren von Layern, die kleine Lernrate sowie das Early-Stopping wirken dabei dem in Abschnitt 2.3.3 beschriebenen *Stability-Plasticity-Dilemma* entgegen, indem sie die Plastizität gezielt begrenzen. Im Gegensatz zum initialen Training auf dem drift-freien Referenzzeitraum (vgl. Abschnitt 3.1.6) werden für die Adaption die drift-behafteten Eingangssignale $\tilde{u}_i^f[k]$ als Features und die ebenfalls drift-behaftete Systemantwort $\tilde{y}^f[k]$ als Label verwendet. Die kontinuierliche Verfügbarkeit des Labels entspricht dabei dem Mühlen-Anwendungsfall, in dem die Zielgröße sensorisch fortlaufend erfasst wird.

Zur Adaption wird der Strom auf ein gröberes Abtastraster mit dem Adaptionsintervall $T_a = 600\text{ s}$ reduziert, was 52 560 Datenpunkten entspricht, und anschließend chunk-weise verarbeitet. Ein *Chunk* bezeichnet in dem Zusammenhang einen zusammenhängenden Abschnitt des Datenstroms fester Länge – hier etwa einen Tag bzw. 144 Datenpunkte – und bildet die Einheit, auf der über eine Anpassung entschieden wird. Der Strom wird somit nicht punktweise, sondern blockweise in solchen Chunks durchlaufen. Innerhalb eines Chunks wird der Strom zunächst in einem Durchgang prädiziert und daraus der Simulationsfehler bestimmt. Anschließend werden die Strategie-spezifischen Bedingungen geprüft und nur bei Bedarf das Nachtraining ausgelöst. Die Verarbeitung folgt durchgängig dem prequentiellen Prinzip (*test-then-train*). Jeder Datenpunkt wird dabei zunächst prädiziert und fließt erst danach in ein etwaiges Nachtraining ein. Trainingsfenster greifen hier zu keinem Zeitpunkt auf zukünftige Datenpunkte zu.

Die *passive* Adaption setzt die in Abschnitt 2.3.3 eingeführte *blinde* Anpassung um. Sie ist rein proaktiv und benötigt weder Fehlerauswertung noch Detektor. In festen Abständen von

N_p Punkten wird das Modell auf dem gleitenden Fenster der letzten N_w Punkte nachtrainiert und so eine kontinuierliche Grundadaption bereitgestellt, deren Eingriffsdichte allein über die Periode N_p gesteuert wird.

Die *aktive* Adaption koppelt das Nachtraining hingegen an die Drift-Detektion aus Abschnitt 3.1.7. Erst ein erkannter Drift löst eine Anpassung aus, die dadurch zielgerichtet, zugleich aber von der Detektionsqualität abhängig ist.

Die *kombinierte* Strategie soll beide Mechanismen zusammenführen, wobei die periodische passive Komponente als Rückfallebene (*Backstop*) wirkt. Sie löst nur aus, wenn im selben Zeitraum keine detektor-getriggerte Anpassung erfolgt ist. Verpasst der Detektor eine Veränderung, fängt die periodische Komponente sie auf.

Da die aktive Strategie von der Wahl des Detektors abhängt, wird dieser nicht manuell festgelegt, sondern anhand eines Screenings bestimmt. Als Trigger stehen die vier statistischen Detektoren KSWIN, ADWIN, DDM und EDDM sowie ein Schwellwert-Detektor zur Auswahl. Der Schwellwert-Detektor bildet die performance-basierte Erkennung (Abschnitt 2.3.2) in ihrer einfachsten Form ab, indem er anschlägt, sobald der über ein gleitendes Fenster berechnete RMSE des Simulationsfehlers $e[k]$ einen festgelegten Schwellwert überschreitet. Für jeden Trigger wird ein vollständiger Adaptionslauf durchgeführt. Derjenige Trigger mit dem kleinsten resultierenden RMSE wird dann als aktive Strategie ausgewählt und als aktive Komponente der kombinierten Strategie übernommen. Die Anforderungen an den Detektor unterscheiden sich dabei von der reinen Detektions-Evaluation aus Abschnitt 3.1.7, denn im geschlossenen Kreis arbeitet er auf dem Simulationsfehler $e[k]$ des sich fortlaufend ändernden Modells, dessen Niveau sich mit jeder Anpassung verschiebt. Fehlerraten-basierte Verfahren wie DDM oder EDDM setzen dazu den Betrag des Simulationsfehlers $|e[k]|$ über die in Abschnitt 3.1.7 eingeführte Schwelle λ_{bin} in ein binäres Richtig/Falsch-Signal um. Diese Schwelle ist am Fehlerniveau des statischen Modells festgelegt. Sobald die laufende Adaption dieses Niveau absenkt, passt sie nicht mehr, und der Detektor arbeitet außerhalb seines vorgesehenen Betriebspunkts. Die Fenster-basierten Verfahren KSWIN und ADWIN vergleichen dagegen ein aktuelles Detektionsfenster mit einem mitlaufenden Referenzfenster und kommen ohne fest kalibrierten Schwellwert aus, sodass sie vom absoluten Fehlerniveau kaum betroffen sind. Um für alle Verfahren einen passenden Betriebspunkt zu erhalten, werden die Detektorparameter nicht aus der reinen Detektion übernommen, sondern im geschlossenen Adaptionskreis neu abgestimmt.

Wird bei der aktiven Strategie zum Zeitschritt k_d ein Drift erkannt, erfolgt unmittelbar ein Nachtraining auf dem zurückliegenden Fenster $[k_d - n_-, k_d]$. Würde bis zum Nachtraining zunächst n_+ Punkte abgewartet um dem Nachtraining mehr Daten zur Verfügung zu stellen, entstünde eine Totzeit, in der der bereits detektierte Drift unberücksichtigt bliebe. Stattdessen wird das Modell während einer Nachführphase der Länge n_+ nachgeführt, das heißt am Ende jedes Chunks durch ein kurzes inkrementelles Nachtraining auf den zuletzt eingetroffenen Datenpunkten an die fortschreitende Veränderung angepasst. Detektionen innerhalb einer

laufenden Phase verlängern diese lediglich, ohne ein weiteres Sofort-Training auszulösen. Der Eingriffsaufwand ist damit von der Detektionsdichte entkoppelt. Auch ein fenster-basiertes Verfahren, das den Incremental Drift als dichte Folge von Einzeldetektionen nachzeichnet (vgl. Abschnitt 3.1.7), wirkt so als Ereignis-Trigger.

Zwischen zwei Sofort-Trainings wird zudem eine Refraktärzeit als Vielfaches der mittleren Dauer stationärer Betriebsphasen erzwungen, die den Eingriffsaufwand begrenzt.

Die Bewertung der Strategien erfolgt über den Simulationsfehler auf dem drift-behafteten Strom über das gesamte Kalenderjahr. Als Kennzahlen dienen der RMSE, der Mittelwert $\bar{e}$ und die Streuung σ des Fehlers, die Korrelation des Fehlerbetrags $|e|$ mit dem Driftbetrag $|c[k]|$ sowie die Anzahl der ausgelösten Nachtrainings.

Letztere quantifiziert den Eingriffsaufwand der jeweiligen Strategie. Da das inkrementelle Nachtraining stochastisch ist, wird die Auswertung zusätzlich über mehrere Läufe mit unterschiedlichen Seeds wiederholt und als Mittelwert und Streuung des RMSE zusammengefasst. Die Seed-Streuung dient dabei als Maß für die Reproduzierbarkeit einer Strategie.

Die Wirksamkeit der adaptiven Strategien hängt maßgeblich von ihren Hyperparametern ab. Diese werden daher analog zur Optimierung der Detektoren in Abschnitt 3.1.7 je Strategie mit dem Optimierungsverfahren `Optuna` eingestellt. Im Unterschied zur Detektion erfordert jeder Versuch hier einen vollständigen Adaptionslauf mit wiederholtem Nachtraining und ist damit rechenintensiver. Das Tuning erfolgt dabei auf einem gröber abgetasteten Strom mit $T_a = 3600$ s. Anders als bei der Detektion wird keine *Ground Truth* der Drifts benötigt, da die gemessene Systemantwort als Label vorliegt und der RMSE direkt bewertet werden kann. Die Zielfunktion kombiniert den auf den Fall ohne Adaption normierten RMSE mit einem Strafterm für die Zahl der Nachtrainings, um den Eingriffsaufwand zu begrenzen:

$$J_a = \omega_r \frac{\text{RMSE}}{\text{RMSE}_{\text{base}}} + \omega_t \frac{N_{\text{train}}}{N_{\text{chunk}}} , \quad (3.16)$$

mit den Gewichten $\omega_r = 1,0$ und $\omega_t = 0,1$, wobei $\text{RMSE}_{\text{base}}$ den RMSE der Baseline, N_{train} die Zahl der Nachtrainings und N_{chunk} die Zahl der Chunks bezeichnet.

Optimiert werden die Lernrate, die Epochenzahl, das Freeze-Muster, das Reset-Verhalten, die strategie-spezifischen Fenster- und Schwellenparameter sowie die Refraktärzeit. Drei Maßnahmen sichern die Übertragbarkeit der auf dem gröber abgetasteten Tuning-Strom gefundenen Konfigurationen auf die Zielauflösung ab:

- Die Refraktärzeit wird nicht als absolute Punktzahl, sondern als dimensionsloses Vielfaches der mittleren Dauer stationärer Betriebsphasen optimiert, sodass derselbe Wert im Tuning und in der Anwendung dieselbe physikalische Dauer bezeichnet und ohne Umskalierung übertragbar ist.
- Die kombinierte Strategie erhält die Optima der beiden Einzelstrategien als Startkandi-

Tabelle 3.5: Kennzahlen der vier Adaptionstrategien auf dem drift-behafteten Strom

Strategie	RMSE	Mittelwert $\bar{e}$	Streuung σ	Korr. $ e , c $	Nachtrainings
Baseline	1,663	0,401	1,614	0,37	0
Passiv	0,669	0,058	0,666	0,22	121
Aktiv	0,771	0,077	0,767	0,27	132
Kombiniert	0,694	0,059	0,692	0,23	206

daten der Optimierung.

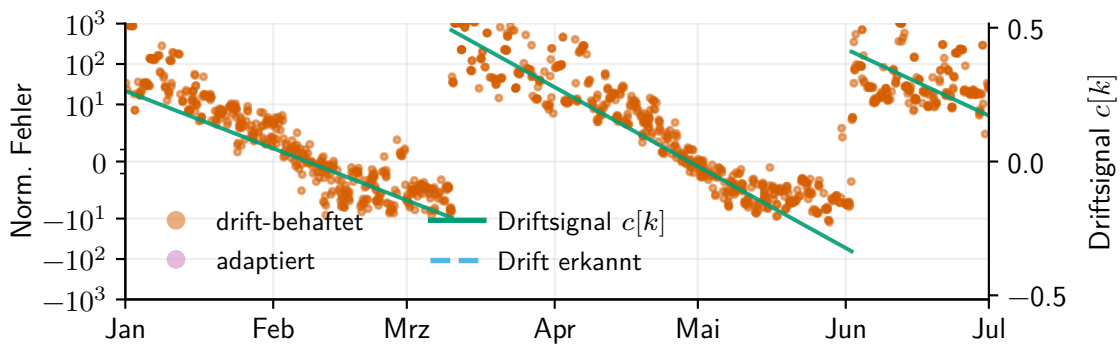
- Die besten Konfigurationen aller drei Strategien werden abschließend auf dem voll aufgelösten Strom gegengeprüft, wobei je Strategie die dort beste Konfiguration übernommen wird.

Das Freeze-Muster, das ausschließlich das Output-Layer freigibt, ist für die Detektor-gestützten Strategien vom Suchraum ausgeschlossen.

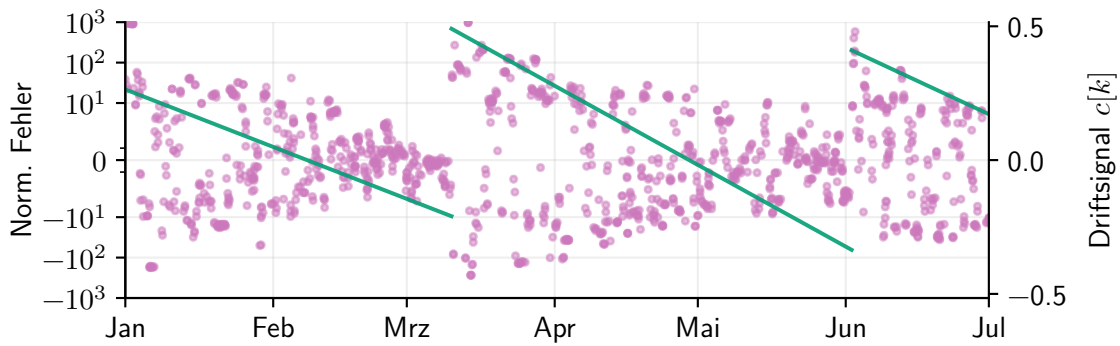
Abbildung 3.10 stellt den normierten Simulationsfehler der vier Strategien dem Driftsignal $c[k]$ gegenüber. Die einzelnen Teilabbildungen zeigen für die Baseline ohne Adaption (a), die passive (b), die aktive (c) und die kombinierte Strategie (d) jeweils den Fehlerverlauf über sechs Monate, wobei in den Detektor-gestützten Varianten zusätzlich die erkannten Drifts markiert sind. Die zugehörigen Kennzahlen fasst Tabelle 3.5 zusammen. Ohne Adaption reproduziert das Modell erwartungsgemäß die in Abschnitt 3.1.6 beschriebene Degradation; der Fehlerverlauf folgt sichtbar der Sägezahnstruktur des Driftsignals.

Mit den getunten Hyperparametern reduzieren alle drei adaptiven Strategien den Simulationsfehler deutlich, wobei sich eine klare Rangfolge ergibt (vgl. Tabelle 3.5). Die passive Adaption senkt den RMSE um rund ein Drittel gegenüber dem Fall ohne Adaption. Die aktive Adaption verbessert diesen Wert bei vergleichbarem Eingriffsaufwand weiter. Den geringsten Fehler erzielt die kombinierte Strategie mit einer Reduktion um knapp vier Fünftel gegenüber dem Fall ohne Adaption, bei zugleich moderatem Eingriffsaufwand. Bemerkenswert ist, dass die kombinierte Strategie diese stärkste Fehlerreduktion mit der geringsten Zahl detektor-ausgelöster Anpassungen erreicht, da die passive Komponente bereits einen Großteil der Drift abfängt.

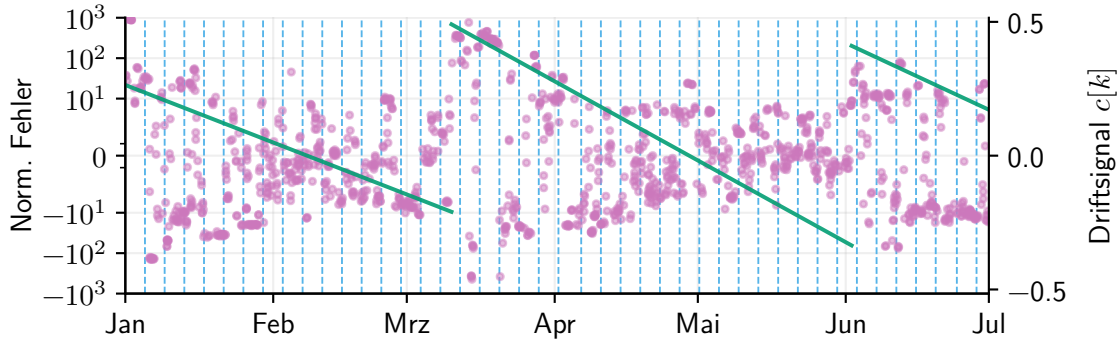
Im Gegensatz zum Fall ohne Adaption, dessen Fehler von einem gerichteten, mit dem Drift korrelierten Anteil dominiert wird, sinken durch die Adaption sowohl der Bias als auch die Streuung deutlich. Während die passive Adaption den mittleren Fehleranteil $\bar{e}$ am stärksten reduziert, erzielt die kombinierte Strategie die geringste Streuung σ und damit den insgesamt kleinsten RMSE. Entsprechend schwächt sich die Korrelation des Fehlerbetrags $|e|$ mit dem Driftbetrag $|c[k]|$ unter der kombinierten Strategie am stärksten ab. Der Restfehler folgt dem Driftsignal kaum noch, der CD wird durch die Adaption somit weitgehend kompensiert.



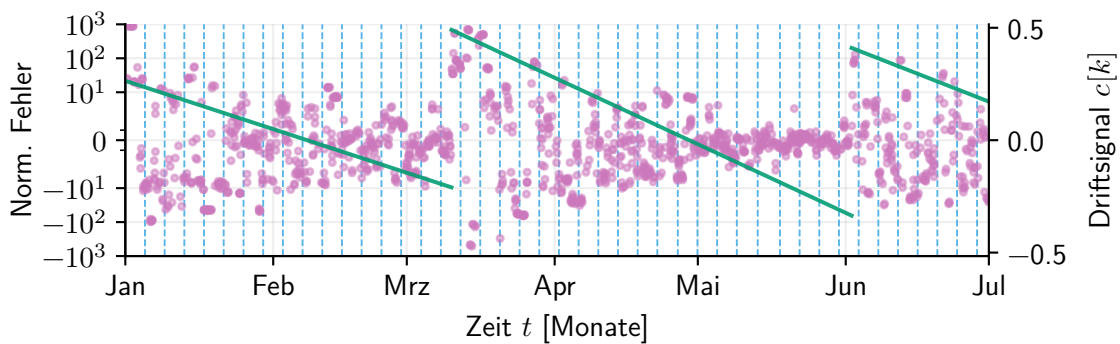
(a) Baseline (keine Adaption)



(b) Passive Adaption



(c) Aktive Adaption



(d) Kombinierte Adaption

Abbildung 3.10: Normierter Simulationsfehler der vier Adaptionstrategien auf dem drift-behafteten Datenstrom im Vergleich zum Driftsignal $c[k]$; erkannte Drifts sind in (c) und (d) markiert

Insgesamt zeigt sich, dass die Adaption den synthetisch erzeugten CD wirksam ausgleichen kann, sofern ihre Hyperparameter geeignet gewählt werden. Insbesondere die Kombination aus kontinuierlicher fehlergetriebener und detektor-getriggelter Anpassung erweist sich als robusteste Strategie, da sie den niedrigsten Gesamtfehler bei überschaubarem Eingriffsaufwand erreicht.

3.2 Der Tennessee-Eastman-Process

Beim TEP handelt es sich um eine nichtlineare Simulation einer realen industriellen Chemieanlage, die primär als Benchmark-Prozess für die Entwicklung, Untersuchung und Bewertung von Prozesssteuerungs- und Überwachungstechnologien dient. 1993 veröffentlicht von Downs und Vogel, wurde das Simulationsmodell von der Eastman Chemical Company entwickelt, um eine realistische industrielle Grundlage für die Forschung zur Erprobung von Steuerungsstrategien bereitzustellen [DV93]. Um vertrauliche Eigenschaften des ursprünglichen Verfahrens zu wahren, wurden jedoch spezifische Komponenten wie die Kinetik und die Betriebsbedingungen verändert.

Bei dem Prozess entstehen zwei flüssige Produkte G und H aus vier gasförmigen Reaktanten A , C , D und E . Zusätzlich tritt eine inerte Komponente B auf und es fällt ein Nebenprodukt F an. Das chemische System umfasst dabei zwei gleichzeitig ablaufende primäre exotherme Gas-Flüssigkeits-Reaktionen und zwei zusätzliche Nebenproduktreaktionen:



Das TEP besteht aus fünf systemischen Hauptkomponenten, die sequenziell miteinander verbunden sind und sich nach der ersten Reaktionsphase in zwei Stränge verzweigen: dem vorwärtsgerichteten Produktstrang und dem rückführenden Recyclingkreislauf. Der Prozessfluss wird im Folgenden beschrieben und kann anschaulich anhand von Abbildung 3.11 nachvollzogen werden.

▪ Schritt 1

Zunächst werden die Reaktanten A , D und E dem System zugeführt. Zusammen mit dem sich bereits im System befindlichen Reaktanten C und dem Inertgas B , das als Verunreinigung zu verstehen ist, werden die Gase in den **Reaktor** geleitet. Im Reaktor finden die vier irreversiblen und exothermen chemischen Reaktionen statt (siehe (3.17)-(3.20)). Die Reaktionen laufen an einem in der flüssigen Phase gelösten, nicht-flüchtigen Katalysator ab, wobei die Produkte G und H sowie das Nebenprodukt F entstehen. Die flüssigen Produkte verdampfen und verlassen den Reaktor zusammen mit den nicht umgesetzten Edukten, während der Katalysator im Reaktor verbleibt. Da die exothermen Reaktionen

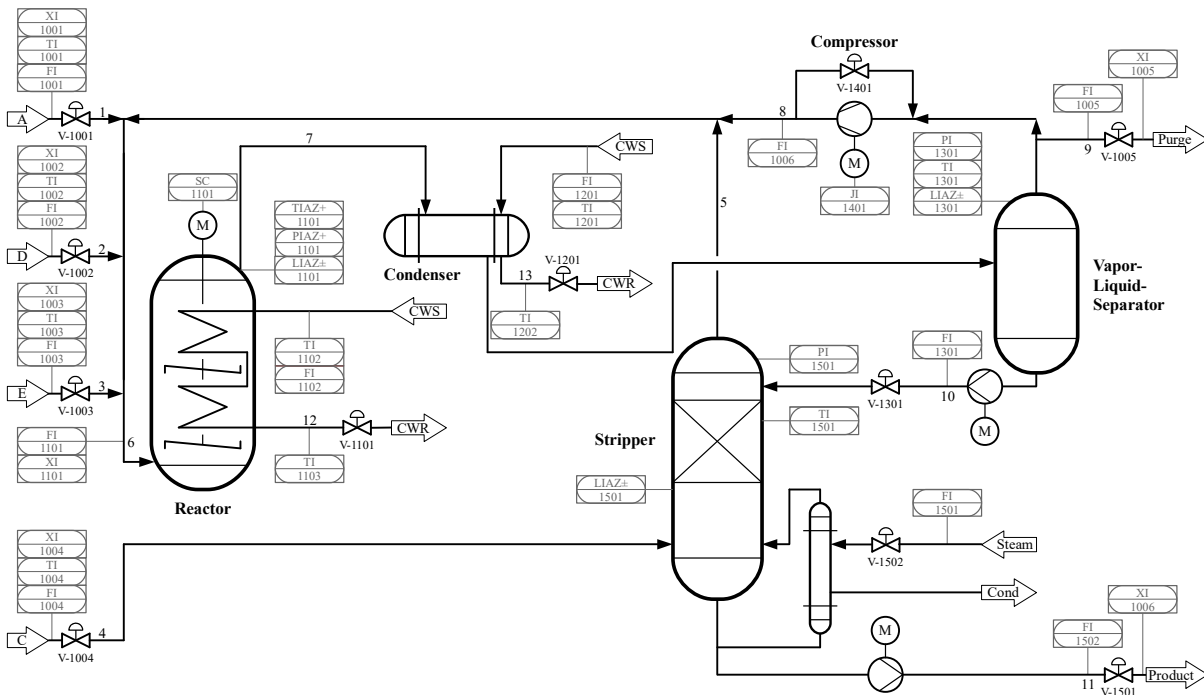


Abbildung 3.11: Anlagenschema des TEP mit seinen fünf Hauptkomponenten, in Anlehnung an [BRJ15], in Schwarz vereinheitlicht

erhebliche Wärme erzeugen, ist der Reaktor mit einem internen Kühlbündel ausgestattet, um die Reaktionswärme abzuführen und die Temperaturstabilität aufrechtzuerhalten.

▪ Schritt 2

Das resultierende Gemisch aus Dämpfen und nicht umgesetzten Gasen, wird anschließend durch den **Kondensator** geleitet und gekühlt, sodass die Produkte kondensieren.

▪ Schritt 3

Das Gemisch gelangt dann in den **Dampf-Flüssigkeits-Separator**, der die flüssigen Produkte von Gasen trennt und den Prozess so in zwei Stränge teilt: den vorwärtsgerichteten Produktstrang und den rückführenden Recyclingkreislauf.

▪ Schritt 4

Die Flüssigkeit aus dem Separator gelangt zum **Produktstripper**, wo unten das Reaktionsgas C zugeführt wird. Die Flüssigkeit wird so durch das aufsteigende Gas von Resten der verbleibenden Reaktanten gereinigt. Die Kopfgase gelangen dann über den Recyclingkreislauf wieder in den Reaktor.

▪ Recyclingkreislauf

Ein Teil der nicht kondensierten Dämpfe aus dem Separator wird abgesaugt, um zu verhindern, dass sich inerte Stoffe und Nebenprodukte im Prozess ansammeln. Die restlichen Dämpfe des Separators werden zusammen mit denen des Strippers durch den (zentrifugalen) **Kompressor** geleitet, der sie weiter im Recyclingkreislauf zurück in den Reaktor führt. Durch die Rückführung der nicht umgesetzten Stoffe wird eine Steigerung der Gesamteffizienz erreicht.

Der TEP verfügt über 41 Messgrößen und 12 Stellgrößen, wobei der Großteil der Stellgrößen zur Ventilsteuerung dient.

Es gibt sechs primäre Betriebsmodi, die durch unterschiedliche Produktmassenverhältnisse von G und H und durch Produktionsraten vorgegeben sind. Ursprünglich wurde das Modell des TEP von [DV93] als FORTRAN-Quellcode veröffentlicht. Später wurde dieser Quellcode in C-Code konvertiert, um ihn als S-Funktion in MATLAB®/Simulink® verwenden zu können. Dieser Code ist im *Tennessee Eastman Challenge Archive* zu finden, welches einige weitere Simulator-Versionen enthält, darunter Ansätze zu Model Predictive Control und dezentralen Steuerungsstrategien, die zwischen 1993 und 1996 vorgestellt wurden [Ri05]. Die aktuelle Version des C-Codes ist das Ergebnis einer Überarbeitung von Bathelt et al., die notwendig war, da die Simulations-Engine von Simulink und die ursprüngliche Struktur des Modells nicht mehr zusammenpassten [BRJ15]. Zur Simulation in Simulink existiert außerdem das Python-Paket *pyTEP*, das eine Open-Source-Schnittstelle zwischen Simulink und Python bereitstellt und so die Steuerung der Simulation über Python ermöglicht, bei der Prozesseingaben verändert und komplexe Szenarien konfiguriert werden können [RE22]. Die Daten lassen sich so unter anderem als Pandas DataFrames speichern. Dass der Prozess in dieser Form als Quellcode vorliegt, eröffnet zugleich die Möglichkeit, gezielt in die Prozessdynamik einzugreifen.

In seiner ursprünglichen Konzeption ist der TEP nicht als Grundlage für die Untersuchung von CD vorgesehen. Downs und Vogel definieren einen festen Katalog vordefinierter Störungen (IDV), die typische Fehlerfälle wie sprunghafte Änderungen der Zulaufzusammensetzung oder Ventilblockaden abbilden, einen kontinuierlichen, schleichenden CD der hier betrachteten Art jedoch nicht umfassen [DV93]. Der Datensatz wird daher über seine ursprüngliche Bestimmung hinaus genutzt und um einen drift-behafteten Betrieb erweitert. Die dafür entwickelte Injektion eines CD-Einflusses unmittelbar im C-Code ergänzt den TEP um einen in den etablierten Störungskatalogen nicht enthaltenen, schleichenden Fehlerfall.

Der so erweiterte Prozess bildet die Grundlage der Verarbeitungskette aus Generierung, Modellierung, Detektion und Adaption analog zu Abschnitt 3.1. Zunächst wird in Abschnitt 3.2.1 die Modellierung des CD im C-Code beschrieben, die den drift-behafteten Fehlerfall realisiert. Abschnitt 3.2.2 erläutert die darauf aufbauende Generierung des Datenstroms über die *pyTEP*-Schnittstelle. Der erzeugte Datenstrom wird anschließend in Abschnitt 3.2.3 modelliert, bevor Abschnitt 3.2.4 die Detektion des CD und Abschnitt 3.2.5 abschließend die Adaption des Modells an den drift-behafteten Prozess behandelt.

3.2.1 Concept Drift

Da der TEP als C-Code vorliegt, kann der CD statt über die Prozesseingänge unmittelbar in der Prozessdynamik selbst realisiert werden, wodurch, wodurch ein bislang nicht abgebildeter, schleichender Fehlerfall entsteht. Unter den definierten Störfällen kommt Fehler IDV(13)

		Mode 1	Mode 2	Mode 3	Mode 4	Mode 5	Mode 6
Products	G/H mass ratio	50/50	10/90	90/10	50/50	10/90	90/10
	Production rate G	7038 $\frac{\text{kg}}{\text{h}}$	1408 $\frac{\text{kg}}{\text{h}}$	10000 $\frac{\text{kg}}{\text{h}}$	max	max	max
	Production rate H	7038 $\frac{\text{kg}}{\text{h}}$	12669 $\frac{\text{kg}}{\text{h}}$	1111 $\frac{\text{kg}}{\text{h}}$	max	max	max
Setpoints	Production	22.89	22.73	18.04	36.04	23.55	20.20
	Stripper level	50	50	50	50	50	50
	Separator level	50	50	50	50	50	50
	Reactor level	65	65	65	65	65	65
	Reactor pressure	2800	2800	2800	2800	2800	2800
	Mol % G	53.80	11.66	90.09	53.35	11.65	90.07
	yA	63.137	64.196	62.110	61.940	64.030	61.468
	yAC	51	54.24	47.43	58.76	54.32	48.79
	Reactor temperature	122.9	124.2	121.9	128.2	124.6	123.0
	Recycle valve pos.	1	1	77.62	1	1	71.166
	Steam valve pos.	1	1	1	1	1	1
	Agitator setting	100	100	100	100	100	100

Tabelle 3.6: Process Setpoints for 6 different Modes

der Zielsetzung dieser Arbeit auf den ersten Blick sehr nahe, da er in der Literatur als *slow drift* der Reaktionskinetik bezeichnet wird [DV93]. Bei dessen Aktivierung multipliziert das Modell die Raten der beiden Primärreaktionen (3.17) und (3.18) mit den zeitvarianten Störfaktoren $r1f$ bzw. $r2f$, die durch je einen eigenen stochastischen Störprozess erzeugt werden. Jeder dieser Prozesse zieht in zufälligen Zeitabständen einen gleichverteilten Zielwert um den Nominalwert 1 (Funktion tesub5) und interpoliert zwischen den Zielwerten mit kubischen Polynomen (Funktion tesub8), sodass sich ein glatter, zufällig schwankender Verlauf ergibt. Die Störfaktoren fluktuieren strikt begrenzt und symmetrisch zwischen zufälligen Zielwerten im Intervall $[0,75; 1,25]$ um ihren Nominalwert. Bei deaktiviertem Fehler hingegen sind die Faktoren konstant 1, sodass die Reaktionskinetik unverändert bleibt.

Der Prozess besitzt damit weder einen Trend noch ein Gedächtnis über den aktuellen Interpolationsabschnitt hinaus und stellt einen stationären stochastischen Prozess mit einer Korrelationszeit von wenigen Stunden dar. Vor dem Hintergrund von Abschnitt 2.3 ist die Bezeichnung *slow drift* daher irreführend: CD setzt gemäß (2.9) eine Veränderung der gemeinsamen Verteilung $P(X, y)$ voraus, die durch IDV(13) induzierte Verteilung ist für Beobachtungsfenster oberhalb der Korrelationszeit jedoch zeitinvariant. Die Fluktuation verbreitert die Verteilung

der betroffenen Prozessgrößen einmalig bei Aktivierung, verschiebt sie anschließend aber nicht weiter, sodass nach der Systematik gemäß Tabelle 2.3 allenfalls eine abrupte Störung mit stationärem Charakter vorliegt [BAK22]. Das Attribut *slow* bezieht sich lediglich auf die Zeitskala der Fluktuation, nicht auf eine Drift im verteilungstheoretischen Sinn.

Für die Zielsetzung dieser Arbeit folgt daraus, dass der TEP in seiner ursprünglichen Form keinen Fehler bereitstellt, der diese Eigenschaften erfüllt. Um den TEP dennoch als Benchmark für die Drift-Erkennung und -Adaption nutzen zu können, wird das Simulationsmodell im Folgenden um eine Störgröße erweitert, die einen strukturierten, anhaltenden Drift der Reaktionskinetik nach dem Vorbild des synthetischen Datenstroms aus Abschnitt 3.1 erzeugt.

3.2.2 Generierung des Datenstroms

Auf Basis des drift-behafteten Prozessmodells wird über die *pyTEP*-Schnittstelle ein Datenstrom erzeugt, der als Grundlage der anschließenden Modellierung, Detektion und Adaption dient.

3.2.3 Modellierung

Der generierte Prozess wird auf dem drift-freien Referenzzeitraum modelliert und anschließend auf den gesamten Datenstrom angewandt, um die drift-bedingte Modellverschlechterung sichtbar zu machen.

3.2.4 Drift-Erkennung

Zur Detektion des im C-Code injizierten CD werden dieselben performance-basierten Detektoren wie im synthetischen Fall auf den Simulationsfehler des Modells angewandt.

3.2.5 Drift-Adaption

Als abschließender Schritt der Verarbeitungskette wird das Modell an den drift-behafteten TEP adaptiert, um der Modellverschlechterung entgegenzuwirken.

- eine *passive* Adaption
- eine *aktive* Adaption
- deren *Kombination*

4 Kapitel 4

5 Kapitel 5

Anhang

A Appendix A

Hier der erste Anhangteil

B Appendix B

Und der zweite Anhangteil

Literatur

Monografien und Artikel

- [Af15] Afkhami, B. et al.: Energy consumption assessment in a cement production plant. *Sustainable Energy Technologies and Assessments* 10, S. 84–89, 2015.
- [Ba06] Baena-Garcia, M. et al.: Early drift detection method. In: *Fourth international workshop on knowledge discovery from data streams*. Bd. 6, S. 77–86, 2006.
- [BAK22] Bayram, F.; Ahmed, B. S.; Kessler, A.: From concept drift to model degradation: An overview on performance-aware drift detectors. *Knowledge-Based Systems* 245, S. 108632, 2022.
- [BBL12] Bontempi, G.; Ben Taieb, S.; Le Borgne, Y.-A.: Machine learning strategies for time series forecasting. In: *European big data management and analytics summer school*. Springer, S. 62–77, 2012.
- [BG07] Bifet, A.; Gavalda, R.: Learning from time-changing data with adaptive windowing. In: *Proceedings of the 2007 SIAM international conference on data mining*. SIAM, S. 443–448, 2007.
- [BN06] Bishop, C.M.; Nasrabadi, N.M.: *Pattern recognition and machine learning*. Springer, 2006.
- [BRJ15] Bathelt, A.; Ricker, N. L.; Jelali, M.: Revision of the Tennessee Eastman process model. *IFAC-PapersOnLine* 48 (8), S. 309–314, 2015.
- [DR09] Dries, A.; Rückert, U.: Adaptive concept drift detection. *Statistical Analysis and Data Mining: The ASA Data Science Journal* 2 (5-6), S. 311–327, 2009.
- [DV93] Downs, J. J.; Vogel, E. F.: A plant-wide industrial process control problem. *Computers & chemical engineering* 17 (3), S. 245–255, 1993.
- [FP20] Forgione, M.; Piga, D.: Model structures and fitting criteria for system identification with neural networks. In: *2020 IEEE 14th International Conference on Application of Information and Communication Technologies (AICT)*. IEEE, S. 1–6, 2020.
- [Ga04] Gama, J. et al.: Learning with drift detection. In: *Brazilian symposium on artificial intelligence*. Springer, S. 286–295, 2004.
- [Ga14] Gama, J. et al.: A survey on concept drift adaptation. *ACM computing surveys (CSUR)* 46 (4), S. 1–37, 2014.
- [GB10] Giri, F.; Bai, E.-W.: *Block-oriented nonlinear system identification*. Springer, 2010.

- [Go16] Goodfellow, I. et al.: Deep learning. MIT press Cambridge, 2016.
- [HPC12] Hoens, T. R.; Polikar, R.; Chawla, N. V.: Learning from streaming data with concept drift and imbalance: an overview. *Progress in artificial intelligence* 1 (1), S. 89–101, 2012.
- [HS97] Hochreiter, S.; Schmidhuber, J.: Long short-term memory. *Neural computation* 9 (8), S. 1735–1780, 1997.
- [In18] International Energy Agency: Technology Roadmap: Low-Carbon Transition in the Cement Industry, Techn. Ber., Published 5 April 2018, International Energy Agency (IEA), 2018, <https://www.iea.org/reports/technology-roadmap-low-carbon-transition-in-the-cement-industry>.
- [Ka60] Kalman, R. E.: A new approach to linear filtering and prediction problems. 1960.
- [KBG04] Kifer, D.; Ben-David, S.; Gehrke, J.: Detecting change in data streams. In: *VLDB*. Bd. 4, Toronto, Canada, S. 180–191, 2004.
- [Ke11] Keesman, K. J.: System identification: an introduction. Springer Science & Business Media, 2011.
- [Ki21] Kiranyaz, S. et al.: 1D convolutional neural networks and applications: A survey. *Mechanical systems and signal processing* 151, S. 107398, 2021.
- [Le02] LeCun, Y. et al.: Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86 (11), S. 2278–2324, 2002.
- [Le08] Lee, E. A.: Cyber physical systems: Design challenges. In: 2008 11th IEEE international symposium on object and component-oriented real-time distributed computing (ISORC). IEEE, S. 363–369, 2008.
- [Lo11] Lopes, R. H. C.: Kolmogorov-Smirnov Test. In (Lovric, M., Hrsg.): *International Encyclopedia of Statistical Science*. Springer Berlin Heidelberg, Berlin, Heidelberg, S. 718–720, 2011, https://doi.org/10.1007/978-3-642-04898-2_326.
- [Lu18] Lu, J. et al.: Learning under concept drift: A review. *IEEE transactions on knowledge and data engineering* 31 (12), S. 2346–2363, 2018.
- [Ma26] Marijan, P. et al.: Exploring model complexity: A model comparison. In: *AIP Conference Proceedings*. Bd. 3381 1. 1, AIP Publishing LLC, S. 020001, 2026.
- [MBC00] McKay, M. D.; Beckman, R. J.; Conover, W. J.: A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics* 42 (1), S. 55–61, 2000.
- [Mo12] Moreno-Torres, J. G. et al.: A unifying view on dataset shift in classification. *Pattern recognition* 45 (1), S. 521–530, 2012.
- [Ne20] Nelles, O.: *Nonlinear System Identification: From Classical Approaches to Neural Networks, Fuzzy Models, and Gaussian Processes*. Springer, Cham, 2020.
- [OS10] Oppenheim, A. V.; Schafer, R. W.: *Discrete-Time Signal Processing*. Pearson, Upper Saddle River, NJ, 2010.
- [Qu08] Quiñonero-Candela, J. et al.: *Dataset shift in machine learning*. Mit Press, 2008.

- [RE22] Reinartz, C.; Enevoldsen, T. T.: pyTEP: A Python package for interactive simulations of the Tennessee Eastman process. *SoftwareX* 18, S. 101053, 2022.
- [RHS20] Raab, C.; Heusinger, M.; Schleif, F.-M.: Reactive soft prototype computing for concept drift streams. *Neurocomputing* 416, S. 340–351, 2020.
- [RHW86] Rumelhart, D. E.; Hinton, G. E.; Williams, R. J.: Learning representations by back-propagating errors. *nature* 323 (6088), S. 533–536, 1986.
- [SB21] Sousa, V.; Bogas, J. A.: Comparison of energy consumption and carbon emissions from clinker and recycled cement production. *Journal of cleaner production* 306, S. 127277, 2021.
- [Sc01] Schaefer, H.: Loesche vertical roller mills for the comminution of ores and minerals. *Minerals engineering* 14 (10), S. 1155–1160, 2001.
- [SK07] Scholz, M.; Klinkenberg, R.: Boosting classifiers for drifting concepts. *Intelligent Data Analysis* 11 (1), S. 3–28, 2007.
- [SL19] Schoukens, J.; Ljung, L.: Nonlinear system identification: A user-oriented road map. *IEEE Control Systems Magazine* 39 (6), S. 28–99, 2019.
- [Ti96] Tibshirani, R.: Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology* 58 (1), S. 267–288, 1996.
- [WB+95] Welch, G.; Bishop, G. et al.: An introduction to the Kalman filter. 1995.
- [We16] Webb, G. I. et al.: Characterizing concept drift. *Data Mining and Knowledge Discovery* 30 (4), S. 964–994, 2016.
- [Wi58] Wiener, N.: *Nonlinear Problems in Random Theory*. MIT Press, Cambridge, MA, 1958.
- [WK96] Widmer, G.; Kubat, M.: Learning in the presence of concept drift and hidden contexts. *Machine learning* 23 (1), S. 69–101, 1996.
- [Wo17] Woywadt, C.: Grinding process optimization—Featuring case studies and operating results of the modular vertical roller mill. In: 2017 IEEE-IAS/PCA Cement Industry Technical Conference. IEEE, S. 1–7, 2017.

Webseiten und Internetreferenzen

- [Ko26] Kompetenzzentrum Klimaschutz in energieintensiven Industrien (KEI): Zementindustrie - Auf dem Weg zur klimaneutralen Industrie, Abgerufen am 2. März 2026, 2026, <https://www.klimaschutz-industrie.de/themen/branchen/zementindustrie/>.
- [Ri05] Ricker, N. L.: Tennessee Eastman Challenge archive, Abgerufen am 19. März 2026, 2005, <http://depts.washington.edu/control/LARRY/TE/download.html>.

Lebenslauf

Persönliche Daten

Name: _____

Anschrift: _____

Geburtsdatum: _____

Geburtsort: _____

Staatsangehörigkeit: _____

Ausbildung

yyyy - yyyy Technische Universität _____

Studiengang: _____

Vertiefung: _____

yyyy - yyyy _____

Berufserfahrung

seit yyyy Deutsches Forschungszentrum für Künstliche Intelligenz GmbH -

FB Innovative Fabriksysteme /

Technologie-Initiative SmartFactory KL e.V., Kaiserslautern

Researcher / Teamleiter

yyyy - yyyy Technologie-Initiative SmartFactory KL e.V., Kaiserslautern

Researcher

yyyy - yyyy _____

yyyy - yyyy _____
